SVEUČILIŠTE J.J. STROSSMAYER U OSIJEKU

FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKH TEHNOLOGIJA OSIJEK

Sveučilišni studij

UGRADBENI RAČUNALNI SUSTAVI

Profesor/mentor: **prof. dr. sc., Tomislav Keser**

TEHNIČKA DOKUMENTACIJA PROJEKTA

PEUGEOT INFOTAINMENT SUSTAV

Mislav Milinković, Ivan Brajinović

Akademska godina 2024/2025

Osijek, 25. 09. 2025.

Sadržaj

1. UVOD

U današnje vrijeme *infotainment* (hrv. informacijsko-zabavni) sustavi predstavljaju ključan dio korisničkog iskustva u vozilima, jer omogućuju vozaču i putnicima pristup informacijama i multimediji na intuitivan i siguran način. Rješenja poput Android Auto, Apple CarPlay te tvorničkih rješenja renomiranih proizvođača automobila (npr. BMW iDrive, VW MIB, Tesla infotainment) postala su industrijski standard. Njihova široka rasprostranjenost proizlazi iz sposobnosti integracije funkcija poput navigacije, hands-free poziva, upravljanja glazbom i sustava vozila, čime značajno doprinose sigurnosti i udobnosti vožnje.

Ono što ta rješenja čini korisnima jest kombinacija modernog korisničkog sučelja i snažne povezivosti s vozilom putem komunikacijskih sabirnica poput CAN-a ili FlexRaya. Standardizirani protokoli i bogata dokumentacija omogućuju proizvođačima jednostavnu implementaciju i brži razvoj. Većina sustava oslanja se na dobro dokumentirane mrežne protokole, brze procesore i modularne arhitekture koje olakšavaju nadogradnju i prilagodbu softvera. Međutim, upravo zbog toga rješenja su često zatvorenog tipa i teško ih je prilagoditi specifičnim starijim modelima automobila, osobito onima koji koriste zastarjele komunikacijske standarde.

Kod starijih vozila, poput Peugeot 206, komunikacija se odvija preko VAN sabirnice koja je specifična za PSA grupu i slabije dokumentirana u odnosu na CAN. To predstavlja izazov za razvoj vlastitog infotainment sustava, jer je potrebno implementirati dekodiranje i generiranje VAN okvira kako bi se uspostavila dvosmjerna komunikacija. Na tržištu gotovo da i nema komercijalno dostupnih modula koji to podržavaju, što motivira razvoj prilagođenih rješenja.

Motiv za izradu ovog projekta leži upravo u potrebi za modernizacijom starijeg vozila i dodavanjem funkcionalnosti koje se nalaze u novijim automobilima – prikaz telemetrije u stvarnom vremenu, multimedijska reprodukcija, informiranje o statusu vozila – uz zadržavanje potpune kompatibilnosti s postojećom elektronikom. Rješenje koje je ovdje predstavljeno ima za cilj stvoriti modularni i proširivi infotainment sustav temeljen na modernom mikroupravljaču i otvorenoj softverskoj platformi, čime se korisniku omogućuje funkcionalnost na razini novijih vozila, ali po pristupačnoj cijeni i uz potpunu kontrolu nad sustavom.

1.1. Zadatak i struktura rada

Zadatak ovog projekta bio je razviti i implementirati infotainment sustav za vozilo Peugeot 206, s naglaskom na kompatibilnost s postojećom elektroničkom infrastrukturom vozila. Glavni izazov je komunikacija putem VAN sabirnice, koja je specifična za PSA grupu i slabo dokumentirana, što zahtijeva izradu vlastitog hardverskog i softverskog rješenja za dekodiranje i generiranje poruka.

Cilj projekta je izraditi potpuno funkcionalan prototip koji omogućuje:

- prikaz telemetrijskih podataka vozila u stvarnom vremenu (brzina, okretaji motora, temperatura, status goriva),
- integraciju multimedijskih funkcija poput prikaza radijskih stanica i reprodukcije sadržaja s vanjskih uređaja,
- stabilan i energetski učinkovit rad koji ne opterećuje akumulator vozila,
- modularnost i mogućnost budućeg proširenja funkcionalnosti.

Očekivani rezultat projekta je pouzdan, proširiv i cjenovno pristupačan infotainment sustav temeljen na modernim mikroupravljačima i otvorenim softverskim rješenjima, koji omogućuje starijem vozilu funkcionalnosti usporedive s novijim modelima.

1.1.1. Struktura rada

Poglavlje 1 – UVOD Daje pregled motivacije za projekt, opisuje specifičnosti Peugeot 206 vozila i problematiku komunikacije putem VAN sabirnice. Objašnjava zašto je potrebno izraditi vlastiti modul i što se planira postići projektom.

Poglavlje 2 – PEUGEOT INFOTAINMENT SUSTAV Objašnjava teorijsku osnovu projekta. Detaljno opisuje principe rada VAN sabirnice, e-Manchester kodiranje, strukturu poruka, te način na koji su moduli u vozilu međusobno povezani. Na kraju se daje prijedlog sklopovskog i programskog rješenja.

Poglavlje 3 – REALIZACIJA PEUGEOT INFOTAINMENT SUSTAVA Sadrži konkretan opis realizacije: korištene komponente, razvoj tiskane pločice, sheme i logiku povezivanja. Objašnjava

kako je izvedena programska podrška – od primanja i slanja VAN okvira do upravljanja napajanjem i događajima.

Poglavlje 4 – TESTIRANJE I REZULTATI Prikazuje metodologiju testiranja sustava u stvarnim uvjetima (u vozilu) te rezultate testova. Opisuje ispravnost prikaza telemetrije, rad multimedije i potvrđuje da sustav radi u skladu s očekivanjima.

Poglavlje 5 – ZAKLJUČAK Sumira naučene lekcije, ističe glavne prednosti i izazove projekta te daje prijedloge za buduća poboljšanja (npr. integracija s tvorničkim zaslonom, dodatna optimizacija potrošnje energije).

Literatura Navodi korištene izvore, datasheetove, tehničku dokumentaciju i prethodne radove na kojima se projekt temelji.

2. PEUGEOT INFOTAINMENT SUSTAV

2.1. Principi i fizikalne zakonitosti

VAN (eng. *Vehicle Area Network*) je serijska komunikacijska sabirnica koju su razvili PSA grupa (Peugeot, Citroën) i Renault kao alternativu CAN sabirnici. Definirana je u ISO standardu **ISO 11519-3**. Koristi isti fizički sloj kao CAN sabirnica (diferencijalni par) i identičnu metodu arbitraže na sabirnici, ali se razlikuje u načinu pakiranja podataka. Gotovo sve informacije navedene preuzete su iz Atmel TSS463C dokumentacije [1]**.**

Kao i CAN, VAN protokol koristi identifikatore za razlikovanje poruka i izvođenje arbitraže na sabirnici. Dodatno, omogućuje tzv. „odgovore unutar okvira" (eng. *In-frame*), veće veličine poruka (28 bajtova umjesto 8 bajtova kao u CAN-u) te mogućnost izravne komunikacije s drugim uređajem.

Podaci na VAN mreži pakiraju se u okvire. Okviri se sastoje od više dijelova: početka okvira (**SOF**), identifikatora poruke (**IDEN**), naredbe (**COM**), samih podataka (**DATA**), kontrolnog zbroja okvira (**FCS**), signala kraja podataka (**EOD**), signala potvrde (**ACK**) te signala kraja okvira (**EOF**).

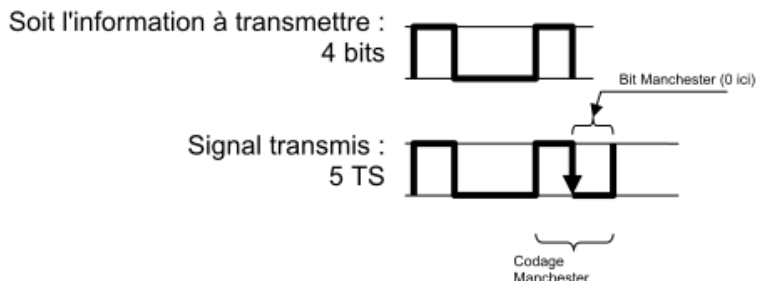
2.1.1. e-Manchester kodiranje

Svi VAN okviri kodirani s pomoću unaprijeđenog Manchesterovog kodiranja (eng. *Enhanced Manchester, e-Manchester*). Kod e-Manchestera, nakon slanja tri NRZ bita, četvrti bit šalje se kao Manchesterov bit. Slanje podataka na ovaj način omogućuje lakšu sinkronizaciju na sabirnici. Ovo je predvidljivija metoda ubacivanja bitova (eng. *bit-stuffing*) u usporedbi s metodom korištenom na CAN sabirnici.

Tablica 2.1. *Oblik VAN okvira*

SOF 10 TS	IDEN 12 bit 15 TS	Command (4 bit, 5 TS)				DATA max. 28 byte 280 TS	FCS 15 bit 18 TS	EOD 2 TS	ACK 2 TS	EOF 4 TS
		EXT	RAK	R/W	RTR					

U tablici 2.1. prikazan je broj bitova za svaki dio okvira, kao i broj vremenskih intervala (eng. *Time Slots*, TS). Budući da Manchester bit zahtjeva dva vremenska intervala za prijenos, četiri bita u nizu kodirana e-Manchester metodom prenose se u 3 + 2 vremenska intervala (slika 2.1.).



Slika 2.1. *NRZ i Manchester bitovi [1]*

2.1.2. Poruke

Uređaj ili modul povezan na mrežu može biti jednog od tri tipa:

- Autonomni modul - glavni uređaj sabirnice. Može slati **SOF** niz, započeti prijenos podataka i primati poruke.
- Modul sa sinkronim pristupom - ne može slati **SOF** niz, ali može započeti prijenos podataka i primati poruke.
- Podređeni modul (eng. *Slave*) - može slati poruke samo s pomoću odgovora unutar okvira (eng. *in-frame*) i također može primati poruke.

VAN protokol podržava više tipova poruka. Razlikuju se s pomoću polja identifikatora (**IDEN**) i naredbe (**COM**). Identifikator je dug 12 bita i slijedi ga 4 bita naredbe. Zbog načina na koji funkcionira arbitraža na sabirnici, manji identifikator ima prednost na sabirnici. Dok je vrijednost identifikatora potpuno proizvoljna, bitovi naredbe su strogo definirani i određuju tip poruke koja će se poslati na sabirnici:

- EXT - uvijek 1
- RAK - '*Request Acknowledge*' (zahtjev za potvrdom). Ako je 1, primatelj mora potvrditi primitak poruke. Ako je 0, signal ACK se ne šalje.
- R/W - '*Read/Write*' (čitanje/pisanje). Označava smjer podataka. Ako je 0, radi se o poruci za pisanje ('*write*'), gdje uređaj šalje podatke namijenjene drugom uređaju. Ako je 1, radi se o poruci za čitanje („*read*"), što podrazumijeva zahtjev za podacima i očekuje odgovor.

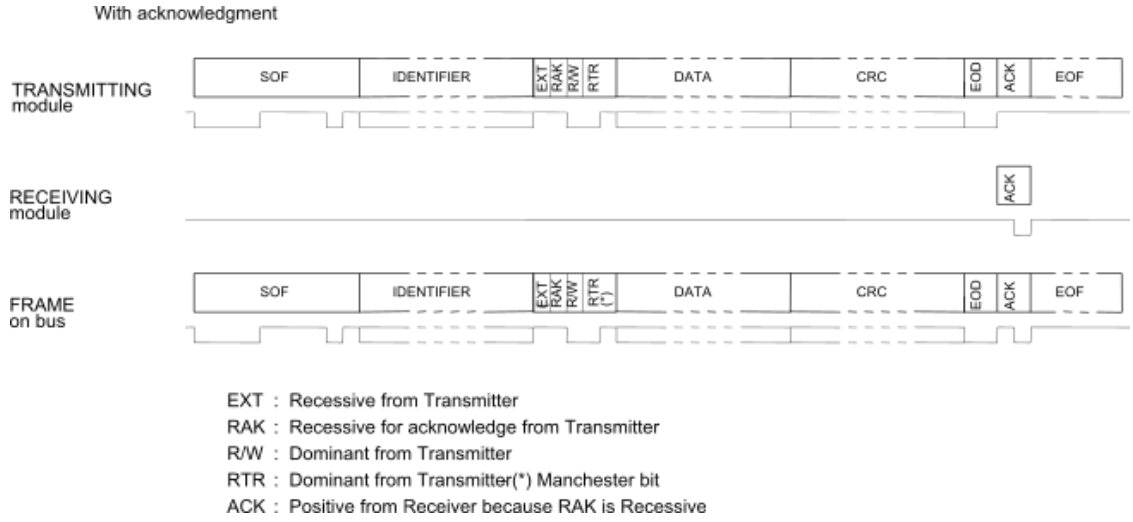
- RTR - '*Remote Transmission Request*' (zahtjev za daljinskim prijenosom). Ako je 0, poruka sadrži podatke. Ako je 1, poruka ne sadrži podatke i implicira zahtjev za odgovor unutar okvira (in-frame response). Kombinacija R/W = 0 i RTR = 1 je zabranjena na sabirnici.

2.1.3. Vrste poruka

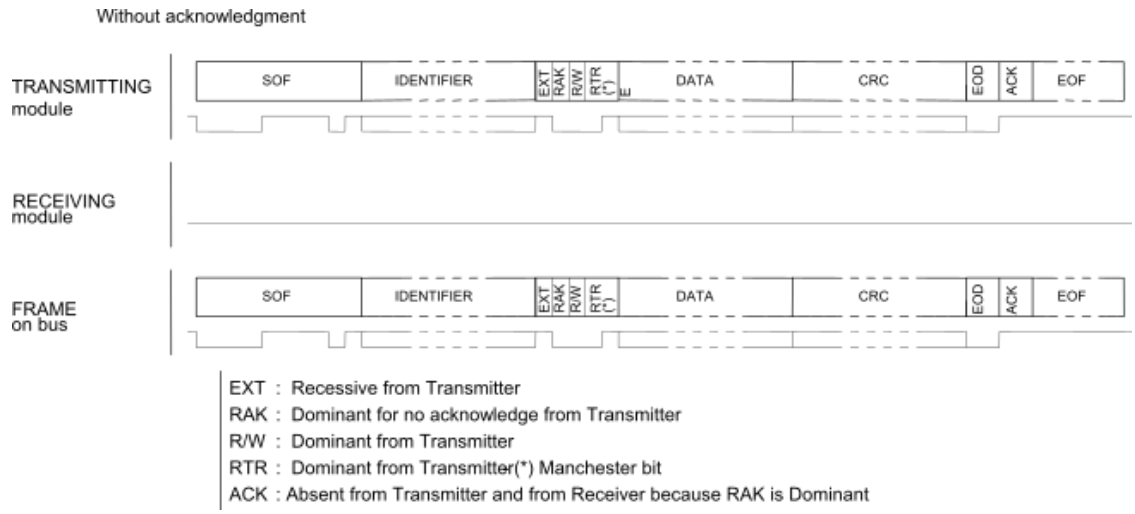
Ovdje će biti navedeni tipovi okvira relevantni za ovaj projekt. Postoje i drugi, ali koriste samo ovi.

Podsjetnik: Dominantno = 0, Recesivno = 1.

Normalni okvir Najjednostavniji tip okvira. Jedan modul šalje cijeli okvir s podacima, te može, ali ne mora, zatražiti potvrdu (ACK). U slučaju da se potvrda ne traži, poruka se tretira kao poruka za emitiranje (eng. *broadcast*). Slike 2.4. i 2.5. prikazuju prijenos normalnog okvira u dvije izvedbe, s potvrdom i bez.

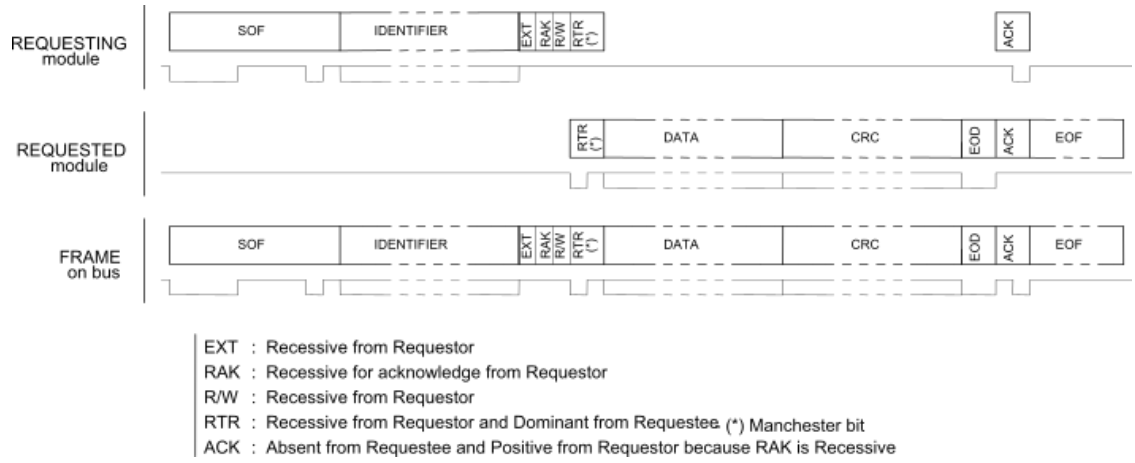


Slika 2.2. *Prijenos normalnog okvira s ACK. [1]*



Slika 2.3. *Prijenos normalnog okvira bez ACK. [1]*

Zahtjev s trenutačnim odgovorom Ovo je jedini okvir u kojem podređeni modul (eng. *slave*) može slati podatke. Jedina razlika na sabirnici je promjena stanja bita R/W u odnosu na normalni okvir. Glavni modul sabirnice (eng. *bus master*) generira **SOF**, inicijator generira identifikator i prva tri bita naredbe, te signal ACK. Ostalo (RTR, podaci i kontrolnu sumu) generira modul koji odgovara. Slika 2.6. prikazuje prijenos zahtjeva s trenutačnim odgovorom.



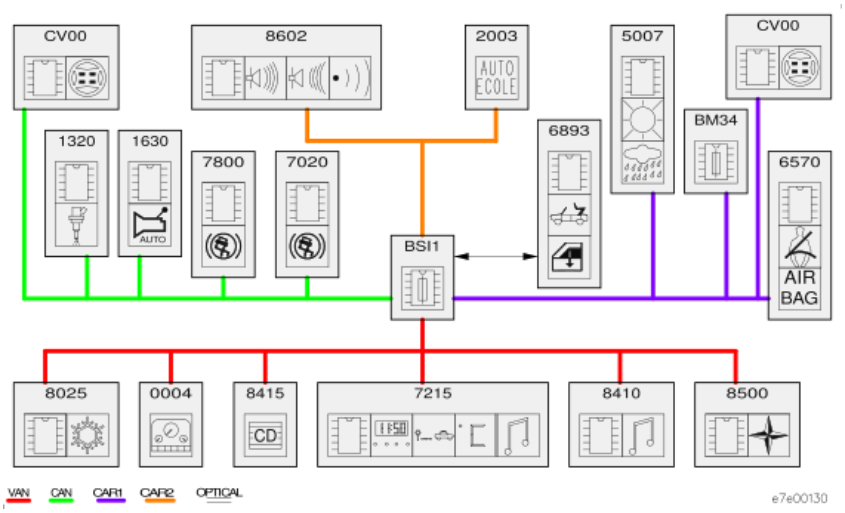
Slika 2.4. *Ilustracija prijenosa zahtjeva s trenutačnim odgovorom [1]*

2.1.4. VAN mreža

Više modula se mogu povezati zajedno kako bi se stvorila mreža. Također je moguće povezati više mreža i koristiti pristupni modul (eng. *gateway*) za usmjeravanje podataka između mreža. U vozilu

Peugeot 206 postoji jedna CAN mreža *CAN Engine* i tri VAN mreže: *VAN Comfort*, *VAN Body 1* i *VAN Body 2*.

Raspored mreža prikazan je na slici 2.7. Tamo također se može vidjeti pristupni modul, BSI. Ovaj modul prisutan je u svim vozilima PSA grupe. Konkretni BSI dolazi iz generacije AEE2001, jedine generacije koja koristi VAN sabirnicu. Druge generacije, AEE2004 i AEE2010, koriste isključivo CAN. Ovaj infotainment sustav bit će spojen umjesto CD mjenjača, modul broj 8415 na slici 2.7. na *VAN Comfort* sabirnici.



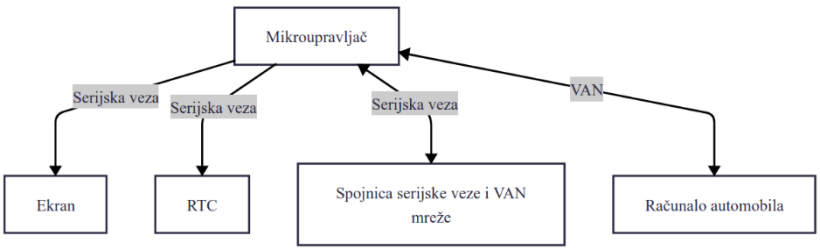
Slika 2.5. *Pregled VAN i CAN mreže u Peugeot 206 automobilu [3]*

Na slici 2.8. je prikazan blok dijagram idejnog rješenja iz kojeg se mogu prepoznati tri dijela, kao i dio koji je centralni za ovaj projekt. Mikroupravljač je zadužen i za komunikaciju s računalom automobila, obradu dobivenih informacija te za njihov prikaz na ekranu.

Slika 2.6. *Blok dijagram idejnog rješenja.*

2.2. Pregled sklopovskog rješenja

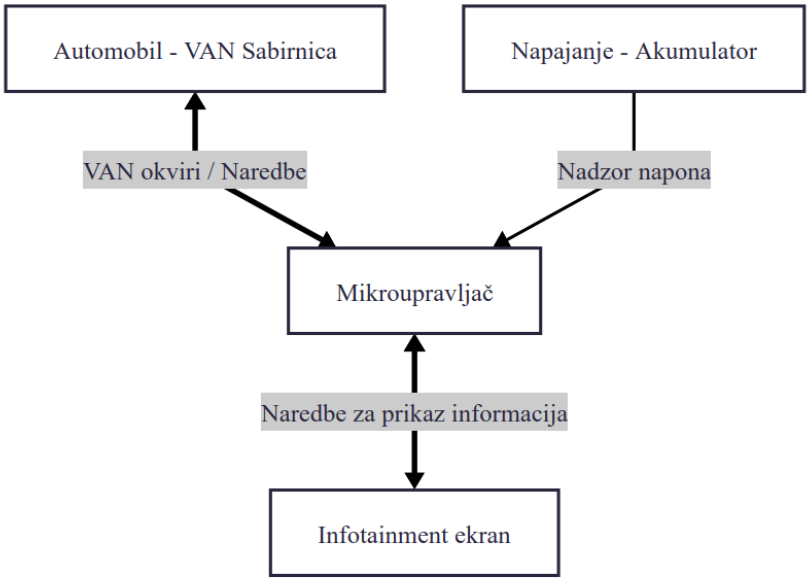
Na slici 2.9. se nalazi prikaz osnovnog sklopovskog dijagrama sustava. Glavni dio sustava je mikroupravljač koji ima ulogu komunikacije informacija s računalom automobila te obradu i prikaz tih istih informacija na ekranu. Kako mikroupravljač ne podržava VAN protokol, potrebno je koristiti upravljačke integrirane krugove koji pružaju sučelje s protokolom koji je podržan od strane mikroupravljača. Za slučaj da se ne pronađe upravljač koji ne podržava diferencijalni VAN protokol, bit će potrebno koristiti tzv. VAN Transceiver (hrv. Primopredajnik) kako bi preveli TTL logiku u diferencijalni par. Također zbog dodatne funkcionalnosti prikaza vremena na ekranu, potrebno je dodati RTC integrirani krug čija je zadaća praćenje stvarnog vremena te komunikacija istog s mikroupravljačem.



Slika 2.7. *Osnovni blok dijagram sklopovskog rješenja*

2.3. Prijedlog programskog rješenja

Na slici 2.10. je prikazan blok dijagram programskog rješenja koji daje apstraktan uvid u moguće programsko rješenje. Glavni program bi se vrtio na mikroupravljaču je odgovaran za slanje i primanje paketa na VAN mreži tj. odgovaran je za komunikaciju s računalom automobila. Nadalje, pored toga je potrebno riješiti problem upravljanja potrošnjom energije i uvjetnog paljenja sustava tj. postavljanjem dijelova sustava u aktivno stanje ili stanje mirovanja. I na kraju je potrebno dobivene informacije obraditi te prikazati na ekranu.



Slika 2.8. *Osnovni blok dijagram programskog rješenja*

3. REALIZACIJA PEUGEOT INFOTAINMENT SUSTAVA

3.1. Korištene komponente, alati i programska podrška

ESP32 - WROOM32E glavni mikroupravljač sustava. Komunicira s računalom automobila, obrađuje zaprimljene informacije te ih prosljeđuje RPi mikroupravljaču.

RPi 4 prihvaća i obrađuje dobivene informacije od ESP mikroupravljača te ih prikazuje na ekranu.

RTC DS3231MZ broji trenutačno vrijeme koje se komunicira s RPi mikroupravljačem u svrhu prikaza istog na ekranu.

TSS463C integrirani strujni krug koji omogućuje jednostavno komuniciranje s VAN mrežom preko SPI sučelja. Apstrahira sam VAN protokol koji je kompleksan te ubrzava razvoj.

MCP2551 je integrirani strujni krug koji pretvara naponske razine logičkih razina VAN sabirnice u diferencijalni signal.

NTS0104 je integrirani strujni krug koji prevodi naponske razine signala.

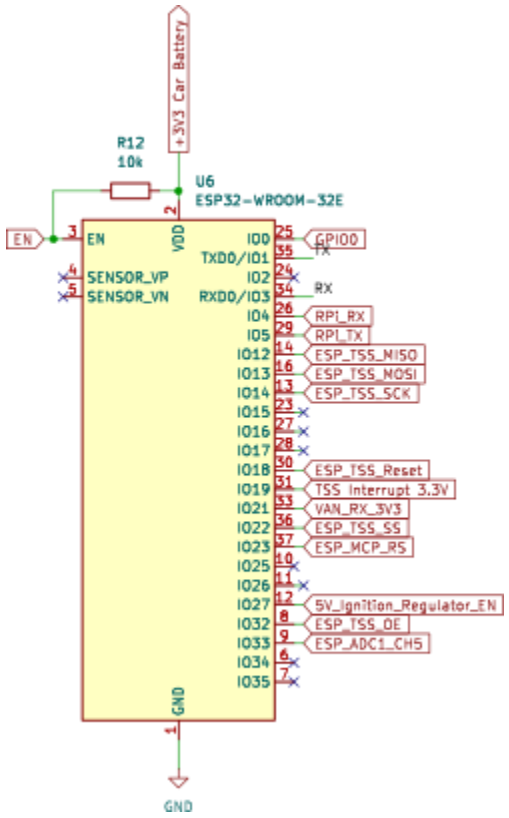
MP8759GD1 je integrirani strujni krug koji služi kao regulator napona. U ovom konkretnom slučaju se koristi za pretvorbu napona od 12 V u 5 V.

NCP691 je integrirani strujni krug koji služi kao regulator napona za 3.3 V.

ESP-IDF programski paket razvijen od Espressif-a se koristi za programiranje za ESP mikroupravljača.

3.2. Realizacija konstrukcijskog i sklopovskog rješenja

Napaja se s 3.3 V sa stalnog izvora napajanja (shema se nalazi na slici 3.12.), EN nožica je spojena preko otpornika na napon napajanja (eng. *pull-up*) kako bi se omogućio rad mikroupravljača. Shema mikroupravljača je prikazana na slici 3.1..



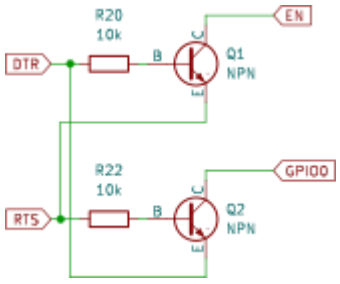
Slika 3.1. *Pregled sheme - ESP32 mikroupravljač*

Prije samog procesa prijenosa koda na ESP, isti se mora postaviti u određeni način rada gdje je spreman prihvatiti programski kod. Zbog toga se koristi gumb SW2A koji kad je pritisnut dovodi nisku naponsku razinu na GPIO0 što je uvjet kod ponovnog pokretanja mikroupravljača za odlazak u spomenuti način rada. Zbog lakšeg ponovnog pokretanja, implementiran je još jedan gumb za ponovno pokretanje koji, kad se pritisne, dovede nisku naponsku razinu na EN nožicu. Shema gumbova se nalazi na slici 3.2..

Slika 3.2. *Pregled sheme - Gumbovi za programiranje*

Dugmad na slici 3.2. se koriste za ručno tj. fizičko programiranje. Međutim, kako bi omogućili programsko programiranje tj. omogućili programiranje mikroupravljača bez fizičkog pritiskanja

gumba, implementirana su dva tranzistora koji pružaju programsko upravljanje naponskih razina nožica mikroupravljača. Shema tih tranzistora je prikazana na slici 3.3..

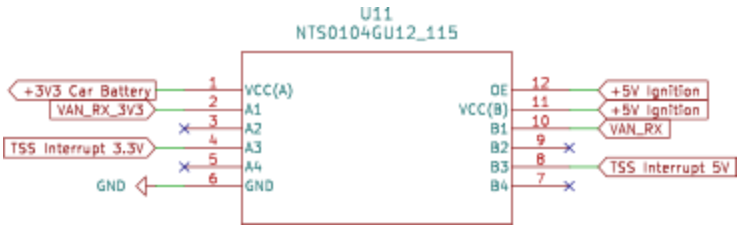


Slika 3.3. *Pregled sheme - Tranzistori za programiranje.*

U svrhe programiranja je kreirano posebno sučelje koje koristi tranzistore za programiranje koji su prethodno spomenuti kako bi odabrali način rada mikroupravljača kao i UART serijski protokol za sam prijenos programskog koda. Shema sučelja je prikazana na slici 3.4..

Slika 3.4. *Pregled sheme - spojnica za programiranje.*

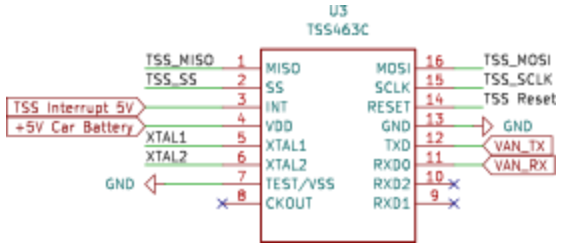
Kako imamo komponente koje rade na različitim naponskim razinama, potrebno je upotrijebiti pretvarač naponskih razina kako bi se pouzdano i bez razmišljanja o pregaranju, razmjenjivale informacije između njih. Pretvarač naponskih razina na slici 3.5. vrši pretvorbu na signalima prekida TSS integriranog kruga te na liniji dolaznih podataka VAN sabirnice.



Slika 3.5. *Pregled sheme - pretvarač naponskih razina A*

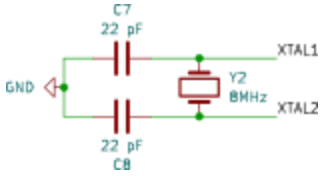
TSS463C je VAN upravljački integrirani krug koji se koristi za komunikaciju sa sustavom automobila. Komunicira s ESP mikroupravljačem putem SPI serijskog protokola. Od periferije

sadržava 8 MHz oscilator koji je neophodan za rad integriranog kruga. Sa sustavom automobila komunicira preko VAN protokola (nožice 11 i 12). Shema spoja je prikazana na slici 3.6..



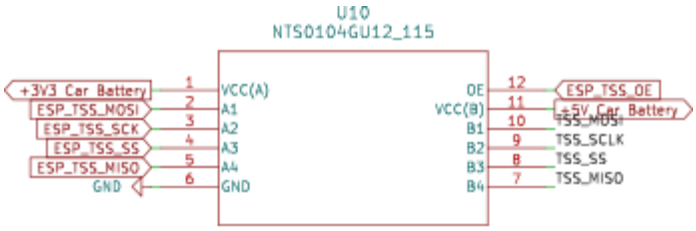
Slika 3.6. *Pregled sheme - TSS*

Slika 3.7. pokazuje oscilator koji je spojen na TSS463C integrirani krug.



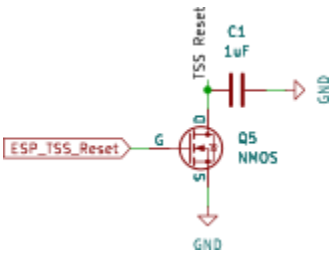
Slika 3.7. *Pregled sheme - TSS oscilator*

Kako TSS ne može direktno komunicirati s mikroupravljačem, između njih je postavljen pretvarač naponskih razina kako bi preveo 5 V signale od TSS-a u 3.3 V signale koje ESP očekuje. Shema spoja je prikazana na slici 3.8..



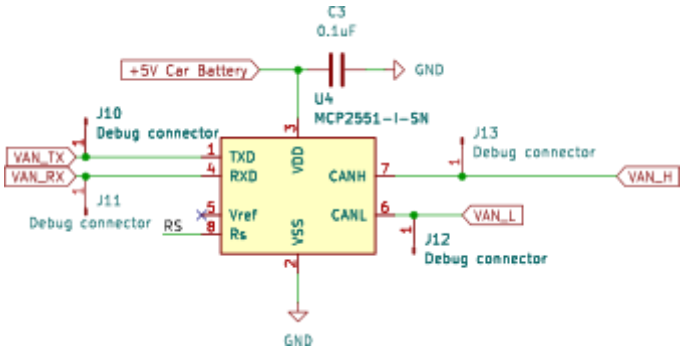
Slika 3.8. *Pregled sheme - Pretvarač naponskih razina B*

Kao mehanizam za ponovno pokretanje TSS integriranog kruga preko ESP mikroupravljača, ugrađen je N kanalni unipolarni tranzistor koji je spojen na nožicu za ponovno pokretanje TSS integriranog kruga. Kako je ta nožica aktivna na nisku naponsku razinu, kad je tranzistor u zasićenju, nožica se spaja na uzemljenje te se tako TSS integrirani krug ponovno pokreće. Shema koja prikazuje spoj ovog tranzistora se nalazi na slici 3.9..



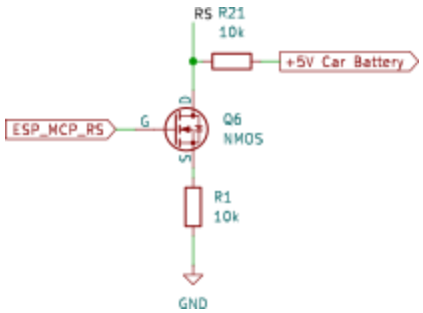
Slika 3.9. *Pregled sheme - Tranzistor za ponovno pokretanje TSS-a.*

Integrirani krug koji prevodi TTL signale u diferencijalni par se nalazi na slici 3.10..



Slika 3.10. *Pregled sheme - MCP*

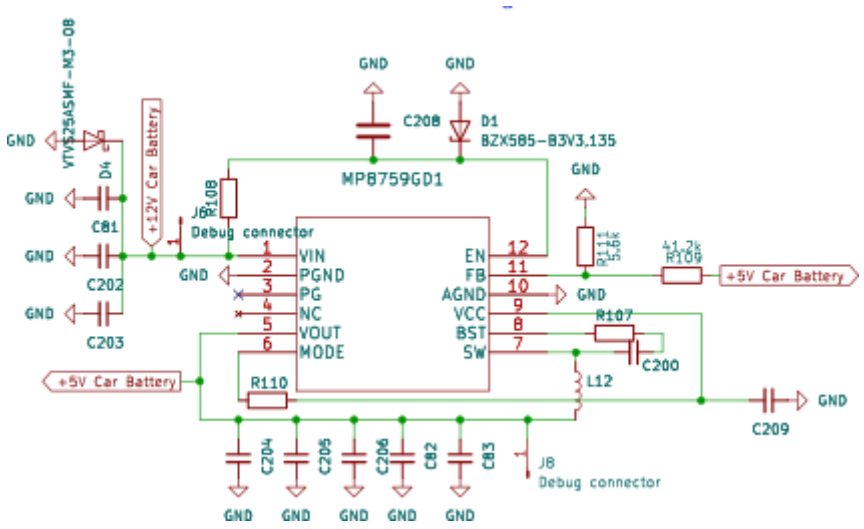
Kao mehanizam ponovnog pokretanja MCP integriranog kruga preko ESP mikroupravljača, implementiran je n kanalni unipolarni tranzistor koji je spojen na nožicu za ponovno pokretanje MCP integriranog kruga, slika 3.11.. Kako je ta nožica aktivna na nisku naponsku razinu, kad je tranzistor u stanju zasićenja, nožica se spaja na pull-down otpornik te se tako MCP integrirani krug ponovno pokreće.



Slika 3.11. *Pregled sheme - Tranzistor za ponovno pokretanje integriranog kruga MCP2551*

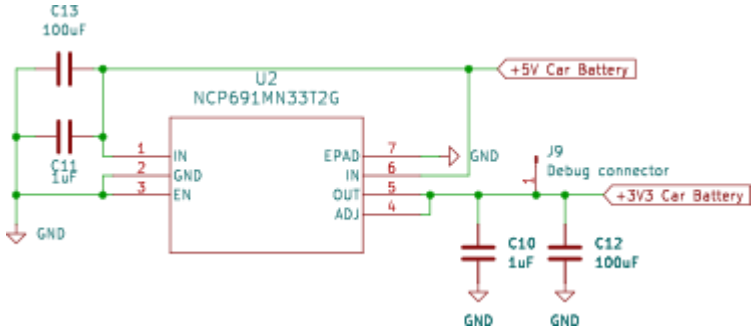
Strujni krug je preuzet iz tehničkog lista [4] integriranog kruga MP8759GD1, na slici 11.Na strujnom krugu se nalazi jedna TVS dioda kako bi osigurala ulaznu nožicu od previsokog napona kao i razne

vrijednosti kondenzatora čija je uloga filtriranje i peglanje oblika ulaznog napona. Na izlazu se nalaze isto tako kondenzatori za filtriranje i peglanje oblika izlaznog napona. Kako je riječ o regulatoru napona koji nije ograničen na jednu vrijednost izlaznog napona, potrebno je odabrati željeni izlazni napon. A to je nešto što se radi na nožici 7 običnog naponskog dijelila R111 i R109. Tablica s parovima vrijednosti za željene vrijednosti izlaznog napona se nalaze u tehničkom listu [4] na stranici 16. Shema regulatora napona je prikazana na slici 3.12..



Slika 3.12. *Pregled sheme - Regulator napona stalnih 5 V*

Linearni regulator napona koji kao ulaz prima 5 V i na izlazu daje 3.3 V je prikazan na slici 3.13.. Posjeduje par kondenzatora na ulazu i izlazu u svrhu filtriranja i peglanja napona.



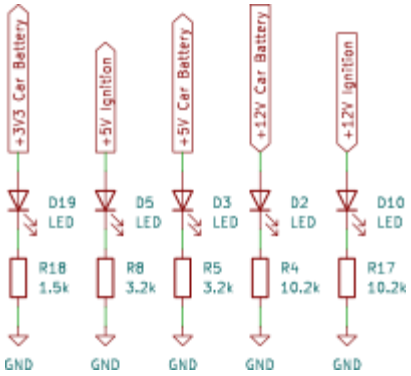
Slika 3.13. *Pregled sheme - 3.3 V regulator napona*

Kako bi se mjerila razina napona baterije automobila, korišten je obični djelitelj napona realiziran s pomoću dva otpornika čiji je spoj prikazan na slici 3.14.. Kako bi se što više ograničila struja koja

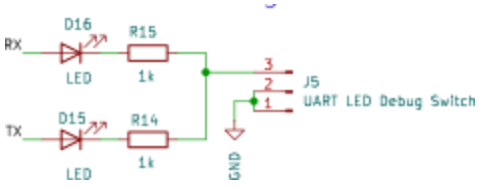
ovim djeliteljem teče odabrane su vrijednosti otpornika u rasponu od stotina tisuća ohma. Omjer otpora koji je korišten je 4:1 tj. na nožici ESP-a će biti 1/5 napona baterije automobila.

Slika 3.14. *Pregled sheme - Djelitelj napona baterije*

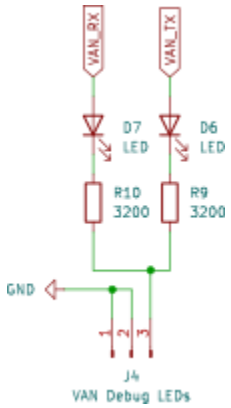
LE diode za detekciju grešaka su postavljene na mnogim bitnim mjestima kako bi ubrzale postupak razvoja. Svaka LED posjeduje otpornik koji ograničava struju koja prolazi kroz LED na 1 mA te se na njemu dešava preostali pad napona. Shema spoja navedenih LE dioda je prikazana na slikama 3.15. , 3.16., 3.17. i 3.18..



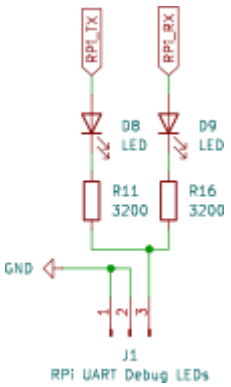
Slika 3.15. *Pregled sheme - LE diode za detekciju i otklanjanje grešaka*



Slika 3.16. *Pregled sheme - LE diode za detekciju i otklanjanje grešaka na UART sabirnici*

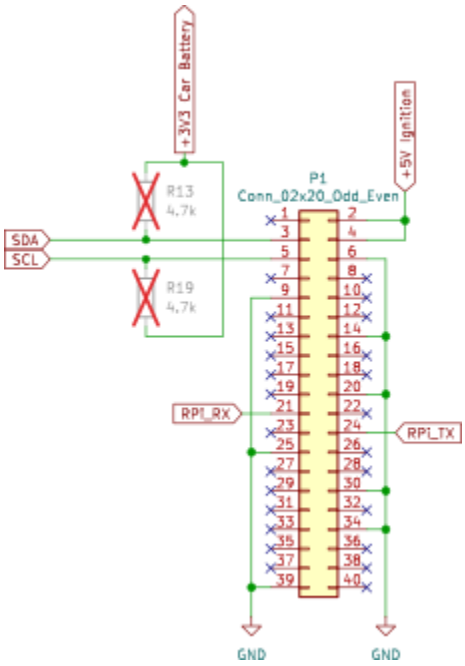


Slika 3.17. *Pregled sheme - LE diode za detekciju i otklanjanje grešaka na VAN sabirnici*



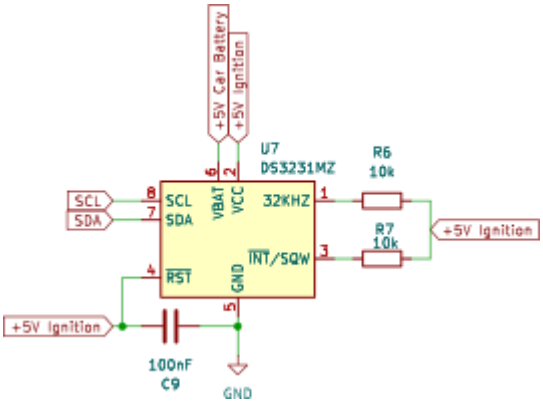
Slika 3.18. *Pregled sheme - Shema spoja LE dioda za detekciju i otklanjanje žohara na UART sabirnici koja povezuje RPi i ESP32*

Spojnica s 40 nožica karakteristična za RPi liniju mikroupravljača je prikazana na slici 3.19.. Preko ove spojnice se UART sabirnicom povezuju ESP i RPi. I2C sabirnica se koristi za komunikaciju RPi mikroupravljača s RTC modulom te LCD ekranom. Kao opcija su ucrtane ali ne postavljene komponente otpornika R13 i R14.



Slika 3.19. *Pregled sheme - Spojnica za RPi mikroupravljač*

Integrirani krug DS3231MZ obnaša ulogu RTC-a a njegov spoj je prikazan na slici 3.20.. Spoj je odrađen prema naputcima iz tehničkog lista [5]. Nožice 1 i 3 se spajaju preko otpornika na napon napajanja kako se ne koriste, nožice 7 i 8 su spojene na I2C sabirnicu RPi mikroupravljača. Postoji stalan izvor napajanja te drugi koji je aktivan kad se auto probudi.



Slika 3.20. *Pregled sheme - Shema spoja RTC modula za RPi mikroupravljač*

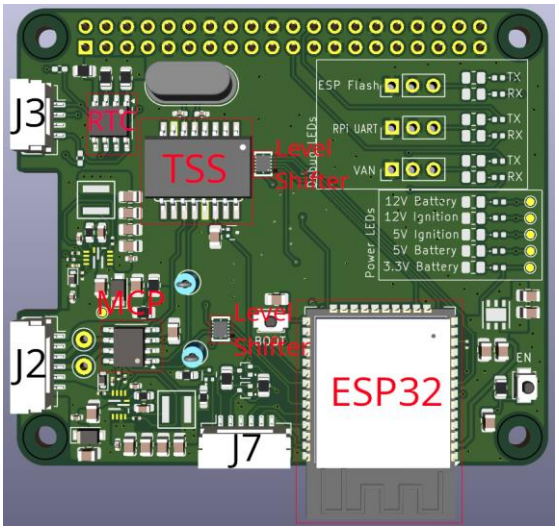
Na slici 3.21. je prikazana JST spojnica za LCD ekran kojeg upogoni RPi. Nudi pristupne točke na I2C sabirnicu kao i izvor napajanja od 5 V.

Slika 3.21. *Pregled sheme - Spojnica za LCD ekran*

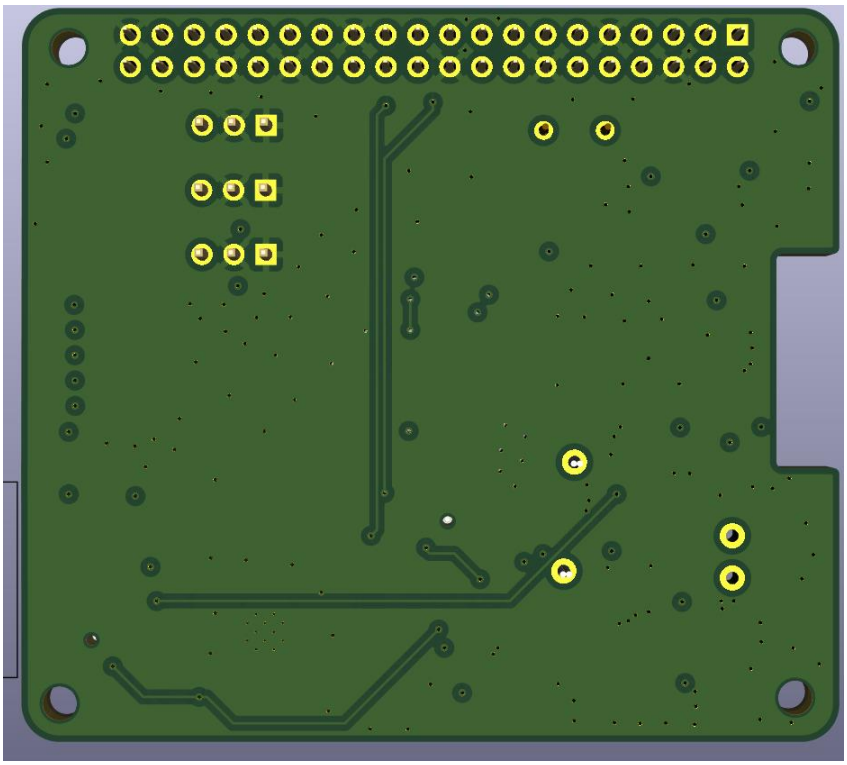
Na slici 3.22. je JST spojnica koja omogućava spajanje na VAN mrežu automobila.

Slika 3.22. *Pregled sheme - Spojnica za VAN mrežu automobila*

Na slikama 3.23. i 3.24. se nalazi izgled printane tiskane pločice koja predstavlja sklopovski dio projekta. Sve komponente se nalaze s iste strane kako bi se olakšao i ubrzao proces lemljenja. Radi lakšeg snalaženja je svaka značajnija komponenta naznačena na slici. Oznake se manifestiraju bojama kao i pratećim tekstom. Spojnica J3 označava mjesto gdje se spaja LCD ekran, J2 označava mjesto povezivanja projekta s mrežom auta te J7 označava mjesto s kojeg je moguće programirati ESP32 mikroupravljač.



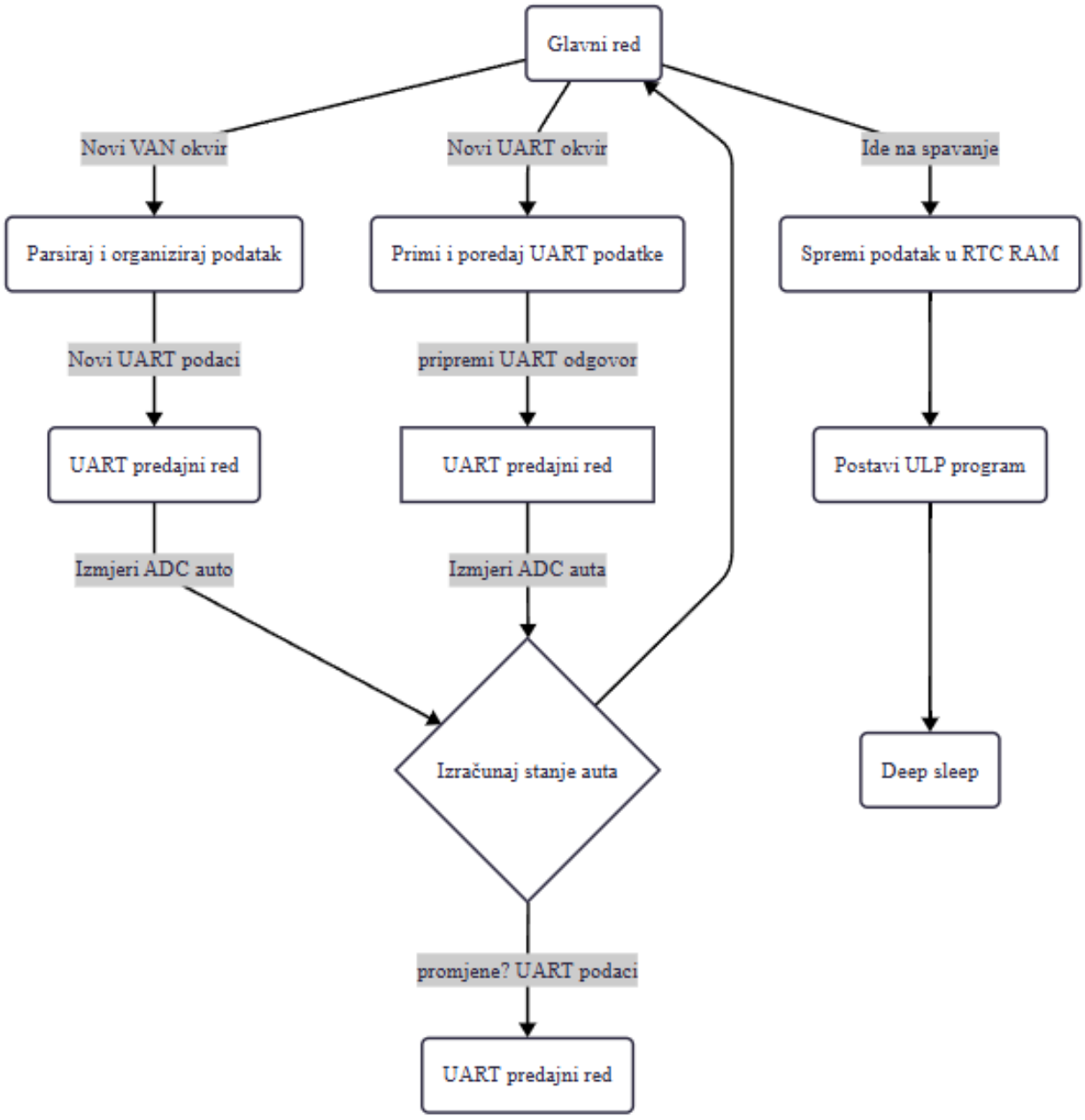
Slika 3.23. *Izgled prototipa - PCB prednja strana*



Slika 3.24. *Izgled prototipa - PCB zadnja strana.*

3.3. Realizacija programskog rješenja

Pristupni modul je najvažniji dio ovog projekta jer se sučeljava s vozilom i odgovoran je za upravljanje napajanjem cijelog sustava. Modul prima i analizira VAN okvire, organizira podatke i zatim ih prosljeđuje infotainment računalu. Također obavlja i obrnuti proces: može primati podatke s infotainment računala i prenijeti odgovarajući VAN okvir na sabirnicu. upravljanje napajanjem također je važan dio programske podrške, budući da prekomjerna potrošnja u stanju mirovanja brzo iscrpljuje akumulator vozila. Kako je programsko rješenje vrlo kompleksno, blok dijagram je razlomljen na četiri slike od kojih svaka opisuje jedan dio sustava. Na slici 3.25. je prikazan blok dijagram glavnog zadatka koji prikazuje programski tok rješenja iznad opisanog programa.



Slika 3.25. *Blok dijagram programskog toka – Glavni zadatak*

3.3.1. Okvir i pristup

Kao program za razvoj koristi se ESP-IDF programski okvir koji pruža Espressif. Okvir se temelji na operativnom sustavu FreeRTOS. Zbog prirode OS-a i programskog okruženja, programska podrška modula napisana je koristeći pristup temeljen na događajima.

Koristimo sljedeće događaje kao osnovu za glavne petlje/zadatake:

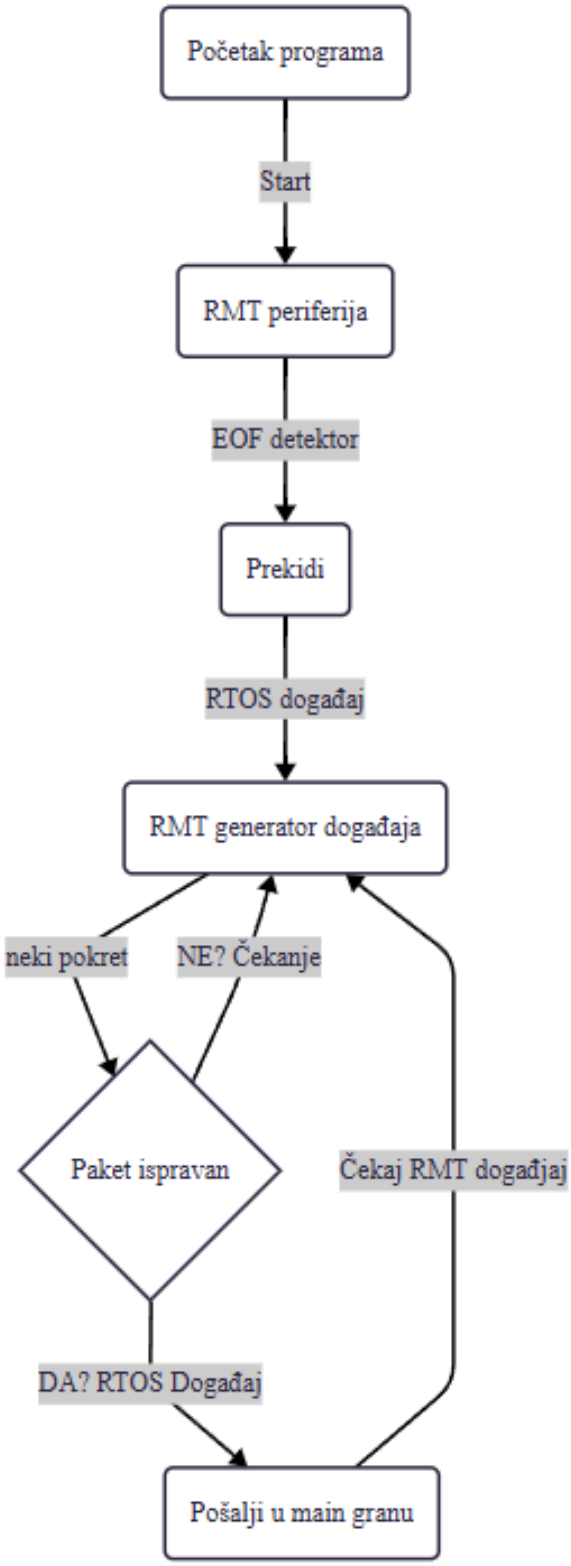
- Primanje VAN okvira
- Primanje UART podataka s računala
- Promjena stanja vozila
- Timer događaji
- Glavni zadatak

Budući da ESP32 ima dvije jezgre, jedna je posvećena isključivo primanju i analiziranju VAN okvira, jer su oni vrlo česti i ne želimo propustiti niti jedan.

3.3.2. Primanje VAN okvira

Za primanje VAN okvira potrebno je ručno čitati i vremenski pratiti stanja RX pina. Srećom, ESP32 ima sklopovski periferni uređaj koji upravo to omogućuje: RMT periferni uređaj (slika 3.26). On može pratiti stanja GPIO pinova i mjeriti koliko dugo su u tom stanju. Nakon što postavimo pragove za minimalnu i maksimalnu duljinu signala, periferni uređaj može generirati prekid (eng. *interrupt*) svaki put kada detektira sekvencu **EOF**.

Kada se detektira sekvenca **EOF**, RMT periferalni upravljački program generira prekid (eng. *interrupt*). U okviru tog prekida, u red zadataka (eng. *task queue*) modula za primanje VAN okvira postavljamo novi događaj i ponovno pokrećemo RMT. Modul za primanje VAN okvira preuzima primljene RMT podatke iz reda i analizira ih.



Slika 3.26. *Blok dijagram programskog toka - Jezgra 0*

VAN okvir rekonstruiramo bit po bit iz tog polja, uzimajući u obzir Manchester bitove. Nakon provjere kontrolnog zbroja (eng. *checksum*), izdvajaju se identifikator i podatci, stavljaju se u novi međuspremnik i postavljaju u red glavnog zadatka (eng. *Main task*) za daljnju obradu. Logika primanja inspirirana je Peter Pinterovom Arduino bibliotekom [6] i prilagođena je arhitekturi programske podrške.

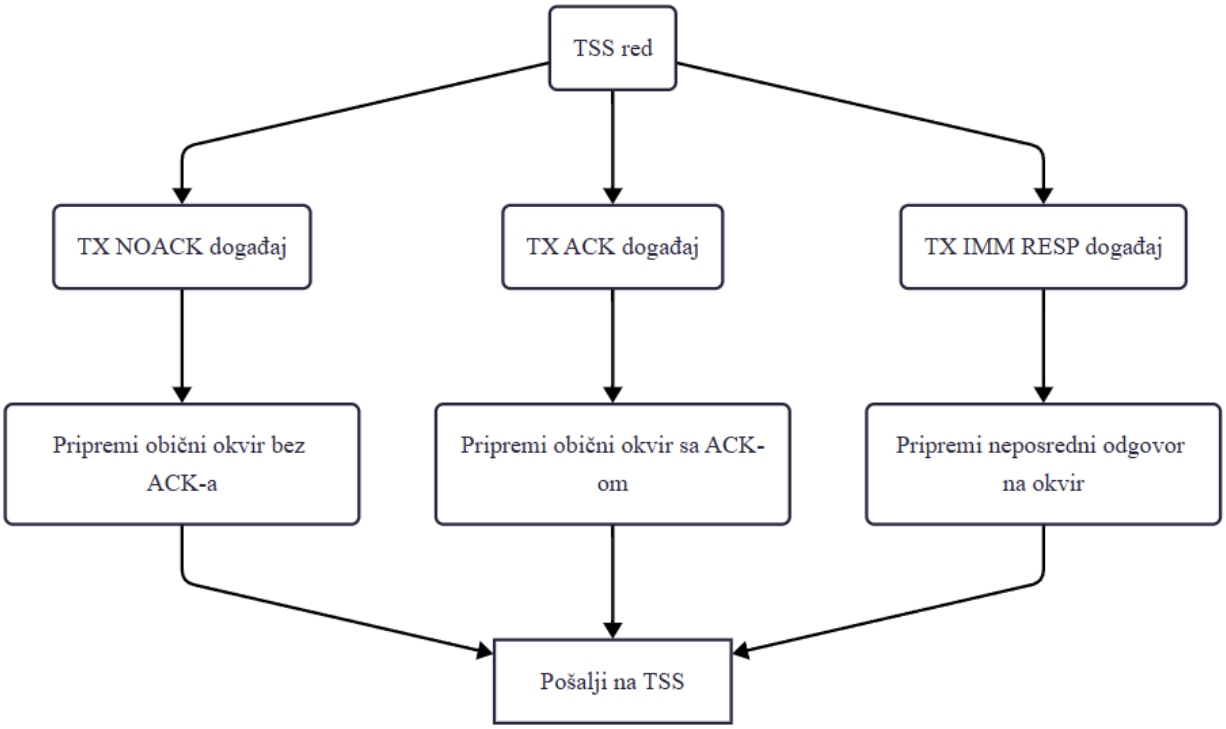
3.3.3. Analiza VAN okvira

Kada modul za primanje VAN okvira postavi novi događaj, glavni zadatak (eng. *Main task*) poziva parser paketa koji provjerava identifikator i, ako je prepoznat, analizira podatke u globalne međuspremnike te postavlja odgovarajuće događaje u red glavnog zadatka ovisno o paketu koji se obrađuje. Velika većina poznatih dekodiranih VAN okvira dostupna je na web stranici Petera Pintera [7].

3.3.4. Slanje VAN okvira

Za slanje VAN okvira prema vozilu se koristi TSS463C. TSS je zapravo dodatnih 128 bajtova RAM-a kojima se mora upravljati, budući da se mora kontrolirati sva memorija korištena za slanje i primanje podataka. Trenutačno se TSS koristi samo za slanje paketa CD izmjenjivača kako bi se mogao emulirati, te za slanje zahtjeva za resetiranje stanja putnog računala. Poruke CD mjenjača su poruke s neposrednim odgovorom, dok je zahtjev za resetiranje stanja putnog računala normalni okvir s zahtjevom za potvrdom (ACK). Paketi CD mjenjača šalju se periodično, svake sekunde, inače će ugrađeni radio smatrati da CD mjenjača nije prisutan i onemogućiti ga.

TSS komunicira preko SPI sučelja i malo je nezgrapan za korištenje, pa je napisan vlastiti upravljački program radi lakšeg rukovanja. Osim toga, postoji zaseban zadatak koji nadzire stanje TSS poruka i stavlja ih u red za slanje. Navedeni zadatak je opisan u blok dijagramu na slici 3.27..

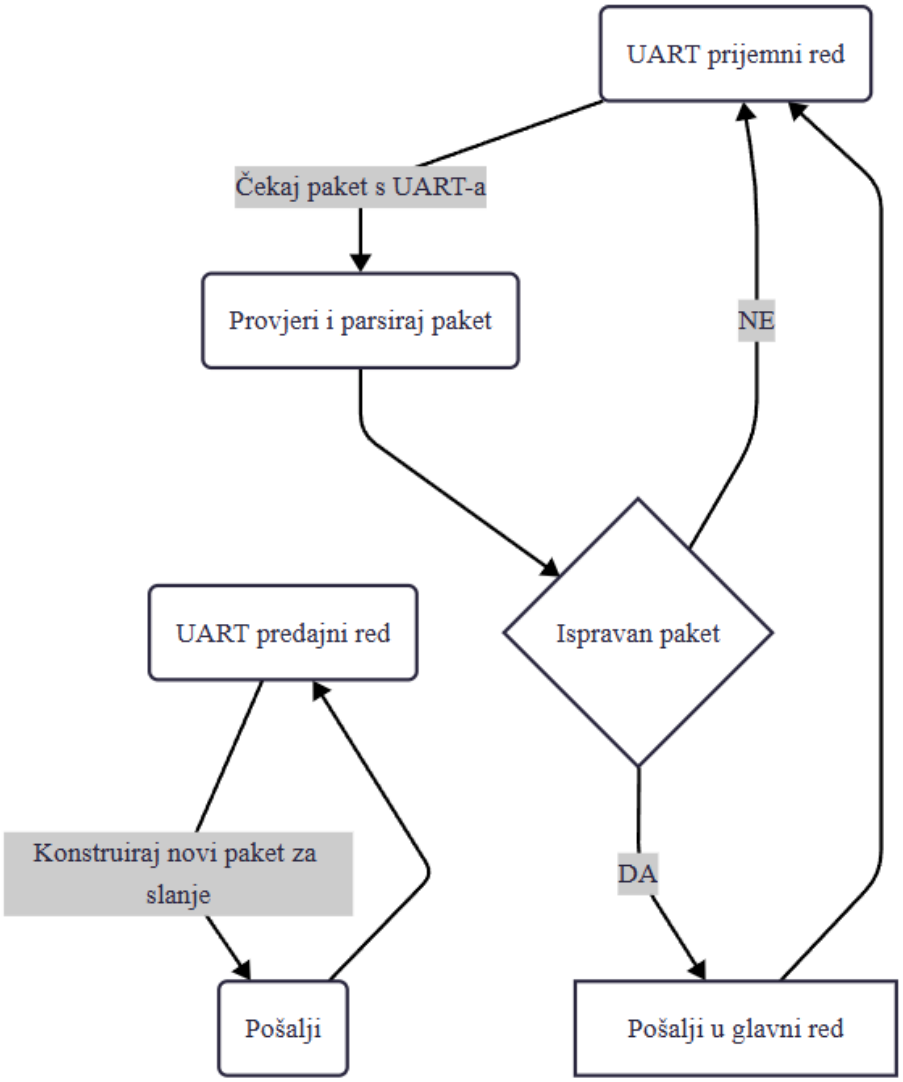


Slika 3.27. *Blok dijagram programskog toka – TSS zadatak*

3.3.5. UART komunikacija

Informacijsko-zabavno računalo komunicira s ESP32 preko UART sučelja. Razlog korištenja UART-a je njegova jednostavnost. ESP32 uglavnom šalje podatke računalu, ali također može primati zahtjeve s računala.

UART kod se izvodi u vlastitom zadatku (taj zadatak je prikazan na slici 3.28.) koji prima događaje iz drugih zadataka i također može slati događaje glavnom zadatku. Kada glavna petlja (eng. *Main loop*) detektira promjenu u globalnim međuspremnicima podataka (primljen VAN okvir, promjena napona akumulatora, itd.), postavlja odgovarajući događaj u UART zadatak s podacima koji se šalju računalu. Protokol i strukture podataka su inspirirane MSP protokolom korištenim u programskoj podršci upravljača leta na FPV dronovima.



Slika 3.28. *Blok dijagram programskog toka – UART zadatak*

3.3.6. Upravljanje događajima i zadacima

Programska podrška koristi nekoliko zadataka koji se izvode u vlastitim petljama i čekaju događaje iz više redova. Ukupno postoje 4 globalna reda u programskoj podršci, po jedan za svaku glavnu komponentu:

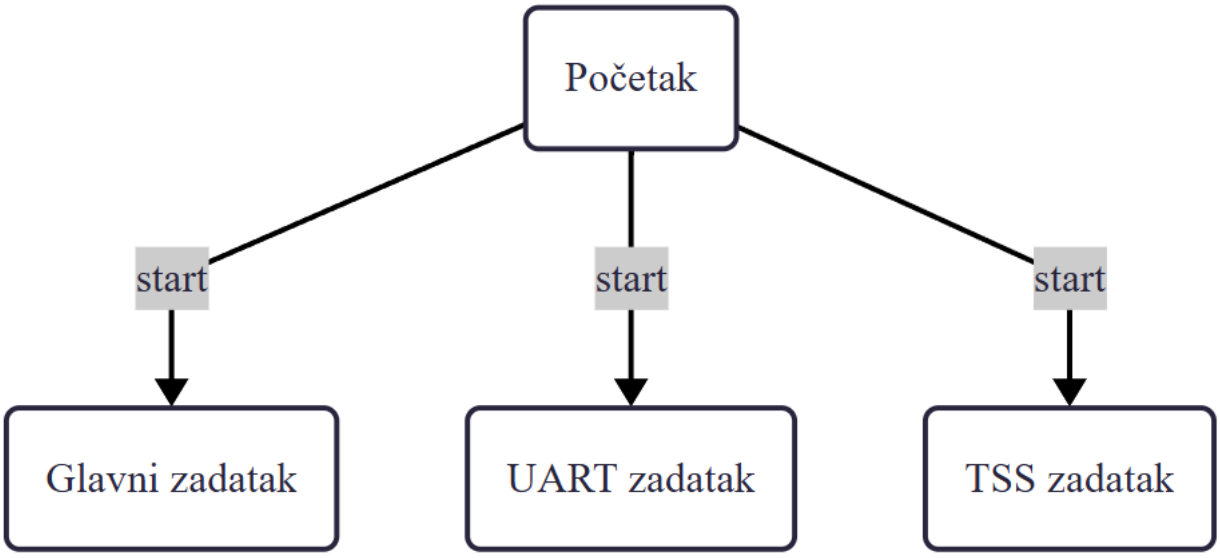
- Glavni red - Čita ga glavni zadatak. Svi ostali zadaci šalju svoje događaje u red glavnog zadatka. Glavni zadatak zatim odlučuje što učiniti i gdje proslijediti događaj.
- TSS red - Čita ga TSS zadatak. Koristi se za stavljanje paketa u red za slanje od strane TSS-a.

- UART red - Čita ga UART zadatak. Koristi se za slanje paketa informacijsko-zabavnom računalu.
- VAN red (neiskorišteno) - Čita ga VAN RMT zadatak. Ostavljen za buduću upotrebu.

Osim toga, postoji interni red u VAN RMT zadatku. Taj red se isključivo koristi između prekida (eng. *interrupt*) RMT uređaja i VAN RMT zadatka.

Događaji su definirani kao struktura s identifikatorom događaja i pokazivačem na podatke. Podaci ovise o primljenom događaju.

Tri spomenuta reda koji su u upotrebi se nalaze na blok dijagramu na slici 3.29.



Slika 3.29. *Blok dijagram programskog toka - Jezgra 1*

3.3.7. Upravljanje režimima napajanja

Važno je upravljati potrošnjom energije sustava. Radi lakšeg upravljanja definirali smo nekoliko „stanja vozila”. Pogledajte tablicu 3.1. za popis tih stanja i ponašanje sustava. Jedna važna komponenta spomenuta u tablici je stanje dubokog spavanja ESP32-a (eng. *deep sleep*). Kada je u dubokom snu, ESP je praktički isključen. Jedino što radi na čipu je posebni dio RAM-a, unutarnji RTC sklop i, po potrebi, ULP koprocesor. U ovom slučaju slučaju korišten je isti ADC pin koji se koristi za mjerenje napona baterije kako bi se ESP probudio. Zbog naponskog raspona nožice, ne

može se koristiti jednostavno buđenje putem GPIO-a, već se mora koristiti ULP koprocesor za mjerenje napona i buđenje ESP-a kada napon prijeđe određeni prag.

Tablica 3.1. *Tablica stanja auta*

Opis	**Stanje računala**	**ESP32 stanje**
Automobil u stanju mirovanja	Isključeno (bez napajanja)	Stanje spavanja
Motor zaustavljen, automobil zaključan i nije u stanju mirovanja	Isključeno	Aktivno stanje
Motor zaustavljen, automobil nije zaključan i nije u stanju mirovanja	Ako dolazi iz gornjih stanja, isključeno, inače samo ekran isključen	Aktivno stanje
Motor zaustavljen, radio uključen	Uključen, ekran uključen	Aktivno stanje
Motor zaustavljen, ključ u poziciji ACC	Uključen, ekran uključen	Aktivno stanje
Motor zaustavljen, ključ u poziciji IGN	Uključen, ekran uključen	Aktivno stanje
Motor se pokreće	Uključen, ekran isključen	Aktivno stanje
Motor je pokrenut	Uključen, ekran uključen	Aktivno stanje

3.3.8. ULP koprocesor

ESP32 ima vrlo niskopotrošni koprocesor. Ima vrlo jednostavnu arhitekturu i troši vrlo malo energije. Idealno je koristiti ga za ADC mjerenja i buđenje ESP-a. Ne može se programirati izravno u C-u, ali postoje dva načina pisanja koda. Prvi način je pisanje asembler koda u zasebnoj datoteci i njegovo integriranje u datoteku firmwarea. Drugi način je korištenje unaprijed definiranih C makroa i pisanje programa u polju unutar C programa te njegovo učitavanje na taj način. Odabrana je druga opcija jer je znatno jednostavnija.

Programski kod opisanog programskog toka se nalazi u prilozima, P.2.

4. TESTIRANJE I REZULTATI

4.1. Metodologija testiranja

Način testiranja je ugraditi projekt u automobil te testirati njegov rad tijekom vožnje. Očekivano ponašanje je ispravan prikaz brzine automobila, obrtaja te trenutačnog stupnja prijenosa na ekranu. Ispravnost prikaza se određuje usporedbom prikazanih vrijednosti s pokaznicama brzine, obrtaja i trenutačne brzine koji se tvornički nalaze u automobilu. Dozvoljeno je malo odstupanje zbog nepreciznosti pokazivača koji se nalaze već par desetljeća u automobilu. Pored Navedenih indikatora tu su još i indikatori za razinu spremnika, temperaturu motora automobila, ukupan broj prijeđenih kilometara te prosječnu potrošnju na 100 kilometara čija točnost se procjenjuje na isti način koji je naveden. Nadalje, testira se i funkcionalnost multimedije tako da se prikazuje izbornik za radiostanice kao i mogućnost reprodukcije multimedijskog sadržaja iz neke vanjske memorije npr. USB uređaj za pohranu podataka.

4.2. Rezultati testiranja

Prvo je testiran prikaz telemetrije automobila čiji prikaz je vidljiv na slici 4.1.. Uzastopnim vožnjama i prometnim kamerama je utvrđeno kako se na ekranu prikazuje točna vrijednost brzine automobila i obrtaja motora. Kako se stupanj prijenosa računa na osnovu obrtaja i brzine kretanja automobila, to je veličina koja bi bila podložna greškama u izračunima međutim u praksi se pokazalo kao vjerodostojan izračun bez krivih indikacija. Indikatori razine spremnika goriva, temperaturu motora automobila te prosječna potrošnja se podudaraju s vrijednostima prikazanih na glavnoj tabli čime zaključujemo kako sva komunikacija ispravno radi.

Slika 4.1 *Prikaz ekrana tijekom testiranja - Telemetrija automobila*

Na slici 4.2. je prikazan test učitavanja multimedijskog sadržaja sa vanjskog uređaja za pohranu podataka. Kako je sadržaj bez ikakvih problema učitan te reproduciran, zaključuje se kako i taj dio sustava radi kao što je predviđeno.

Slika 4.2 *Prikaz ekrana tijekom testiranja - Multimedija - Uređaj za pohranu podataka*

Na slikama 4.3. i 4.4. je prikazan test prikaza aktivne radio stanice (ako je radio upaljen) iz kojih se može vidjeti kako i ta funkcionalnost ima predviđeno ponašanje.

Slika 4.3 *Prikaz ekrana tijekom testiranja - Multimedija - Radio*

Slika 4.4 *Infotainment sustav smješten u autu*

5. ZAKLJUČAK

Realizacija ovog projekta pokazala je da je moguće uspješno integrirati moderan infotainment sustav u starije vozilo koristeći kombinaciju vlastitog hardverskog modula i open-source softverskih alata. Ključni izazovi bili su razumijevanje i implementacija VAN protokola, osiguranje stabilnog napajanja te izrada pouzdanog firmwarea za ESP32 mikroupravljač. Pažljivim dizajnom tiskane pločice, korištenjem provjerenih komponenti i pisanjem modularnog koda, postignuta je robusna i stabilna komunikacija s vozilom.

Prednosti rješenja su otvorenost i mogućnost proširenja funkcionalnosti, niska cijena u odnosu na komercijalne alternative te potpuna kontrola nad softverom i hardverom. Sustav se pokazao pouzdanim tijekom testiranja – prikaz telemetrije, statusa vozila i reprodukcija multimedije radili su bez većih odstupanja od tvorničkih pokazivača. Najveći izazovi, poput dekodiranja VAN okvira i implementacije režima štednje energije, uspješno su savladani korištenjem ESP-IDF okvira i pažljivim upravljanjem događajima i zadacima.

Ipak, postoje područja za daljnje unaprjeđenje. Potrebno je dodatno optimizirati potrošnju energije u stanju mirovanja kako bi se maksimalno smanjilo pražnjenje akumulatora te doraditi korisničko sučelje na Raspberry Pi računalu radi intuitivnijeg korištenja. Budući razvoj mogao bi uključivati potpunu integraciju s postojećim zaslonom u vozilu, dodavanje bežične povezivosti (Wi-Fi/Bluetooth sinkronizacija s pametnim telefonom) te naprednije funkcije poput OTA (Over-the-Air) nadogradnje softvera. Time bi se sustav dodatno približio funkcionalnosti i fleksibilnosti modernih tvorničkih infotainment rješenja.

LITERATURA

[**1**] TSS463C Datasheet, TSS463C pdf, TSS463C Download, TSS463C Description, TSS463C Datasheet, TSS463C view ::: ALLDATASHEET ::: (pogledano 25.09.2025)

[**2**] Info Tech n°6, http://graham.auld.me.uk/ projects/vanbus/datasheets/Mux1.pdf (pogledano 25.9.2025.).

[**3**] PSAPeugeot Citroën. Peugeot Service Box backup- SEDRE.

[**4**] MP8759 Datasheet, https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP8759GD Z/document_id/1011/ (pogledano 25.9.2025.)

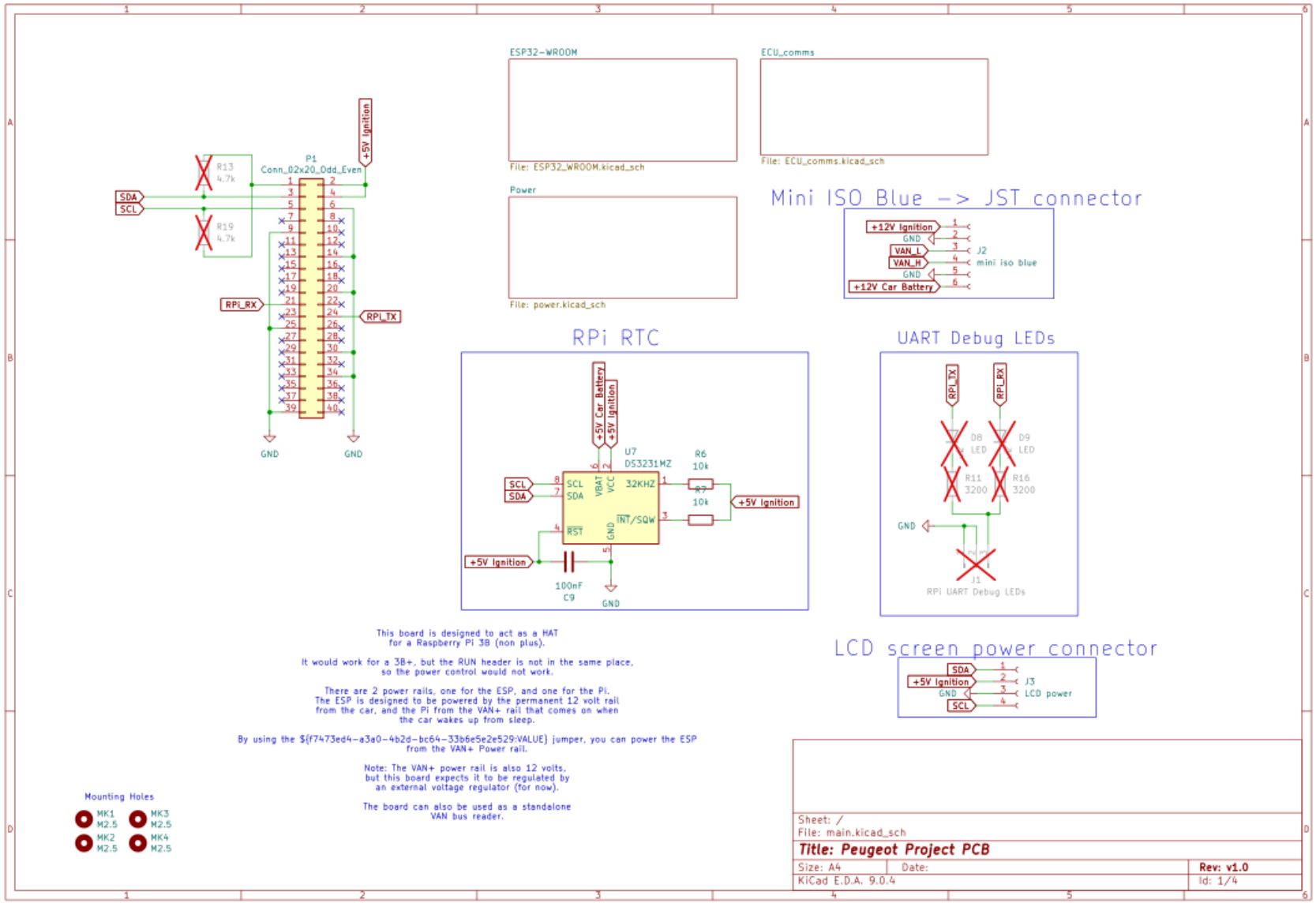
[**5**] DS3231MZ Datasheet, https://www.alldatasheet.com/datasheet-pdf/download/423153/MAXIM/DS3231MZ.html, (pogledano 25.9.2025.)

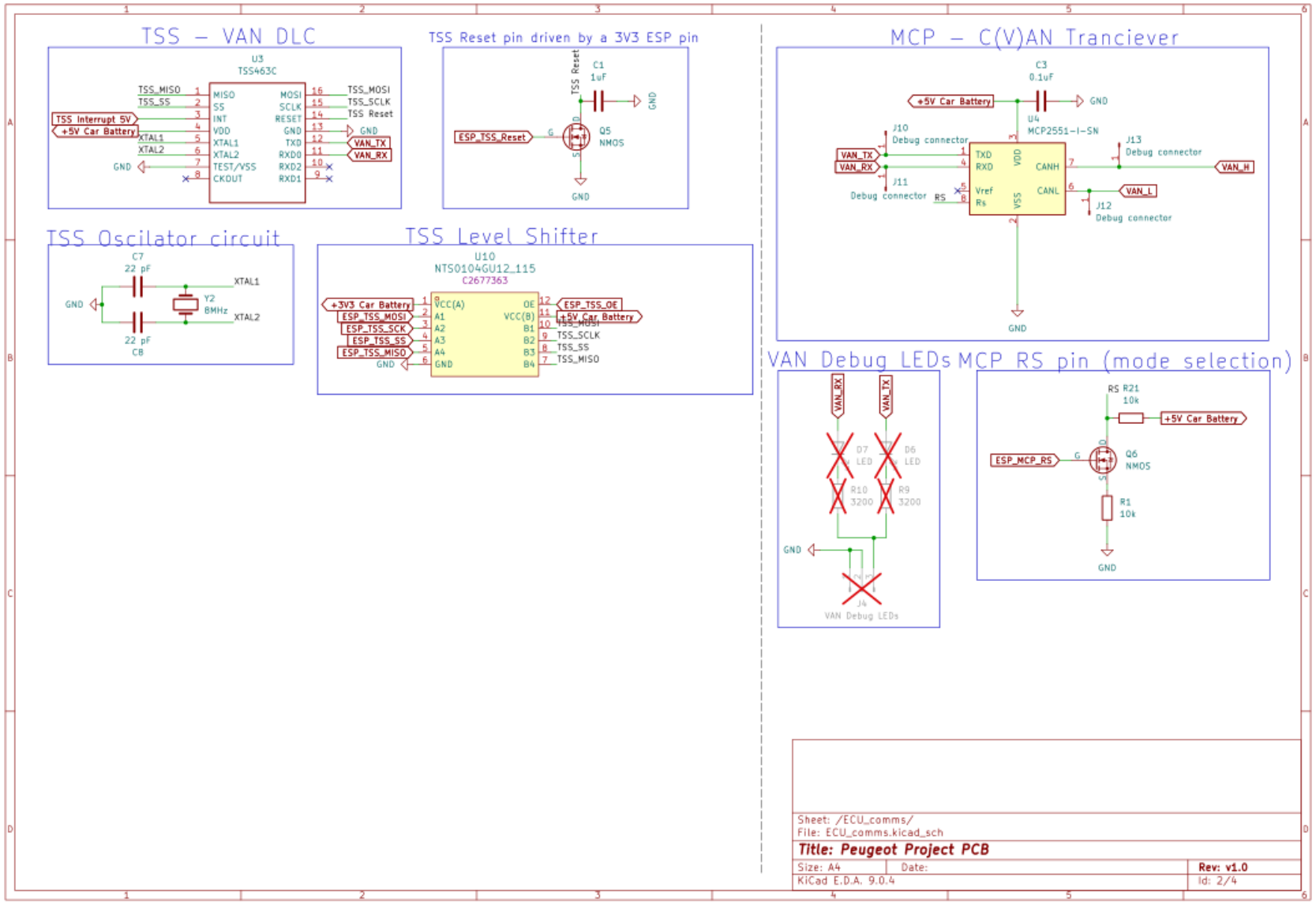
[**6**] ESP32 RMT peripheral Vehicle Area Network (VAN bus) reader, https://github.com/morcibacsi/esp32_rmt_van_rx (pogledano 25.9.2025.)

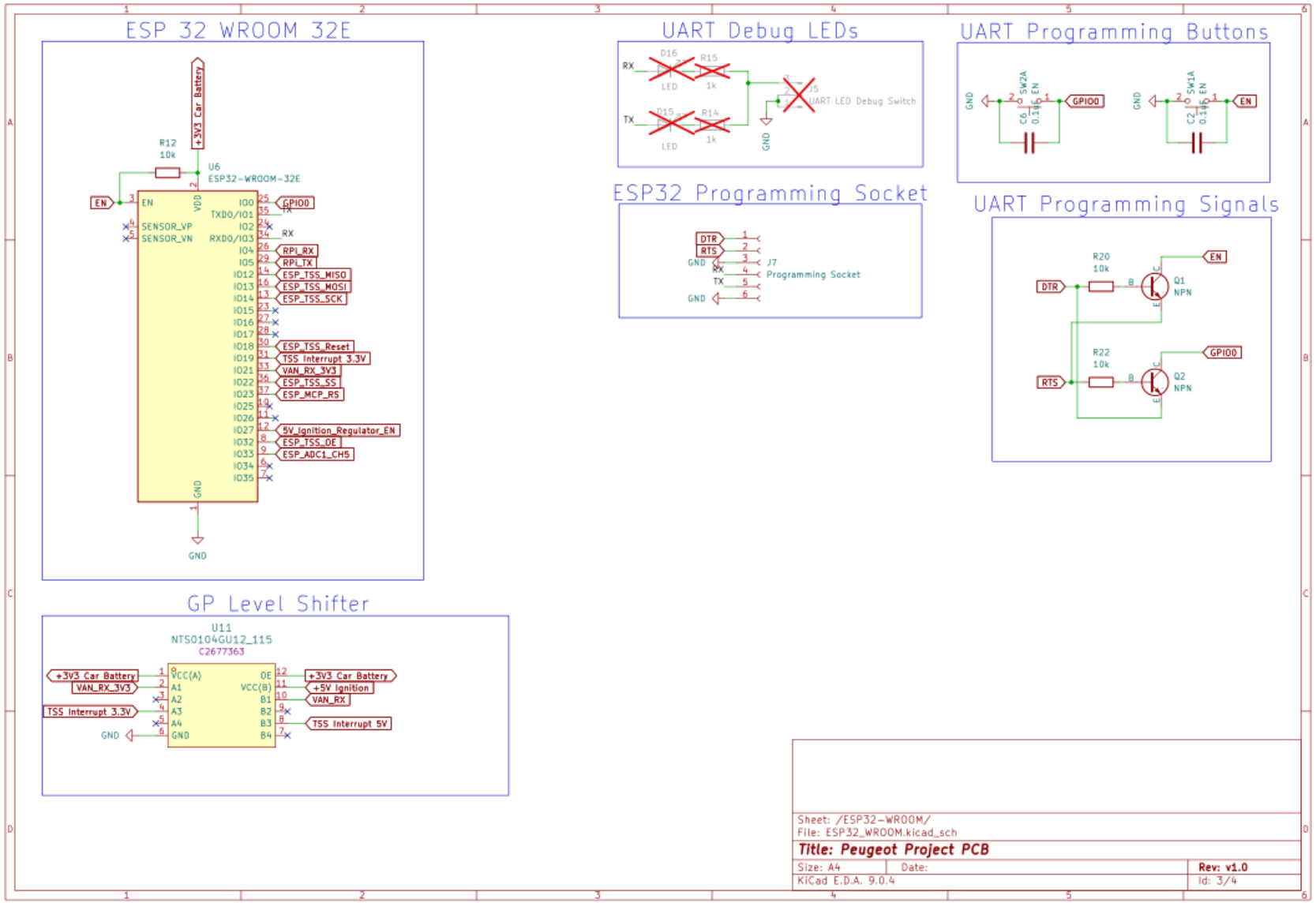
[**7**] PSA VAN bus, All data related to Peugeot 206 307 406 S2, Citroen C3 2004., http://pinterpeti.hu/psavanbus/PSA-VAN.html (pogledano 25.9.2025.)

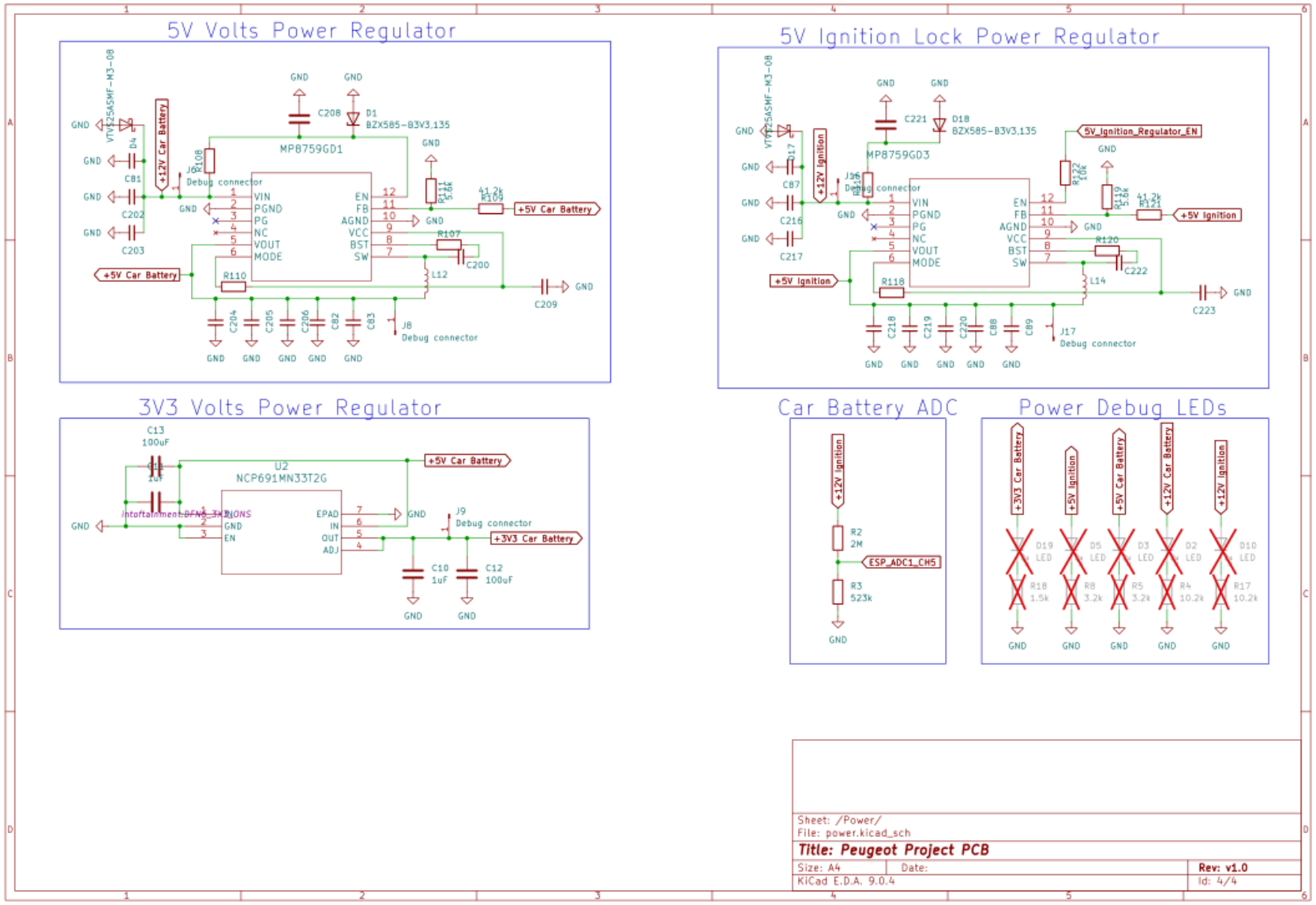
PRILOZI I DODACI

P.1 Kompletna shema sklopovlja


ESP32-WROOM
File: ESP32_WROOM.kicad_sch
ECU_comms
File: ECU_comms.kicad_sch
Power
File: power.kicad_sch
Mini ISO Blue -> JST connector
+12V Ignition
GND
VAN_L
VAN_H
GND
+12V Car Battery
J2
mini iso blue
P1
Conn_02x20_Odd_Even
+5V Ignition
R13
4.7k
R19
4.7k
SDA
SCL
RPi_RX
RPi_TX
GND
GND
RPi RTC
+5V Car Battery
+5V Ignition
U7
DS3231MZ
SCL
SDA
VBAT
VCC
32KHZ
RST
GND
INT/SQW
R6
10k
R7
10k
+5V Ignition
+5V Ignition
100nF
C9
GND
UART Debug LEDs
RPi_TX
RPi_RX
D8
LED
D9
LED
R11
3200
R16
3200
GND
J1
RPi UART Debug LEDs
LCD screen power connector
SDA
+5V Ignition
GND
SCL
J3
LCD power
This board is designed to act as a HAT
for a Raspberry Pi 3B (non plus).
It would work for a 3B+, but the RUN header is not in the same place,
so the power control would not work.
There are 2 power rails, one for the ESP, and one for the Pi.
The ESP is designed to be powered by the permanent 12 volt rail
from the car, and the Pi from the VAN+ rail that comes on when
the car wakes up from sleep.
By using the ${f7473ed4-a3a0-4b2d-bc64-33b6e5e2e529:VALUE} jumper, you can power the ESP
from the VAN+ Power rail.
Note: The VAN+ power rail is also 12 volts,
but this board expects it to be regulated by
an external voltage regulator (for now).
The board can also be used as a standalone
VAN bus reader.
Mounting Holes
MK1
M2.5
MK2
M2.5
MK3
M2.5
MK4
M2.5
Sheet: /
File: main.kicad_sch
Title: Peugeot Project PCB
Size: A4
Date:
Rev: v1.0
KiCad E.D.A. 9.0.4
Id: 1/4


TSS – VAN DLC
U3
TSS463C
TSS_MISO
TSS_SS
TSS Interrupt 5V
+5V Car Battery
XTAL1
XTAL2
GND
MISO
SS
INT
VDD
XTAL1
XTAL2
TEST/VSS
CKOUT
MOSI
SCLK
RESET
GND
TXD
RXD0
RXD2
RXD1
TSS_MOSI
TSS_SCLK
TSS Reset
GND
VAN_TX
VAN_RX
TSS Reset pin driven by a 3V3 ESP pin
C1
1uF
TSS Reset
GND
ESP_TSS_Reset
Q5
NMOS
GND
MCP – C(V)AN Tranciever
C3
0.1uF
+5V Car Battery
GND
U4
MCP2551-I-SN
J10
Debug connector
VAN_TX
VAN_RX
J11
Debug connector
RS
TXD
RXD
Vref
Rs
VDD
VSS
CANH
CANL
J13
Debug connector
VAN_H
VAN_L
J12
Debug connector
GND
TSS Oscilator circuit
C7
22 pF
GND
XTAL1
Y2
8MHz
XTAL2
22 pF
C8
TSS Level Shifter
U10
NTS0104GU12_115
C2677363
+3V3 Car Battery
ESP_TSS_MOSI
ESP_TSS_SCK
ESP_TSS_SS
ESP_TSS_MISO
GND
VCC(A)
A1
A2
A3
A4
GND
OE
VCC(B)
B1
B2
B3
B4
ESP_TSS_OE
+5V Car Battery
TSS_MOSI
TSS_SCLK
TSS_SS
TSS_MISO
VAN Debug LEDs
VAN_RX
VAN_TX
D7
LED
D6
LED
R10
3200
R9
3200
GND
J4
VAN Debug LEDs
MCP RS pin (mode selection)
RS R21
10k
+5V Car Battery
ESP_MCP_RS
Q6
NMOS
R1
10k
GND
Sheet: /ECU_comms/
File: ECU_comms.kicad_sch
Title: Peugeot Project PCB
Size: A4
Date:
Rev: v1.0
KiCad E.D.A. 9.0.4
Id: 2/4


ESP 32 WROOM 32E
+3V3 Car Battery
R12
10k
U6
ESP32-WROOM-32E
EN
SENSOR_VP
SENSOR_VN
VDD
GND
IO0
TXD0/IO1
RXD0/IO3
IO2
IO4
IO5
IO12
IO13
IO14
IO15
IO16
IO17
IO18
IO19
IO21
IO22
IO23
IO25
IO26
IO27
IO32
IO33
IO34
IO35
GPIO0
TX
RX
RPi_RX
RPi_TX
ESP_TSS_MISO
ESP_TSS_MOSI
ESP_TSS_SCK
ESP_TSS_Reset
TSS Interrupt 3.3V
VAN_RX_3V3
ESP_TSS_SS
ESP_MCP_RS
5V_Ignition_Regulator_EN
ESP_TSS_OE
ESP_ADC1_CH5
GND
UART Debug LEDs
RX
TX
D16
D15
LED
R15
R14
1k
J5
UART LED Debug Switch
GND
UART Programming Buttons
GND
SW2A
SW1A
EN
C6
C2
0.1µF
GPIO0
EN
ESP32 Programming Socket
DTR
RTS
GND
TX
J7
Programming Socket
UART Programming Signals
R20
10k
R22
10k
DTR
RTS
Q1
Q2
NPN
EN
GPIO0
GP Level Shifter
U11
NTS0104GU12_115
C2677363
VCC(A)
VCC(B)
OE
A1
A2
A3
A4
B1
B2
B3
B4
+3V3 Car Battery
VAN_RX_3V3
TSS Interrupt 3.3V
GND
+3V3 Car Battery
+5V Ignition
VAN_RX
TSS Interrupt 5V
Sheet: /ESP32-WROOM/
File: ESP32_WROOM.kicad_sch
Title: Peugeot Project PCB
Size: A4
Date:
Rev: v1.0
KiCad E.D.A. 9.0.4
Id: 3/4


5V Volts Power Regulator
MP8759GD1
BZX585-B3V3,135
Debug connector
+12V Car Battery
+5V Car Battery
5V Ignition Lock Power Regulator
MP8759GD3
5V_Ignition_Regulator_EN
+12V Ignition
+5V Ignition
3V3 Volts Power Regulator
NCP691MN33T2G
+3V3 Car Battery
Car Battery ADC
ESP_ADC1_CH5
Power Debug LEDs
+3V3 Car Battery
+5V Ignition
+5V Car Battery
+12V Car Battery
+12V Ignition
Sheet: /Power/
File: power.kicad_sch
Title: Peugeot Project PCB
Size: A4
Date:
Rev: v1.0
KiCad E.D.A. 9.0.4
Id: 4/4

P.2 Kompletan izvorni kod firmware-a na ESP32.

```
#include "freertos/FreeRTOS.h"
#include <stdio.h>
#include <string.h>

#include "freertos/queue.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "include/global_config.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"
#include "driver/gpio.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali_scheme.h"
#include "ulp.h"
#include "ulp_adc.h"
#include "ulp_common.h"
#include "soc/rtc_cntl_reg.h"
#include "driver/rtc_io.h"
#include "esp_sleep.h"
#include "driver/uart.h"

#include "esp_timer.h"

#include "include/mive/common.h"
#include "include/global_init.h"
#include "include/van/tss463c.h"
#include "include/van/van_packet_iden.h"
#include "include/van_structs/van_cdc_emulator_structs.h"
#include "include/van/van_packet_parser.h"
#include "include/mive/psa_packet_defs.h"
```

```
#include "include/mive/psa_helper.h"


#define ARRAY_SIZE_OFFSET   5
mive_global_state_t g_global_state = {0};
volatile int g_global_car_state = PSA_STATE_CAR_SLEEP;
volatile int g_global_ext_power_state = 1;
volatile float g_bat_voltage = 0.0f;

static const char *TAG = "MAIN";

static const float ADC_R2 = 523000;
static const float ADC_R1 = 2000000;
static const int audio_menu_duration_ms = 4000;

static int timer_num = 0;

struct psa_output_data_buffers global_libpsa_buffers;

mive_uart_task_packet_t* global_uart_send_buffers;
mive_uart_task_packet_t* global_uart_receive_buffers;



RTC_DATA_ATTR int rtc_data_valid = 0;
RTC_DATA_ATTR struct psa_preset_data rtc_preset_data[4];

void main_task(void* params);
void stats_task(void *arg);

static char* car_state_str[] = {
    [PSA_STATE_CAR_SLEEP] = __STRINGIFY(PSA_STATE_CAR_SLEEP),
    [PSA_STATE_ECONOMY_MODE] = __STRINGIFY(PSA_STATE_ECONOMY_MODE),
    [PSA_STATE_CAR_LOCKED] = __STRINGIFY(PSA_STATE_CAR_LOCKED),
```

```
    [PSA_STATE_CAR_OFF] = __STRINGIFY(PSA_STATE_CAR_OFF),
    [PSA_STATE_RADIO_ON] = __STRINGIFY(PSA_STATE_RADIO_ON),
    [PSA_STATE_ACCESSORY] = __STRINGIFY(PSA_STATE_ACCESSORY),
    [PSA_STATE_IGNITION] = __STRINGIFY(PSA_STATE_IGNITION),
    [PSA_STATE_CRANKING] = __STRINGIFY(PSA_STATE_CRANKING),
    [PSA_STATE_ENGINE_ON] = __STRINGIFY(PSA_STATE_ENGINE_ON),
    [PSA_STATE_UNKNOWN] = __STRINGIFY(PSA_STATE_UNKNOWN)
};

IRAM_ATTR static void timer_callback_f(void* user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
    QueueHandle_t main_queue = (QueueHandle_t)user_data;
    struct mive_global_event timer_event = {
        .ev_data = NULL,
        .event = MIVE_EVENT_TIMER_1S,
    };
    xQueueSendFromISR(main_queue, &timer_event, &high_task_wakeup);
    if(high_task_wakeup)
    {
        esp_timer_isr_dispatch_need_yield();
    }
}

IRAM_ATTR static void timer_callback_audio_timeout_f(void* user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
    QueueHandle_t main_queue = (QueueHandle_t)user_data;
    struct mive_global_event timer_event = {
        .ev_data = NULL,
        .event = MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,
    };
    xQueueSendFromISR(main_queue, &timer_event, &high_task_wakeup);
    if(high_task_wakeup)
```

```
    {
        esp_timer_isr_dispatch_need_yield();
    }
}

IRAM_ATTR static void timer_callback_oneshot_f(void* user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
    QueueHandle_t main_queue = (QueueHandle_t)user_data;
    struct mive_global_event timer_event = {
        .ev_data = NULL,
        .event = MIVE_EVENT_TIMER_1MS,
    };
    timer_num++;
    xQueueSendFromISR(main_queue, &timer_event, &high_task_wakeup);
    if(high_task_wakeup)
    {
        esp_timer_isr_dispatch_need_yield();
    }
}

static esp_err_t print_real_time_stats(TickType_t xTicksToWait)
{
    TaskStatus_t *start_array = NULL, *end_array = NULL;
    UBaseType_t start_array_size, end_array_size;
    configRUN_TIME_COUNTER_TYPE start_run_time, end_run_time;
    esp_err_t ret;

    //Allocate array to store current task states
    start_array_size = uxTaskGetNumberOfTasks() + ARRAY_SIZE_OFFSET;
    start_array = malloc(sizeof(TaskStatus_t) * start_array_size);
    if (start_array == NULL) {
        ret = ESP_ERR_NO_MEM;
        goto exit;
```

```
}
//Get current task states
start_array_size = uxTaskGetSystemState(start_array, start_array_size, &start_run_time);
if (start_array_size == 0) {
    ret = ESP_ERR_INVALID_SIZE;
    goto exit;
}

vTaskDelay(xTicksToWait);

//Allocate array to store tasks states post delay
end_array_size = uxTaskGetNumberOfTasks() + ARRAY_SIZE_OFFSET;
end_array = malloc(sizeof(TaskStatus_t) * end_array_size);
if (end_array == NULL) {
    ret = ESP_ERR_NO_MEM;
    goto exit;
}
//Get post delay task states
end_array_size = uxTaskGetSystemState(end_array, end_array_size, &end_run_time);
if (end_array_size == 0) {
    ret = ESP_ERR_INVALID_SIZE;
    goto exit;
}

//Calculate total_elapsed_time in units of run time stats clock period.
uint32_t total_elapsed_time = (end_run_time - start_run_time);
if (total_elapsed_time == 0) {
    ret = ESP_ERR_INVALID_STATE;
    goto exit;
}

printf("| Task | Run Time | Percentage\n");
//Match each task in start_array to those in the end_array
for (int i = 0; i < start_array_size; i++) {
```

```
        int k = -1;
        for (int j = 0; j < end_array_size; j++) {
            if (start_array[i].xHandle == end_array[j].xHandle) {
                k = j;
                //Mark that task have been matched by overwriting their handles
                start_array[i].xHandle = NULL;
                end_array[j].xHandle = NULL;
                break;
            }
        }
        //Check if matching task found
        if (k >= 0) {
            uint32_t task_elapsed_time = end_array[k].ulRunTimeCounter - start_array[i].ulRunTimeCounter;
            uint32_t percentage_time = (task_elapsed_time * 100UL) / (total_elapsed_time * CONFIG_FREERT
OS_NUMBER_OF_CORES);
            printf("| %s | %"PRIu32" | %"PRIu32"%%\n", start_array[i].pcTaskName, task_elapsed_time, percent
age_time);
        }
    }

    //Print unmatched tasks
    for (int i = 0; i < start_array_size; i++) {
        if (start_array[i].xHandle != NULL) {
            printf("| %s | Deleted\n", start_array[i].pcTaskName);
        }
    }
    for (int i = 0; i < end_array_size; i++) {
        if (end_array[i].xHandle != NULL) {
            printf("| %s | Created\n", end_array[i].pcTaskName);
        }
    }
    ret = ESP_OK;

exit:    //Common return path
```

```
    free(start_array);
    free(end_array);
    return ret;
}

/**
 * @brief Setup the ULP as a wake source to wake the ESP up
 * when the ADC reading goes above `high_adc_treshold`.
 *
 * @param high_adc_treshold
 */
void ulp_adc_wake_up(unsigned int high_adc_treshold)
{
    esp_err_t err;
    adc_oneshot_unit_handle_t adc1_handle = g_global_state.adc_handle;

    ulp_adc_cfg_t adc_cfg = {
        .adc_n = PSA_ADC_UNIT,
        .channel = PSA_ADC_CHANNEL,
        .width = ADC_BITWIDTH_12,
        .atten = ADC_ATTEN_DB_12,
        .ulp_mode = ADC_ULP_MODE_FSM
    };

    const ulp_insn_t program[] = {
        I_MOVI(R0, 0),              // Set reg. R0 to initial 0
        I_MOVI(R2, 0),              // Set reg. R2 to initial 0
        M_LABEL(1),                 // LABEL 1 - Start of measurement
        I_ADDI(R0, R0, 1),          // Increment cycle counter (reg. R0)
        I_ADC(R1, PSA_ADC_UNIT, PSA_ADC_CHANNEL),          // Read ADC value to reg. R1
        I_ADDR(R2, R2, R1),         // Add ADC value from reg R1 to reg. R2
        M_BL(1, 4),                 // If cycle counter is less than 4, go to LABEL 1
        I_RSHI(R0, R2, 2),          // Divide accumulated ADC value in reg. R2 by 4 and save it to reg. R0
        M_BGE(2, high_adc_treshold),  // If average ADC value from reg. R0 is higher or equal than high_adc_
```

```
treshold, go to LABEL 2
        I_HALT(),                   // Halt the coprocessor, it will be woken up by the timer again
        M_LABEL(2),                 // LABEL 3
        I_WAKE(),                   // Wake up ESP32
        I_END(),                    // Stop ULP program timer
        I_HALT(),                   // Halt the coprocessor
    };

    adc_oneshot_del_unit(adc1_handle);

    ESP_ERROR_CHECK(ulp_adc_init(&adc_cfg));

    rtc_gpio_init(GPIO_NUM_33);


    size_t size = sizeof(program)/sizeof(ulp_insn_t);
    ESP_ERROR_CHECK(ulp_process_macros_and_load(0, program, &size));
    ulp_set_wakeup_period(0, 20000);
    err = ulp_run(0);
    ESP_ERROR_CHECK(err);
    ESP_ERROR_CHECK(esp_sleep_enable_ulp_wakeup());
}

void go_to_sleep(void)
{
    tss_sleep(&global_tss_instance);
    gpio_set_level(32, 0);
    gpio_set_level(27, 0);
    rtc_gpio_isolate(GPIO_NUM_12);
    rtc_gpio_isolate(GPIO_NUM_15);
    printf("Going to sleep\n");
    ulp_adc_wake_up(1000);
    esp_deep_sleep_start();
}
```

```
void update_ext_power_state(int power)
{
   // gpio_set_level(PSA_EXT_REG_PIN, power ? 1 : 0);
   // gpio_set_level(TSS_OE_ENABLE_PIN, power ? 1 : 0);

   g_global_ext_power_state = power;
}

void update_car_state(int new_state)
{
   if(new_state == g_global_car_state)
   {
      return;
   }

   ESP_LOGI(TAG, "[%s] New state = %s", __func__, car_state_str[new_state]);
   g_global_car_state = new_state;
   global_libpsa_buffers.status_data->car_state = g_global_car_state;
   libpsa_send_packet(PSA_IDENT_CAR_STATUS);
}

void calculate_car_state(void)
{
   // ESP_LOGI(TAG, "[%s]", __func__);
   int ext_power_status = g_global_ext_power_state;
   int engine_state = global_libpsa_buffers.dash_data->engine_running;
   int acc_state = global_libpsa_buffers.dash_data->accesories_on;
   int ign_state = global_libpsa_buffers.dash_data->ignition_on;
   int headunit_state = global_libpsa_buffers.headunit_data->unit_powered_on;
   int lock_state = global_libpsa_buffers.status_data->doors_locked;

   // PSA_STATE_ENGINE_ON
   if (engine_state)
```

```
{
    update_car_state(PSA_STATE_ENGINE_ON);
    ext_power_status = 1;
}
// PSA_STATE_ECONOMY_MODE
// else if (dash_packet.packet.data.economy_mode)
// {
//     update_car_state(PSA_STATE_ECONOMY_MODE);
//     ext_power_status = 0;
// }
// PSA_STATE_CRANKING
else if (acc_state == 0 &&
         ign_state == 1)
{
    update_car_state(PSA_STATE_CRANKING);
    ext_power_status = 1;
}
// PSA_STATE_IGNITION
else if (acc_state == 1 &&
         ign_state == 1)
{
    update_car_state(PSA_STATE_IGNITION);
    ext_power_status = 1;
}
// PSA_STATE_ACCESSORY
else if (acc_state == 1 &&
         ign_state == 0)
{
    update_car_state(PSA_STATE_ACCESSORY);
    ext_power_status = 1;
}
// PSA_STATE_RADIO_ON
// PSA_STATE_CAR_LOCKED
// PSA_STATE_CAR_OFF
```

```
    else if (acc_state == 0 &&
             ign_state == 0)
    {
        if (headunit_state)
        {
            update_car_state(PSA_STATE_RADIO_ON);
            ext_power_status = 1;
        }
        else if (lock_state)
        {
            update_car_state(PSA_STATE_CAR_LOCKED);
            ext_power_status = 0;
        }
        else
        {
            update_car_state(PSA_STATE_CAR_OFF);
        }
    }

    update_ext_power_state(ext_power_status);
}

void app_main(void)
{
    // To TSS task
    QueueHandle_t tss_queue = xQueueCreate(10, sizeof(struct mive_global_event));

    // To VAN task
    QueueHandle_t van_queue = xQueueCreate(10, sizeof(struct mive_global_event));

    // To main task
    QueueHandle_t main_queue = xQueueCreate(20, sizeof(struct mive_global_event));

    // To uart task
```

```
    QueueHandle_t uart_queue = xQueueCreate(10, sizeof(struct mive_global_event));

    g_global_state.global_tss_queue = tss_queue;
    g_global_state.global_van_queue = van_queue;
    g_global_state.global_main_queue = main_queue;
    g_global_state.global_uart_queue = uart_queue;

    init_adc();

    init_libpsa_packets();

    if(rtc_data_valid)
    {
        memcpy(global_libpsa_buffers.presets_data_am, &rtc_preset_data[PSA_PRESET_AM], sizeof(struct p
sa_preset_data));
        memcpy(global_libpsa_buffers.presets_data_fm_1, &rtc_preset_data[PSA_PRESET_FM_1], sizeof(str
uct psa_preset_data));
        memcpy(global_libpsa_buffers.presets_data_fm_2, &rtc_preset_data[PSA_PRESET_FM_2], sizeof(str
uct psa_preset_data));
        memcpy(global_libpsa_buffers.presets_data_fm_ast, &rtc_preset_data[PSA_PRESET_FMAST], sizeof(
struct psa_preset_data));
    }

    xTaskCreatePinnedToCore(
        main_task,
        "main_task",
        10240,
        NULL, 20, NULL, 0);

    xTaskCreatePinnedToCore(
        tss_task,
        "tss_task",
        5120,
        &g_global_state, 10, NULL, 0);
```

```
    xTaskCreatePinnedToCore(
        van_rmt_task,
        "van_rx_task",
        5120,
        &g_global_state, 15, NULL, 1);

    xTaskCreatePinnedToCore(
        uart_task,
        "uart_task",
        5120,
        &g_global_state, 15, NULL, 0);

    // xTaskCreatePinnedToCore(stats_task, "stats", 4096, NULL, 3, NULL, tskNO_AFFINITY);
}


void stats_task(void *params)
{
    //Print real time stats periodically
    while (1) {
        printf("\n\nGetting real time stats over %"PRIu32" ticks\n", pdMS_TO_TICKS(1000));
        if (print_real_time_stats(pdMS_TO_TICKS(1000)) == ESP_OK) {
            printf("Real time stats obtained\n");
        } else {
            printf("Error getting real time stats\n");
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void main_task(void* params)
{
    struct mive_global_event event = {0};
```

```
    struct mive_global_event tss_event = {0};
    struct mive_global_event tss_trip_event = {0};
    struct van_cd_changer_packet* cdc_packet;
    mive_tss_task_packet_t *tss_packet = malloc(sizeof(*tss_packet));
    mive_tss_task_packet_t *tss_trip_packet = malloc(sizeof(*tss_packet));
    int ret = 0;
    int count = 0;
    int i;
    int num_timer = 0;
    int num_timer_ms = 0;
    int audio_menu_open = 0;
    int audio_menu_setting = 0;
    QueueHandle_t main_queue = g_global_state.global_main_queue;
    QueueHandle_t tss_queue = g_global_state.global_tss_queue;
    QueueHandle_t van_queue = g_global_state.global_van_queue;
    QueueHandle_t uart_queue = g_global_state.global_uart_queue;
    adc_oneshot_unit_handle_t adc1_handle = g_global_state.adc_handle;
    adc_cali_handle_t cali_handle = g_global_state.cali_handle;

    esp_timer_handle_t timer_handle;
    esp_timer_handle_t timer_handle_oneshot;
    esp_timer_handle_t timer_handle_audio_timeout;

    uint32_t ret_num = 0;
    mive_van_packet_t* van_packet = NULL;
    mive_uart_queue_packet_t* uart_queue_packet = NULL;
    mive_uart_task_packet_t* uart_recv_packet = NULL;

#if (ESP32_BOARD_TYPE == PEZO)
    esp_rom_gpio_pad_select_gpio(PSA_EXT_REG_PIN);
    esp_rom_gpio_pad_select_gpio(TSS_OE_ENABLE_PIN);

    gpio_set_direction(PSA_EXT_REG_PIN, GPIO_MODE_OUTPUT);
    gpio_set_direction(TSS_OE_ENABLE_PIN, GPIO_MODE_OUTPUT);
```

```
    gpio_set_level(PSA_EXT_REG_PIN, 1);
    gpio_set_level(TSS_OE_ENABLE_PIN, 1);

#endif


    tss_packet->iden = PSA_VAN_IDEN_CDCHANGER;
    tss_packet->packet_size = sizeof(*cdc_packet);
    tss_packet->message_type = TSS_IMM_REPLY;
    tss_packet->message_channel = 2;

    tss_trip_packet->iden = PSA_VAN_IDEN_MFD_STATUS; // 0x5E4
    tss_trip_packet->packet_size = 2;
    tss_trip_packet->message_type = TSS_TRANSMIT;
    tss_trip_packet->message_channel = 1;
    tss_trip_packet->packet[0] = 0x00;
    tss_trip_packet->packet[1] = 0xFF;

    cdc_packet = (struct van_cd_changer_packet*)tss_packet->packet;

    memset(cdc_packet, 0, sizeof(*cdc_packet));

    cdc_packet->cd_flag = 1;
    cdc_packet->cd_num = 1;
    cdc_packet->cd_present = VAN_CDC_CD_PRESENT;
    cdc_packet->header = 0x80;
    cdc_packet->footer = 0x80;
    cdc_packet->minutes = 1;
    cdc_packet->seconds = 1;
    cdc_packet->shuffle = 0;
    cdc_packet->status = VAN_CDC_ON_PLAYING;
    cdc_packet->total_tracks = 1;
```

```
cdc_packet->track_num = 1;

esp_timer_create_args_t timer_create_args = {
    .arg = g_global_state.global_main_queue,
    .callback = timer_callback_f,
    .dispatch_method = ESP_TIMER_ISR,
    .name = NULL,
    .skip_unhandled_events = true
};

esp_timer_create_args_t timer_create_args_oneshot = {
    .arg = g_global_state.global_main_queue,
    .callback = timer_callback_oneshot_f,
    .dispatch_method = ESP_TIMER_ISR,
    .name = NULL,
    .skip_unhandled_events = true
};

esp_timer_create_args_t timer_create_args_audio_menu = {
    .arg = g_global_state.global_main_queue,
    .callback = timer_callback_audio_timeout_f,
    .dispatch_method = ESP_TIMER_ISR,
    .name = NULL,
    .skip_unhandled_events = true
};

esp_timer_create(&timer_create_args, &timer_handle);
esp_timer_create(&timer_create_args_oneshot, &timer_handle_oneshot);
esp_timer_create(&timer_create_args_audio_menu, &timer_handle_audio_timeout);
esp_timer_start_periodic(timer_handle, 1000000);
esp_timer_start_once(timer_handle_oneshot, 1000000);

int adc_raw;
int voltage_in;
```

```
float voltage_out;
while(1)
{
  // vTaskDelay(1000 / portTICK_PERIOD_MS);

  ret = xQueueReceive(main_queue, &event, pdMS_TO_TICKS(1000));

  if(ret == pdPASS)
  {
    // printf("[%s] Received queue item\n", __func__);

    switch (event.event)
    {
    case MIVE_EVENT_RMT_NEW_VAN_FRAME:
      {
        int stat_idx = 0;
        van_packet = (mive_van_packet_t*)event.ev_data;

        ret = psa_parse_van_packet(
          van_packet->iden,
          van_packet->packet_size,
          van_packet->packet,
          &global_libpsa_buffers);

        if (ret != MIVE_OK)
        {
          ESP_LOGE(TAG, "Error parsing packet 0x%03x", van_packet->iden);
        }

        // calculate_car_state();
        // printf("[%s] Received: 0x%3x ", __func__, van_packet->iden);
        // for (int i = 0; i < van_packet->packet_size; i++)
        // {
        //   printf("%02x ", van_packet->packet[i]);
```

```
        // }
        // printf("\n");
    }

    vTaskDelay(pdMS_TO_TICKS(10));
    break;

case MIVE_EVENT_TIMER_1S:
    {
        uint8_t temp_val = cdc_packet->header;
        temp_val = (temp_val >= 0x87) ? 0x80 : temp_val + 1;
        cdc_packet->header = temp_val;
        cdc_packet->footer = temp_val;
        tss_event.event = MIVE_EVENT_TSS_WRITE_FRAME;
        tss_event.ev_data = tss_packet;
        xQueueSendToBack(tss_queue, &tss_event, pdMS_TO_TICKS(100));
    }
    break;
case MIVE_EVENT_TIMER_1MS:

    ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, ADC_CHANNEL_5, &adc_raw));
    ESP_ERROR_CHECK(adc_cali_raw_to_voltage(cali_handle, adc_raw, &voltage_in));

    g_bat_voltage = (voltage_in * 0.001f * (ADC_R1 + ADC_R2)) / ADC_R2;
    global_libpsa_buffers.status_data->voltage = (uint16_t)(g_bat_voltage * 1000);
    libpsa_send_packet(PSA_IDENT_CAR_STATUS);
    esp_timer_start_once(timer_handle_oneshot, 50000);

    if (unlikely(g_bat_voltage < 1))
    {
        struct mive_global_event sleep_event = {
            .ev_data = NULL,
            .event = MIVE_EVENT_GLOBAL_SLEEP
        };
```

```
            xQueueSendToFront(main_queue, &sleep_event, 0);
        }

        break;
    case MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM:
        if(!audio_menu_open)
        {
            esp_timer_start_once(timer_handle_audio_timeout, audio_menu_duration_ms * 1000);
        }
        if (audio_menu_setting >= 6)
        {
            audio_menu_open = 0;
            audio_menu_setting = 0;
            esp_timer_stop(timer_handle_audio_timeout);
        }
        else
        {
            audio_menu_open = 1;
            audio_menu_setting++;
            esp_timer_restart(timer_handle_audio_timeout, audio_menu_duration_ms * 1000);
        }
        ESP_LOGI(TAG, "[MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM] menu_open: %d, men
u_setting: %d" , audio_menu_open, audio_menu_setting);
        libpsa_update_audio_settings_packet(audio_menu_open, audio_menu_setting);
        libpsa_send_packet(PSA_IDENT_HEADUNIT);
        break;
    case MIVE_EVENT_VAN_UPDATE_AUDIO_MENU:
        if(audio_menu_open)
        {
            esp_timer_restart(timer_handle_audio_timeout, audio_menu_duration_ms * 1000);
            ESP_LOGI(TAG, "[MIVE_EVENT_VAN_UPDATE_AUDIO_MENU] menu_open: %d, menu
_setting: %d" , audio_menu_open, audio_menu_setting);
            libpsa_update_audio_settings_packet(audio_menu_open, audio_menu_setting);
```

```
            libpsa_send_packet(PSA_IDENT_HEADUNIT);
        }
        break;
    case MIVE_EVENT_VAN_CLOSE_AUDIO_MENU:
        audio_menu_open = 0;
        audio_menu_setting = 0;
        ESP_LOGI(TAG, "[MIVE_EVENT_VAN_CLOSE_AUDIO_MENU] menu_open: %d, menu_sett
ing: %d" , audio_menu_open, audio_menu_setting);
        libpsa_update_audio_settings_packet(audio_menu_open, audio_menu_setting);
        libpsa_send_packet(PSA_IDENT_HEADUNIT);
        break;
    case MIVE_EVENT_UART_NEW_DATA:
        uart_queue_packet = (mive_uart_queue_packet_t*)event.ev_data;

        for(i = 0; i < uart_queue_packet->num_idens; ++i)
        {
            libpsa_send_packet(uart_queue_packet->idens[i]);
        }
        break;
    case MIVE_EVENT_UART_RECEIVE:
        uart_recv_packet = (mive_uart_task_packet_t*)event.ev_data;

        if(uart_recv_packet->iden >= PSA_MSP_MIN_IDENT && uart_recv_packet->iden < PSA_IDEN
T_SET_CD_CHANGER_DATA)
        {
            ESP_LOGI(TAG, "Sending iden 0x%04x", uart_recv_packet->iden);
            libpsa_send_packet(uart_recv_packet->iden);
        } else if(uart_recv_packet->iden == PSA_IDENT_SET_TRIP_RESET)
        {
            ESP_LOGI(TAG, "Sending Trip reset");
            struct psa_trip_reset_data* data = (struct psa_trip_reset_data*)uart_recv_packet->data;
            tss_trip_event.event = MIVE_EVENT_TSS_WRITE_FRAME;
            tss_trip_event.ev_data = tss_trip_packet;
            tss_trip_packet->packet_size = 2;
```

```
            switch (data->trip_meter)
            {
            case PSA_TRIP_A:
               tss_trip_packet->packet[0] = 0xA0;
               xQueueSendToBack(tss_queue, &tss_trip_event, 0);
               break;
            case PSA_TRIP_B:
               tss_trip_packet->packet[0] = 0x60;
               xQueueSendToBack(tss_queue, &tss_trip_event, 0);
               break;
            default:
               break;
            }
         }

         break;
      case MIVE_EVENT_GLOBAL_SLEEP:
         memcpy(&rtc_preset_data[PSA_PRESET_AM], global_libpsa_buffers.presets_data_am, sizeof(str
uct psa_preset_data));
         memcpy(&rtc_preset_data[PSA_PRESET_FM_1], global_libpsa_buffers.presets_data_fm_1, sizeo
f(struct psa_preset_data));
         memcpy(&rtc_preset_data[PSA_PRESET_FM_2], global_libpsa_buffers.presets_data_fm_2, sizeo
f(struct psa_preset_data));
         memcpy(&rtc_preset_data[PSA_PRESET_FMAST], global_libpsa_buffers.presets_data_fm_ast, si
zeof(struct psa_preset_data));
         rtc_data_valid = 1;
         go_to_sleep();
         break;
      default:
         break;
      }

   }
```

```
        calculate_car_state();
    }
}

#include <stdint.h>

#include "include/mive/common.h"
#include "esp_attr.h"

static const unsigned char crc8tab[256] = {
  0x00, 0xD5, 0x7F, 0xAA, 0xFE, 0x2B, 0x81, 0x54,
  0x29, 0xFC, 0x56, 0x83, 0xD7, 0x02, 0xA8, 0x7D,
  0x52, 0x87, 0x2D, 0xF8, 0xAC, 0x79, 0xD3, 0x06,
  0x7B, 0xAE, 0x04, 0xD1, 0x85, 0x50, 0xFA, 0x2F,
  0xA4, 0x71, 0xDB, 0x0E, 0x5A, 0x8F, 0x25, 0xF0,
  0x8D, 0x58, 0xF2, 0x27, 0x73, 0xA6, 0x0C, 0xD9,
  0xF6, 0x23, 0x89, 0x5C, 0x08, 0xDD, 0x77, 0xA2,
  0xDF, 0x0A, 0xA0, 0x75, 0x21, 0xF4, 0x5E, 0x8B,
  0x9D, 0x48, 0xE2, 0x37, 0x63, 0xB6, 0x1C, 0xC9,
  0xB4, 0x61, 0xCB, 0x1E, 0x4A, 0x9F, 0x35, 0xE0,
  0xCF, 0x1A, 0xB0, 0x65, 0x31, 0xE4, 0x4E, 0x9B,
  0xE6, 0x33, 0x99, 0x4C, 0x18, 0xCD, 0x67, 0xB2,
  0x39, 0xEC, 0x46, 0x93, 0xC7, 0x12, 0xB8, 0x6D,
  0x10, 0xC5, 0x6F, 0xBA, 0xEE, 0x3B, 0x91, 0x44,
  0x6B, 0xBE, 0x14, 0xC1, 0x95, 0x40, 0xEA, 0x3F,
  0x42, 0x97, 0x3D, 0xE8, 0xBC, 0x69, 0xC3, 0x16,
  0xEF, 0x3A, 0x90, 0x45, 0x11, 0xC4, 0x6E, 0xBB,
  0xC6, 0x13, 0xB9, 0x6C, 0x38, 0xED, 0x47, 0x92,
  0xBD, 0x68, 0xC2, 0x17, 0x43, 0x96, 0x3C, 0xE9,
  0x94, 0x41, 0xEB, 0x3E, 0x6A, 0xBF, 0x15, 0xC0,
  0x4B, 0x9E, 0x34, 0xE1, 0xB5, 0x60, 0xCA, 0x1F,
  0x62, 0xB7, 0x1D, 0xC8, 0x9C, 0x49, 0xE3, 0x36,
  0x19, 0xCC, 0x66, 0xB3, 0xE7, 0x32, 0x98, 0x4D,
  0x30, 0xE5, 0x4F, 0x9A, 0xCE, 0x1B, 0xB1, 0x64,
```

```
  0x72, 0xA7, 0x0D, 0xD8, 0x8C, 0x59, 0xF3, 0x26,
  0x5B, 0x8E, 0x24, 0xF1, 0xA5, 0x70, 0xDA, 0x0F,
  0x20, 0xF5, 0x5F, 0x8A, 0xDE, 0x0B, 0xA1, 0x74,
  0x09, 0xDC, 0x76, 0xA3, 0xF7, 0x22, 0x88, 0x5D,
  0xD6, 0x03, 0xA9, 0x7C, 0x28, 0xFD, 0x57, 0x82,
  0xFF, 0x2A, 0x80, 0x55, 0x01, 0xD4, 0x7E, 0xAB,
  0x84, 0x51, 0xFB, 0x2E, 0x7A, 0xAF, 0x05, 0xD0,
  0xAD, 0x78, 0xD2, 0x07, 0x53, 0x86, 0x2C, 0xF9
};

/* Rounds a number to the nearest multiple */
IRAM_ATTR uint8_t round_to_nearest(uint8_t numToRound, uint8_t multiple)
{
    if (multiple == 0)
        return numToRound;

    uint8_t remainder = numToRound % multiple;

    // If the remainder is greater than half of the multiple, then round up. Otherwise, round down.
    if (remainder > multiple / 2)
        return numToRound + multiple - remainder;
    else
        return numToRound - remainder;
}

IRAM_ATTR uint16_t get_iden_from_bytes(uint8_t const byte1, uint8_t const byte2)
{
    uint16_t const iden = ((uint16_t)byte1 << 8) | (byte2 & 0xf0);
    return iden >> 4;
}

uint8_t dec_to_bcd(uint8_t input)
{
    if (input == 0xff)
```

```
    {
        return 0xff;
    }
    else if (input > 99)
    {
        return 0x99;
    }
    else
    {
        return ((input / 10 * 16) + (input % 10));
    }
}

uint8_t bcd_to_dec(uint8_t value)
{
    uint8_t ms_nibble = ((value & 0xF0) >> 4);
    uint8_t ls_nibble = (value & 0x0F);
    if (ms_nibble >= 10 || ls_nibble >= 10)
    {
        return value;
    }
    else
    {
        return ms_nibble * 10 + ls_nibble;
    }
}


uint8_t psa_crc8_checksum(const uint8_t * ptr, uint32_t len)
{
  uint8_t crc = 0;
  for (uint32_t i = 0; i < len; i++) {
    crc = crc8tab[crc ^ *ptr++];
  }
```

```
  return crc;
}

#include "freertos/FreeRTOS.h"
#include <stdint.h>

#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_continuous.h"
#include "esp_adc/adc_cali_scheme.h"

#include "include/global_init.h"
#include "include/mive/common.h"
#include "include/global_config.h"
#include "include/global_state_def.h"

#include "soc/soc_caps.h"


void init_adc()
{
    adc_oneshot_unit_handle_t adc1_handle;
    adc_cali_handle_t cali_handle;

    adc_oneshot_unit_init_cfg_t init_config1 = {
        .unit_id = PSA_ADC_UNIT,
        .ulp_mode = ADC_ULP_MODE_DISABLE,
    };

    adc_oneshot_chan_cfg_t config = {
        .bitwidth = ADC_BITWIDTH_12,
        .atten = ADC_ATTEN_DB_12,
    };

    adc_cali_line_fitting_config_t lf_config = {
```

```
        .atten = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_12,
        .default_vref = 0,
        .unit_id = PSA_ADC_UNIT
    };

    //ESP_ERROR_CHECK(adc_continuous_new_handle(&adc_config, &adc_c_handle));

    ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config1, &adc1_handle));

    ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL_5, &config));

    ESP_ERROR_CHECK(adc_cali_create_scheme_line_fitting(&lf_config, &cali_handle));

    g_global_state.adc_handle = adc1_handle;
    g_global_state.cali_handle = cali_handle;
}

#include "freertos/FreeRTOS.h"

#include <stdint.h>
#include <string.h>

#include "include/mive/psa_helper.h"
#include "include/van/van_packet_parser.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"

static int uart_send_buffer_num = 0;

static mive_uart_task_packet_t* get_uart_send_buffer(void)
{
    mive_uart_task_packet_t* to_ret = &global_uart_send_buffers[uart_send_buffer_num];
```

```
    uart_send_buffer_num = (uart_send_buffer_num >= (PSA_MAIN_UART_SEND_BUFFERS_NUM - 1))
? 0 : uart_send_buffer_num + 1;

    return to_ret;
}

void libpsa_send_packet(uint16_t ident)
{
    mive_uart_task_packet_t* packet = get_uart_send_buffer();
    struct mive_global_event global_event = {
        .event = MIVE_EVENT_UART_SEND,
        .ev_data = packet,
    };

    switch (ident)
    {
    case PSA_IDENT_ENGINE:
        packet->data_size = sizeof(*global_libpsa_buffers.engine_data);
        memcpy(
            packet->data,
            global_libpsa_buffers.engine_data,
            packet->data_size);
        packet->iden = ident;
        break;
    case PSA_IDENT_RADIO:
        packet->data_size = sizeof(*global_libpsa_buffers.radio_data);
        memcpy(
            packet->data,
            global_libpsa_buffers.radio_data,
            packet->data_size);
        packet->iden = ident;
        break;
    case PSA_IDENT_VIN:
        packet->data_size = 17;
```

```
    memcpy(
        packet->data,
        global_libpsa_buffers.vin_data,
        packet->data_size);
    packet->iden = ident;
    break;
case PSA_IDENT_RADIO_PRESETS:
    packet->data_size = sizeof(*global_libpsa_buffers.presets_data_am);
    switch(global_libpsa_buffers.radio_data->band)
    {
        case PSA_AM_1:
            memcpy(
                packet->data,
                global_libpsa_buffers.presets_data_am,
                packet->data_size);
            break;
        case PSA_FM_1:
            memcpy(
                packet->data,
                global_libpsa_buffers.presets_data_fm_1,
                packet->data_size);
            break;
        case PSA_FM_2:
            memcpy(
                packet->data,
                global_libpsa_buffers.presets_data_fm_2,
                packet->data_size);
            break;
        case PSA_FM_AST:
            memcpy(
                packet->data,
                global_libpsa_buffers.presets_data_fm_ast,
                packet->data_size);
            break;
```

```
        default:
            memset(packet->data, 0, packet->data_size);
            break;
    }
    packet->iden = ident;
    break;
case PSA_IDENT_CAR_STATUS:
    packet->data_size = sizeof(*global_libpsa_buffers.status_data);
    memcpy(
        packet->data,
        global_libpsa_buffers.status_data,
        packet->data_size);
    packet->iden = ident;
    global_libpsa_buffers.status_data->cd_changer_command = PSA_CD_CHANGER_COMM_NONE;
    break;
case PSA_IDENT_HEADUNIT:
    packet->data_size = sizeof(*global_libpsa_buffers.headunit_data);
    memcpy(
        packet->data,
        global_libpsa_buffers.headunit_data,
        packet->data_size
    );
    packet->iden = ident;
    break;
case PSA_IDENT_CD_PLAYER:
    packet->data_size = sizeof(*global_libpsa_buffers.cd_player_data);
    memcpy(
        packet->data,
        global_libpsa_buffers.cd_player_data,
        packet->data_size
    );
    packet->iden = ident;
    break;
case PSA_IDENT_DASHBOARD:
```

```
        packet->data_size = sizeof(*global_libpsa_buffers.dash_data);
        memcpy(
            packet->data,
            global_libpsa_buffers.dash_data,
            packet->data_size
        );
        packet->iden = ident;
        break;
    case PSA_IDENT_TRIP:
        packet->data_size = sizeof(*global_libpsa_buffers.trip_data);
        memcpy(
            packet->data,
            global_libpsa_buffers.trip_data,
            packet->data_size
        );
        packet->iden = ident;
        break;
    case PSA_IDENT_DOORS:
        packet->data_size = sizeof(*global_libpsa_buffers.door_data);
        memcpy(
            packet->data,
            global_libpsa_buffers.door_data,
            packet->data_size
        );
        packet->iden = ident;
        break;
    default:
        packet->iden = 0;
        packet->data_size = 0;
        break;
    }

    xQueueSendToBack(g_global_state.global_uart_queue, &global_event, 0);
}
```

```
void init_libpsa_packets(void)
{
    memset(&global_libpsa_buffers, 0, sizeof(global_libpsa_buffers));

    global_libpsa_buffers.engine_data = calloc(1, sizeof(struct psa_engine_data));
    global_libpsa_buffers.radio_data = calloc(1, sizeof(struct psa_radio_data));
    global_libpsa_buffers.presets_data_am = calloc(1, sizeof(struct psa_preset_data));
    global_libpsa_buffers.presets_data_fm_1 = calloc(1, sizeof(struct psa_preset_data));
    global_libpsa_buffers.presets_data_fm_2 = calloc(1, sizeof(struct psa_preset_data));
    global_libpsa_buffers.presets_data_fm_ast = calloc(1, sizeof(struct psa_preset_data));
    global_libpsa_buffers.headunit_data = calloc(1, sizeof(struct psa_headunit_data));
    global_libpsa_buffers.cd_player_data = calloc(1, sizeof(struct psa_cd_player_data));
    global_libpsa_buffers.vin_data = calloc(1, 17);
    global_libpsa_buffers.dash_data = calloc(1, sizeof(struct psa_dash_data));
    global_libpsa_buffers.door_data = calloc(1, sizeof(struct psa_door_data));
    global_libpsa_buffers.trip_data = calloc(1, sizeof(struct psa_trip_data));
    global_libpsa_buffers.status_data = calloc(1, sizeof(struct psa_status_data));

    global_uart_send_buffers = calloc(PSA_MAIN_UART_SEND_BUFFERS_NUM, sizeof(*global_uart_se
nd_buffers));
    global_uart_receive_buffers = calloc(PSA_MAIN_UART_SEND_BUFFERS_NUM, sizeof(*global_uart_
send_buffers));
}

void libpsa_update_audio_settings_packet(
    int audio_menu_open,
    int audio_menu_setting)
{
    struct psa_headunit_data *audio_settings_output_buffer = global_libpsa_buffers.headunit_data;
    audio_settings_output_buffer->settings_menu_open = audio_menu_open;
    switch (audio_menu_setting)
    {
    case 0:
```

```
    audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
    audio_settings_output_buffer->settings_changing.balance_changing = 0;
    audio_settings_output_buffer->settings_changing.bass_changing = 0;
    audio_settings_output_buffer->settings_changing.fader_changing = 0;
    audio_settings_output_buffer->settings_changing.loudness_changing = 0;
    audio_settings_output_buffer->settings_changing.treble_changing = 0;
    audio_settings_output_buffer->settings_changing.volume_changing = 0;

    break;
case 1: // Bass
    audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
    audio_settings_output_buffer->settings_changing.balance_changing = 0;
    audio_settings_output_buffer->settings_changing.bass_changing = 1;
    audio_settings_output_buffer->settings_changing.fader_changing = 0;
    audio_settings_output_buffer->settings_changing.loudness_changing = 0;
    audio_settings_output_buffer->settings_changing.treble_changing = 0;
    audio_settings_output_buffer->settings_changing.volume_changing = 0;
    break;

case 2: // Treble
    audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
    audio_settings_output_buffer->settings_changing.balance_changing = 0;
    audio_settings_output_buffer->settings_changing.bass_changing = 0;
    audio_settings_output_buffer->settings_changing.fader_changing = 0;
    audio_settings_output_buffer->settings_changing.loudness_changing = 0;
    audio_settings_output_buffer->settings_changing.treble_changing = 1;
    audio_settings_output_buffer->settings_changing.volume_changing = 1;
    break;

case 3: // Loudness
    audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
    audio_settings_output_buffer->settings_changing.balance_changing = 0;
    audio_settings_output_buffer->settings_changing.bass_changing = 0;
```

```
    audio_settings_output_buffer->settings_changing.fader_changing = 0;
    audio_settings_output_buffer->settings_changing.loudness_changing = 1;
    audio_settings_output_buffer->settings_changing.treble_changing = 0;
    audio_settings_output_buffer->settings_changing.volume_changing = 0;
    break;

  case 4: // Fader
    audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
    audio_settings_output_buffer->settings_changing.balance_changing = 0;
    audio_settings_output_buffer->settings_changing.bass_changing = 0;
    audio_settings_output_buffer->settings_changing.fader_changing = 1;
    audio_settings_output_buffer->settings_changing.loudness_changing = 0;
    audio_settings_output_buffer->settings_changing.treble_changing = 0;
    audio_settings_output_buffer->settings_changing.volume_changing = 0;
    break;
  case 5: // Balance

    audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
    audio_settings_output_buffer->settings_changing.balance_changing = 1;
    audio_settings_output_buffer->settings_changing.bass_changing = 0;
    audio_settings_output_buffer->settings_changing.fader_changing = 0;
    audio_settings_output_buffer->settings_changing.loudness_changing = 0;
    audio_settings_output_buffer->settings_changing.treble_changing = 0;
    audio_settings_output_buffer->settings_changing.volume_changing = 0;
    break;
  case 6: // Auto-volume
    audio_settings_output_buffer->settings_changing.auto_volume_changing = 1;
    audio_settings_output_buffer->settings_changing.balance_changing = 0;
    audio_settings_output_buffer->settings_changing.bass_changing = 0;
    audio_settings_output_buffer->settings_changing.fader_changing = 0;
    audio_settings_output_buffer->settings_changing.loudness_changing = 0;
    audio_settings_output_buffer->settings_changing.treble_changing = 0;
    audio_settings_output_buffer->settings_changing.volume_changing = 0;
    break;
```

```
    default:
        break;
    }
}

#include "freertos/FreeRTOS.h"

#include <string.h>
#include <stdlib.h>

#include "esp_attr.h"
#include "freertos/queue.h"

#include "include/mive/common.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"
#include "include/global_config.h"

#include "include/van/tss463c.h"
#include "include/van/tss463_regs.h"
#include "include/van/van_packet_iden.h"
#include "esp_log.h"

tss_instance_t global_tss_instance = {0};

static const char *TAG = "tss_task";

enum tss_transmission_status
{
    MIVE_TSS_TX_DONE = 0,
    MIVE_TSS_TX_INPROGRESS,
    MIVE_TSS_TX_FAIL,
};
```

```
static uint8_t tss_get_memory_addr(uint16_t iden)
{
   switch (iden)
   {
   case PSA_VAN_IDEN_MFD_STATUS:
      return 00;
      break;
   case PSA_VAN_IDEN_CDCHANGER:
      return 10;
      break;
   default:
      return 90;
      break;
   }
}

static uint8_t tss_get_channel_to_use(uint16_t iden)
{
   switch (iden)
   {
   case PSA_VAN_IDEN_MFD_STATUS:
      return 1;
      break;
   case PSA_VAN_IDEN_CDCHANGER:
      return 2;
      break;
   default:
      return 7;
      break;
   }
}

static int tss_send_frame(tss_instance_t* instance, mive_tss_task_packet_t* packet)
{
```

```
int retval = MIVE_OK;
uint16_t iden = packet->iden;
uint8_t memory_addr = tss_get_memory_addr(iden);
uint8_t channel = tss_get_channel_to_use(iden);
enum tss_message_type msg_type = packet->message_type;

switch (msg_type)
{
case TSS_TRANSMIT_NOACK:
   retval = tss_transmit_message(
      instance, channel, iden, packet->packet, packet->packet_size,
      memory_addr, 0);
   break;

case TSS_TRANSMIT:
   retval = tss_transmit_message(
      instance, channel, iden, packet->packet, packet->packet_size,
      memory_addr, 1);
   break;

case TSS_IMM_REPLY:
   retval = tss_immediate_reply_message(
      instance, channel, iden, packet->packet, packet->packet_size,
      memory_addr);
   break;

case TSS_DEF_REPLY:
case TSS_REPLY_REQUEST:
   retval = -MIVE_ERR_NOT_IMPLEMENTED;
   break;
case TSS_NONE:
default:
   retval = -MIVE_ERR_INVALID_ARGUMENT;
   break;
```

```
    }

    ESP_LOGD(TAG, "[%s] Retval = %d", __func__, retval);

    if(retval == MIVE_OK)
    {
        packet->message_channel = channel;
    }
    else if(retval == -MIVE_ERR_CHANNEL_BUSY)
    {
        // Do nothing
        retval = MIVE_OK;
    }
    else
    {
        packet->message_channel = 0xff;
    }

    return retval;
}

static int tss_check_channel_status(tss_instance_t* instance, mive_tss_task_packet_t* packet)
{
    int retval = -MIVE_ERR_INVALID_ARGUMENT;

    uint8_t channel = packet->message_channel;
    message_length_and_status_register_t reg_value = {0};

    reg_value.Value = tss_get_channel_status_register(instance, channel);

    switch (packet->message_type)
    {
    case TSS_TRANSMIT:
```

```
    case TSS_TRANSMIT_NOACK:
    case TSS_IMM_REPLY:
    case TSS_DEF_REPLY:
        if(reg_value.data.CHTx)
        {
            retval = MIVE_TSS_TX_DONE;
        }
        else
        {
            retval = MIVE_TSS_TX_INPROGRESS;
        }
        break;
    case TSS_REPLY_REQUEST:
        if(reg_value.data.CHRx)
        {
            retval = MIVE_TSS_TX_DONE;
        }
        else
        {
            retval = MIVE_TSS_TX_INPROGRESS;
        }
        break;
    default:
        retval = -MIVE_ERR_INVALID_ARGUMENT;
        break;
    }

    if(retval == MIVE_TSS_TX_DONE)
    {
        tss_free_channel(instance, channel);
    }

    return retval;
}
```

```
void tss_task(void* params)
{
    int task_running = true;
    int ret = 0;
    mive_global_state_t* state = (mive_global_state_t*) params;
    tss_instance_t* instance = &global_tss_instance;
    struct mive_global_event event_data = {0};
    mive_tss_task_packet_t *task_packet = NULL;
    QueueHandle_t tss_queue = state->global_tss_queue;

    tss_create(instance);

    tss_start(instance);

    while (task_running)
    {
        ret = xQueueReceive(tss_queue, &event_data, pdMS_TO_TICKS(100));
        if(ret == pdPASS)
        {
            ESP_LOGD(TAG, "[%s] Got event from queue: %d", __func__, event_data.event);
            switch (event_data.event)
            {
            case MIVE_EVENT_TSS_WRITE_FRAME:
                ret = tss_send_frame(instance, (mive_tss_task_packet_t*)event_data.ev_data);
                // if(ret == MIVE_OK)
                // {
                //     event_data.event = MIVE_EVENT_TSS_MONITOR_CHANNEL;
                //     xQueueSendToBack(tss_queue, &event_data, 0);
                // }
                break;


            case MIVE_EVENT_TSS_MONITOR_CHANNEL:
```

```
            ret = tss_check_channel_status(instance, (mive_tss_task_packet_t*)event_data.ev_data);
            if(ret == MIVE_TSS_TX_INPROGRESS)
            {
               // Send back to queue
               xQueueSendToBack(tss_queue, &event_data, 0);
            }
            break;

         case MIVE_EVENT_TSS_RESET:
            tss_start(instance);
            break;

         case MIVE_EVENT_TSS_ACTIVATE:
            tss_activate(instance);
            break;

         case MIVE_EVENT_TSS_IDLE:
            tss_idle(instance);
            break;

         case MIVE_EVENT_TSS_SLEEP:
            tss_sleep(instance);
            break;
         default:
            break;
         }
      }
   }
}

#include "freertos/FreeRTOS.h"

#include <unistd.h>
#include <string.h>
```

```c
#include <stdbool.h>

#include "include/van/tss463_regs.h"
#include "include/van/tss463c.h"
#include "esp_log.h"

#include "include/mive/common.h"

#include "driver/spi_master.h"
#include "driver/gpio.h"

#include "include/global_config.h"

static const char *TAG = "tss463c";

static portMUX_TYPE tss_spinlock = portMUX_INITIALIZER_UNLOCKED;

#define get_bytes_from_iden(iden, byte1, byte2) \
    byte1 = (uint8_t)(((iden << 4) & 0xff00) >> 8); \
    byte2 = (uint8_t)(iden & 0xF);

IRAM_ATTR static void pre_transaction_cb(spi_transaction_t *trans)
{
    return;
}

IRAM_ATTR static void post_transaction_cb(spi_transaction_t *trans)
{
    return;
}

// IRAM_ATTR static void get_bytes_from_iden(uint16_t iden, uint8_t* byte1, uint8_t* byte2)
// {
```

```
//     *byte1 = (uint8_t)(((iden << 4) & 0xff00) >> 8);
//     *byte2 = (uint8_t)(iden & 0xF);
// }

IRAM_ATTR static int is_channel_valid(tss_instance_t* instance, uint8_t channel, uint16_t iden)
{
    tss_channel_t *chan = NULL;

    if(channel > TSS_MAX_CHANNEL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    // chan = &instance->channels[channel];

    // if((!chan->is_busy) && ((chan->is_occupied == 0) || (chan->is_occupied && chan->identifier == ide
n)))
    // {
    //     return MIVE_OK;
    // }

    // return -MIVE_ERR_CHANNEL_BUSY;
    return MIVE_OK;
}

static int tss_spi_master_init(tss_instance_t *instance)
{
    spi_device_handle_t tss_handle;

    #if (ESP32_BOARD_TYPE == DEVKIT)

    spi_bus_config_t spi_config = {
        .mosi_io_num = 23,
```

```
    .miso_io_num = 19,
    .sclk_io_num = 18,
    .max_transfer_sz = 0,
    .data2_io_num = -1,
    .data3_io_num = -1,
    .data4_io_num = -1,
    .data5_io_num = -1,
    .data6_io_num = -1,
    .data7_io_num = -1
};

#else

spi_bus_config_t spi_config = {
    .mosi_io_num = 13,
    .miso_io_num = 12,
    .sclk_io_num = 14,
    .max_transfer_sz = 0,
    .data2_io_num = -1,
    .data3_io_num = -1,
    .data4_io_num = -1,
    .data5_io_num = -1,
    .data6_io_num = -1,
    .data7_io_num = -1
};

#endif

spi_device_interface_config_t device_config = {
    .clock_speed_hz = 4000000,
    .mode = 3,
    .spics_io_num = -1,
    .queue_size = 1,
    .pre_cb = pre_transaction_cb,
```

```
        .post_cb = post_transaction_cb,

    };

    ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &spi_config, SPI_DMA_CH_AUTO));

    ESP_ERROR_CHECK(spi_bus_add_device(SPI2_HOST, &device_config, &tss_handle));

    instance->tss_handle = tss_handle;

    return 0;
}

/**
 * @brief Performs an Asynchronous Reset and puts the TSS into motorolla mode
 *
 * @param instance
 * @return int
 */
static int tss_motorolla_mode(tss_instance_t *instance)
{
    int return_val = 0;
    // gpio_set_level(instance->cs_pin, 0);
    spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
    TSS_SELECT();

    usleep(1);
    spi_transaction_t transaction;
    memset(&transaction, 0, sizeof(transaction));
    transaction.user = instance;
    transaction.tx_data[0] = 0x00;
    transaction.length = 8;
    transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
```

```
ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0xAA)
{
   printf("Error setting motorolla mode: Invalid response 0x%02X / 0xAA\n", transaction.rx_data[0]);
   return_val = 1;
   // goto error;
}

usleep(4);

transaction.user = instance;
transaction.tx_data[0] = 0x00;
transaction.length = 8;
transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;

ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0x55)
{
   printf("Error setting motorolla mode: Invalid response 0x%02X / 0x55\n", transaction.rx_data[0]);
   return_val = 1;
   // goto error;
}

usleep(2);

// error:
// gpio_set_level(instance->cs_pin, 1);
TSS_DESELECT();
spi_device_release_bus(instance->tss_handle);
```

```
    instance->chip_mode = TSS_MODE_IDLE;

    return return_val;
}

static int tss_disable_channel(tss_instance_t *instance, uint8_t channel_id)
{
    ESP_LOGI(TAG, "[%s] Disabling channel %d", __func__, channel_id);

    int return_val = 0;
    if(channel_id > TSS_MAX_CHANNEL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    /*
     * Datasheet pg. 37
     * The easiest way to disable an channel register is to set the
     * received and transmitted bits to 1 in the Message Length and Status Registe
     */
    static const uint8_t data[] = {0x00, 0x00, 0x00, 0x0F, 0x00, 0x00, 0x00, 0x00};

    return_val = tss_registers_set(instance, TSS_CHANNEL_ADDR(channel_id), data, sizeof(data));

    memset(&instance->channels[channel_id], 0, sizeof(instance->channels[channel_id]));

    return return_val;
}

/**
 * @brief Put the TSS in active mode
 *
 * @param instance
```

```
 * @return int
 */
int tss_activate(tss_instance_t *instance)
{
    // Sending the Activate command
    command_register_t comm_register = {0};
    comm_register.data.ACTI = 1;

    if (tss_register_set(instance, TSS_COMMANDREGISTER, comm_register.Value) != MIVE_OK)
    {
        return 1;
    }

    instance->chip_mode = TSS_MODE_ACTIVE;

    return 0;
}

/**
 * @brief Put the TSS in sleep mode
 *
 * @param instance
 * @return int
 */
int tss_sleep(tss_instance_t *instance)
{
    // Sending the Sleep command
    command_register_t comm_register = {0};
    comm_register.data.SLEEP = 1;

    if (tss_register_set(instance, TSS_COMMANDREGISTER, comm_register.Value))
    {
        return 1;
    }
```

```
    instance->chip_mode = TSS_MODE_SLEEP;

    return 0;
}

/**
 * @brief Put the TSS in idle mode
 *
 * @param instance
 * @return int
 */
int tss_idle(tss_instance_t *instance)
{
    // Sending the Idle command
    command_register_t comm_register = {0};
    comm_register.data.IDLE = 1;

    if (tss_register_set(instance, TSS_COMMANDREGISTER, comm_register.Value))
    {
        return 1;
    }

    instance->chip_mode = TSS_MODE_IDLE;

    return 0;
}

IRAM_ATTR int tss_register_set(
    tss_instance_t *instance,
    uint8_t reg_addr,
    uint8_t reg_value)
{
```

```
int return_val = 0;
spi_transaction_t transaction = {0};

if(unlikely(instance->chip_mode == TSS_MODE_SLEEP))
{
   ESP_LOGE(TAG, "[%s] Chip in sleep mode", __func__);
   return -1;
}

xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);

// gpio_set_level(instance->cs_pin, 0);
TSS_SELECT();
transaction.user = instance;
transaction.tx_data[0] = reg_addr;
transaction.length = 8;
transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;

usleep(1);

ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0xAA)
{
   printf("Error setting regiser: Invalid response 0x%02x / 0xAA\n", transaction.rx_data[0]);
   return_val = 1;
   goto error;
}

transaction.tx_data[0] = TSS_WRITE;
transaction.length = 8;
transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
```

```
    usleep(2);
    ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

    if (transaction.rx_data[0] != 0x55)
    {
        printf("Error setting regiser: Invalid response 0x%02x / 0x55\n", transaction.rx_data[0]);
        return_val = 1;
        goto error;
    }

    usleep(4);

    transaction.tx_data[0] = reg_value;
    transaction.length = 8;
    transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;

    ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

error:
    TSS_DESELECT();
    spi_device_release_bus(instance->tss_handle);
    xSemaphoreGive(instance->tss_spi_semaphore);

    return return_val;
}

IRAM_ATTR int tss_registers_set(
    tss_instance_t *const instance,
    uint8_t const reg_addr,
    uint8_t const *const values,
    uint8_t const count)
{
```

```
unsigned int i = 0;
int return_val = 0;
spi_transaction_t transaction = {0};

xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
// printf("Setting %d registers starting from addr %d\n", count, reg_addr);
spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
TSS_SELECT();

transaction.user = instance;
transaction.tx_data[0] = reg_addr;
transaction.length = 8;
transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;

usleep(1);
spi_device_transmit(instance->tss_handle, &transaction);
if (transaction.rx_data[0] != 0xAA)
{
   printf("Error setting regiser: Invalid response 0x%02x / 0xAA\n", transaction.rx_data[0]);
   return_val = 1;
   goto error;
}

transaction.tx_data[0] = TSS_WRITE;

usleep(2);
spi_device_transmit(instance->tss_handle, &transaction);

if (transaction.rx_data[0] != 0x55)
{
   printf("Error setting regiser: Invalid response 0x%02x / 0x55\n", transaction.rx_data[0]);
   return_val = 1;
   goto error;
}
```

```
    usleep(4);

    // printf("Transmitting: ");
    for (i = 0; i < count; ++i)
    {
        transaction.tx_data[0] = values[i];
        spi_device_transmit(instance->tss_handle, &transaction);
        printf("%02x ", transaction.tx_data[0]);
        usleep(3);
    }

    printf("\n");

    usleep(3);

error:

    TSS_DESELECT();
    spi_device_release_bus(instance->tss_handle);

    xSemaphoreGive(instance->tss_spi_semaphore);

    return return_val;
}

IRAM_ATTR int tss_register_get(
    tss_instance_t *const instance,
    uint8_t const reg_addr,
    uint8_t *const reg_value)
{
    int return_val = 0;
    spi_transaction_t transaction = {0};
```

```
xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);

// gpio_set_level(instance->cs_pin, 0);
TSS_SELECT();
transaction.user = instance;
transaction.tx_data[0] = reg_addr;
transaction.length = 8;
transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;

usleep(1);

ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0xAA)
{
   printf("Error getting regiser: Invalid response 0x%02x / 0xAA\n", transaction.rx_data[0]);
   return_val = 1;
   goto error;
}

transaction.tx_data[0] = TSS_READ;

usleep(2);
ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0x55)
{
   printf("Error getting regiser: Invalid response 0x%02x / 0x55\n", transaction.rx_data[0]);
   return_val = 1;
   goto error;
}
```

```
    usleep(4);

    transaction.tx_data[0] = 0xFF;
    ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

    *reg_value = transaction.rx_data[0];
error:
    TSS_DESELECT();
    spi_device_release_bus(instance->tss_handle);

    xSemaphoreGive(instance->tss_spi_semaphore);

    return return_val;
}

IRAM_ATTR int tss_registers_get(
    tss_instance_t *const instance,
    uint8_t const reg_addr,
    uint8_t *const buffer,
    uint8_t count)
{
    int return_val = 0;
    spi_transaction_t transaction = {0};

    if (unlikely(buffer == NULL))
    {
        return 1;
    }

    if(unlikely(instance->chip_mode == TSS_MODE_SLEEP))
    {
        ESP_LOGE(TAG, "[%s] Chip in sleep mode", __func__);
```

```
    return -1;
}

xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);

// gpio_set_level(instance->cs_pin, 0);
TSS_SELECT();
memset(&transaction, 0, sizeof(transaction));
transaction.user = instance;
transaction.tx_data[0] = reg_addr;
transaction.length = 8;
transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;

usleep(1);

ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0xAA)
{
    printf("Error setting regiser: Invalid response 0x%02x / 0xAA\n", transaction.rx_data[0]);
    return_val = 1;
    goto error;
}

transaction.tx_data[0] = TSS_READ;

usleep(2);
ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));

if (transaction.rx_data[0] != 0x55)
{
    printf("Error setting regiser: Invalid response 0x%02x / 0x55\n", transaction.rx_data[0]);
```

```
        return_val = 1;
        goto error;
    }

    usleep(4);

    for (int i = 0; i < count; ++i)
    {

        transaction.tx_data[0] = 0xFF;
        ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
        buffer[i] = transaction.rx_data[0];
        usleep(3);
    }

error:
    TSS_DESELECT();
    spi_device_release_bus(instance->tss_handle);

    xSemaphoreGive(instance->tss_spi_semaphore);

    return return_val;
}

uint8_t tss_get_channel_status_register(
    tss_instance_t* const instance,
    uint8_t const channel_num)
{
    uint8_t reg_value;
    if(channel_num > TSS_MAX_CHANNEL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }
```

```
    tss_register_get(instance, TSS_CHANNEL_MESS_LEN_AND_STATUS(channel_num), &reg_value);

    instance->channels[channel_num].message_len_and_status_reg_value = reg_value;

    return reg_value;
}

uint8_t tss_set_channel_status_register(
    tss_instance_t* const instance,
    uint8_t const channel_num,
    uint8_t const reg_value)
{
    uint8_t new_reg_value;
    if(channel_num > TSS_MAX_CHANNEL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }
    tss_register_set(instance, TSS_CHANNEL_MESS_LEN_AND_STATUS(channel_num), reg_value);

    tss_register_get(instance, TSS_CHANNEL_MESS_LEN_AND_STATUS(channel_num), &new_reg_valu
e);

    instance->channels[channel_num].message_len_and_status_reg_value = new_reg_value;

    return new_reg_value;
}

int tss_abort_channel(
    tss_instance_t* const instance,
    uint8_t const channel_num)
{
    message_length_and_status_register_t reg;
```

```
    reg.Value = tss_get_channel_status_register(instance, channel_num);
    reg.data.CHER = 1;

    tss_set_channel_status_register(instance, channel_num, reg.Value);

    return MIVE_OK;
}

int tss_free_channel(
    tss_instance_t* const instance,
    uint8_t const channel_num)
{
    tss_channel_t* channel = &instance->channels[channel_num];

    channel->is_busy = false;
    tss_disable_channel(instance, channel_num);
    return MIVE_OK;
}

int tss_create(tss_instance_t* instance)
{
    if (unlikely(instance == NULL))
    {
        printf("tss_start -> instance is NULL\n");
        return 1;
    }

    memset(instance, 0, sizeof(*instance));

    if (tss_spi_master_init(instance))
    {
        return 1;
    }
```

```
    instance->cs_pin = TSS_CS_PIN;
    instance->buffers_size = 256;
    instance->interrupt_pin = TSS_INT_PIN;
    instance->tss_spi_semaphore = xSemaphoreCreateMutex();
    instance->chip_mode = TSS_MODE_IDLE;

    xSemaphoreGive(instance->tss_spi_semaphore);

    gpio_config_t io_conf = {
        .intr_type = GPIO_INTR_DISABLE,
        .mode = GPIO_MODE_OUTPUT_OD,
        .pin_bit_mask = (1 << instance->cs_pin),
        .pull_up_en = GPIO_PULLUP_ENABLE,
    };

    // Configure CS pin as output
    gpio_config(&io_conf);

    TSS_DESELECT();

    instance->tx_buffer = heap_caps_calloc(1, instance->buffers_size, MALLOC_CAP_DMA);
    instance->rx_buffer = heap_caps_calloc(1, instance->buffers_size, MALLOC_CAP_DMA);

    return 0;
}

int tss_start(tss_instance_t *instance)
{

    if (tss_motorolla_mode(instance))
    {
        // return 1;
```

```
}
usleep(3);

for (uint8_t i = 0; i < TSS_MAX_CHANNEL; i++)
{
   tss_disable_channel(instance, i);
}

if (tss_register_set(instance, TSS_LINECONTROL, TSS_8MHz_125kBPS))
{
   return 1;
}

transmit_control_register_t tc_register;
tc_register.Value = 0;
tc_register.data.max_retries = 5;
tc_register.data.VER = 1;
tc_register.data.module_type = 1;

if (tss_register_set(instance, TSS_TRANSMITCONTROL, tc_register.Value))
{
   return 1;
}

// Select interrupts to enable
interrupt_register_t it_register;
it_register.Value = 0;
it_register.data.RESET = 1; // Must be 1
it_register.data.TOK = 0;   // Change this to enable interrupt on successful transmissions
it_register.data.ROK = 0;   // Change this to enable interrupt on reception with RAK
it_register.data.RNOK = 0;  // Change this to enable interrupt on reception without RAK
it_register.data.RE = 0;    // Change this to enable interrupt on reception error
it_register.data.TE = 0;    // Change this to enable interrupt on transmission error
```

```
if (tss_register_set(instance, TSS_INTERRUPTENABLE, it_register.Value))
{
   return 1;
}

tss_register_get(instance, TSS_INTERRUPTSTATUS, &(it_register.Value));

printf("\tRNOK: %d\n", it_register.data.RNOK);
printf("\tROK: %d\n", it_register.data.ROK);
printf("\tRE: %d\n", it_register.data.RE);
printf("\tTOK: %d\n", it_register.data.TOK);
printf("\tTE: %d\n", it_register.data.TE);
printf("\tRESET: %d\n", it_register.data.RESET);

// Reset all interrupt statuses
it_register.data.RESET = 1;
it_register.data.TOK = 1;
it_register.data.ROK = 1;
it_register.data.RNOK = 1;
it_register.data.RE = 1;
it_register.data.TE = 1;

if (tss_register_set(instance, TSS_INTERRUPTRESET, it_register.Value))
{
   return 1;
}

// Memsetting the TSS's RAM to zeros
uint8_t zeroarray[128] = {0};
if (tss_registers_set(instance, TSS_GETMAIL(0), zeroarray, 128))
{
   return 1;
}
```

```
    it_register.Value = 0;
    it_register.data.RESET = 1; // Must be 1
    it_register.data.TOK = 1;   // Change this to enable interrupt on successful transmissions
    it_register.data.ROK = 1;   // Change this to enable interrupt on reception with RAK
    it_register.data.RNOK = 1;  // Change this to enable interrupt on reception without RAK
    it_register.data.RE = 1;    // Change this to enable interrupt on reception error
    it_register.data.TE = 1;    // Change this to enable interrupt on transmission error

    if (tss_register_set(instance, TSS_INTERRUPTENABLE, it_register.Value))
    {
        return 1;
    }

    if (tss_activate(instance))
    {
        return 1;
    }

    return 0;
}

int tss_transmit_message(
    tss_instance_t* const instance,
    uint8_t channel,
    uint16_t const iden,
    uint8_t *const data,
    uint8_t const data_size,
    uint8_t const memory_offset,
    uint8_t const rak)
{
    uint8_t id1 = 0;
    uint8_t id2 = 0;
    uint8_t memory_address;
    id2_and_command_register_t id2_reg = {0};
```

```
message_pointer_register_t mp_reg = {0};
message_length_and_status_register_t mls_reg = {0};
uint8_t channel_data[8] = {0};
tss_channel_t* chan = NULL;
int retval;

retval = is_channel_valid(instance, channel, iden);

if(retval != MIVE_OK)
{
   printf("[%s] Invalid channel %d (%d)\n", __func__, channel, retval);
   return retval;
}
if(memory_offset & 0x80)
{
   memory_address = memory_offset;
}
else
{
   memory_address = TSS_GETMAIL(memory_offset);
}

ESP_LOGD(TAG, "[%s] Using memory addr 0x%02x", __func__, memory_address);

get_bytes_from_iden(iden, id1, id2);



id2_reg.data.ID = id2;
id2_reg.data.RNW = 0;
id2_reg.data.RTR = 0;
id2_reg.data.EXT = 1;
id2_reg.data.RAK = rak;
```

```
    mp_reg.data.DRAK = 0;
    mp_reg.data.message_pointer = memory_address & 0x7F;

    mls_reg.data.CHTx = 0;
    mls_reg.data.M_L = data_size + 1;

    // When writing to the bus, the first element in the buffer is ignored,
    // so we skip one (memory_address + 1)
    tss_registers_set(instance, memory_address + 1, data, data_size);

    channel_data[0] = id1;
    channel_data[1] = id2_reg.Value;
    channel_data[2] = mp_reg.Value;
    channel_data[3] = mls_reg.Value;
    channel_data[6] = id1;
    channel_data[7] = id2;

    tss_registers_set(instance, TSS_CHANNEL_ADDR(channel), channel_data, 8);

    chan = &instance->channels[channel];
    chan->data_size = data_size + 1;
    chan->ram_address = memory_address;
    chan->identifier = iden;
    chan->is_occupied = true;
    chan->is_busy = true;
    chan->message_len_and_status_reg_value = mls_reg.Value;
    chan->tx_attempt = 1;
    chan->message_type = TSS_TRANSMIT;

    return MIVE_OK;
}

int tss_immediate_reply_message(
```

```
    tss_instance_t* const instance,
    uint8_t channel,
    uint16_t const iden,
    uint8_t *const data,
    uint8_t const data_size,
    uint8_t const memory_offset)
{
    uint8_t id1 = 0;
    uint8_t id2 = 0;
    uint8_t memory_address;
    id2_and_command_register_t id2_reg = {0};
    message_pointer_register_t mp_reg = {0};
    message_length_and_status_register_t mls_reg = {0};
    uint8_t channel_data[8] = {0};
    tss_channel_t* chan = NULL;
    int retval;

    retval = is_channel_valid(instance, channel, iden);

    if(retval != MIVE_OK)
    {
        printf("[%s] Invalid channel %d (%d)\n", __func__, channel, retval);
        return retval;
    }

    if(memory_offset & 0x80)
    {
        memory_address = memory_offset;
    }
    else
    {
        memory_address = TSS_GETMAIL(memory_offset);
    }
```

```
ESP_LOGD(TAG, "[%s] Using memory addr 0x%02x", __func__, memory_address);

get_bytes_from_iden(iden, id1, id2);



id2_reg.data.ID = id2;
id2_reg.data.RNW = 1;
id2_reg.data.RTR = 0;
id2_reg.data.EXT = 1;
id2_reg.data.RAK = 0;

mp_reg.data.DRAK = 0;
mp_reg.data.message_pointer = memory_address & 0x7F;

mls_reg.data.CHTx = 0;
mls_reg.data.CHRx = 0;
mls_reg.data.M_L = data_size + 1;

// When writing to the bus, the first element in the buffer is ignored,
// so we skip one (memory_address + 1)
tss_registers_set(instance, memory_address + 1, data, data_size);

channel_data[0] = id1;
channel_data[1] = id2_reg.Value;
channel_data[2] = mp_reg.Value;
channel_data[3] = mls_reg.Value;
channel_data[6] = id1;
channel_data[7] = id2;

tss_registers_set(instance, TSS_CHANNEL_ADDR(channel), channel_data, 8);

chan = &instance->channels[channel];
chan->data_size = data_size + 1;
```

```
    chan->ram_address = memory_address;
    chan->identifier = iden;
    chan->is_occupied = true;
    chan->is_busy = true;
    chan->message_len_and_status_reg_value = mls_reg.Value;
    chan->tx_attempt = 1;
    chan->message_type = TSS_IMM_REPLY;

    return MIVE_OK;
}

#include "freertos/FreeRTOS.h"

#include <stdint.h>
#include <string.h>
#include <stdlib.h>

#include "include/global_config.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"

#include "include/mive/common.h"
#include "include/mive/psa_packet_defs.h"
#include "include/mive/psa_helper.h"

#include "driver/uart.h"
#include "esp_timer.h"
#include "esp_log.h"

static const char *TAG = "UART";
static int uart_recv_buffer_num = 0;

IRAM_ATTR static void uart_timer_callback_f(void* user_data)
{
```

```
    BaseType_t high_task_wakeup = pdFALSE;
    QueueHandle_t uart_queue = (QueueHandle_t)user_data;
    struct mive_global_event timer_event = {
        .ev_data = NULL,
        .event = MIVE_EVENT_TIMER_1MS,
    };
    xQueueSendFromISR(uart_queue, &timer_event, &high_task_wakeup);
    if(high_task_wakeup)
    {
        esp_timer_isr_dispatch_need_yield();
    }
}


static mive_uart_task_packet_t* get_uart_recv_buffer(void)
{
    mive_uart_task_packet_t* to_ret = &global_uart_receive_buffers[uart_recv_buffer_num];

    uart_recv_buffer_num = (uart_recv_buffer_num >= (PSA_MAIN_UART_SEND_BUFFERS_NUM - 1))
? 0 : uart_recv_buffer_num + 1;

    return to_ret;
}

static void uart_rx_task(void* params);

void uart_task(void* params)
{
    int ret = 0;
    const int uart_buffer_size = 2048;
    struct mive_global_event event = {0};
    QueueHandle_t main_queue = g_global_state.global_main_queue;
    QueueHandle_t tss_queue = g_global_state.global_tss_queue;
    QueueHandle_t van_queue = g_global_state.global_van_queue;
```

```
    QueueHandle_t uart_queue = g_global_state.global_uart_queue;
    QueueHandle_t uart_driver_queue;
    esp_timer_handle_t uart_timer_handle;
    mive_uart_task_packet_t* uart_packet = NULL;
    uint8_t* uart_buffer = malloc(256);
    uint8_t* uart_data_buffer = NULL;
    uint8_t uart_crc;
    struct psa_header* psa_packet_header = (struct psa_header*)uart_buffer;


    esp_timer_create_args_t uart_timer_create_args = {
        .arg = uart_queue,
        .callback = uart_timer_callback_f,
        .dispatch_method = ESP_TIMER_ISR,
        .name = NULL,
        .skip_unhandled_events = true
    };

    uart_config_t uart_config = {
        .baud_rate = 500000,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    };

    ESP_ERROR_CHECK(uart_driver_install(UART_NUM_2, uart_buffer_size, 0, 10, &uart_driver_queue,
0));
    ESP_ERROR_CHECK(uart_param_config(UART_NUM_2, &uart_config));
    ESP_ERROR_CHECK(uart_set_pin(UART_NUM_2, 4, 5, -1, -1));

    xTaskCreatePinnedToCore(
        uart_rx_task,
        "uart_rx_task",
```

```
    5120,
    &g_global_state, 11, NULL, xPortGetCoreID());

  esp_timer_create(&uart_timer_create_args, &uart_timer_handle);
  esp_timer_start_once(uart_timer_handle, 1000000);

  while(true)
  {
    ret = xQueueReceive(uart_queue, &event, pdMS_TO_TICKS(500));
    if(ret == pdPASS)
    {
      switch (event.event)
      {
      case MIVE_EVENT_TIMER_UART_SEND:
        // Send data
        esp_timer_start_once(uart_timer_handle, 1000);
        break;

      case MIVE_EVENT_UART_SEND:
        uart_packet = (mive_uart_task_packet_t*)event.ev_data;
        if(uart_packet->iden > PSA_MSP_MIN_IDENT && uart_packet->data_size < PSA_MSP_MAX_
SIZE)
        {
          uart_data_buffer = uart_buffer + sizeof(*psa_packet_header);

          psa_packet_header->ident = uart_packet->iden;
          psa_packet_header->direction = '>';
          psa_packet_header->size = uart_packet->data_size;
          psa_packet_header->start = PSA_MSP_START_MAGIC;

          memcpy(uart_data_buffer, uart_packet->data, uart_packet->data_size);

          uart_crc = psa_crc8_checksum(uart_buffer + 2u, uart_packet->data_size + 3u);
```

```
                uart_data_buffer[psa_packet_header->size] = uart_crc;

                uart_write_bytes(
                    UART_NUM_2,
                    uart_buffer,
                    uart_packet->data_size + sizeof(*psa_packet_header) + 1u);
            }
            vTaskDelay(1);
            break;
        default:
            break;
        }
    }
  }

  vTaskDelete(NULL);
  return;
}

void uart_rx_task(void* params)
{
  QueueHandle_t main_queue = g_global_state.global_main_queue;
  QueueHandle_t tss_queue = g_global_state.global_tss_queue;
  QueueHandle_t van_queue = g_global_state.global_van_queue;
  QueueHandle_t uart_queue = g_global_state.global_uart_queue;
  struct mive_global_event global_event = {
    .event = MIVE_EVENT_UART_RECEIVE,
    .ev_data = NULL,
  };
  uint8_t* rx_buffer = malloc(2048);
  uint8_t* data;
  uint8_t calc_crc;
  mive_uart_task_packet_t* task_packet;
```

```
    struct psa_header* header = (struct psa_header*)rx_buffer;
    data = rx_buffer + sizeof(*header);

    while(1)
    {
        int rxBytes = uart_read_bytes(UART_NUM_2, rx_buffer, 5, pdMS_TO_TICKS(1000));
        if(rxBytes > 0)
        {
            ESP_LOGD(TAG, "[%s] Got %d bytes", __func__, rxBytes);

            if(rxBytes == sizeof(*header))
            {
                rxBytes = uart_read_bytes(UART_NUM_2, data, header->size + 1, pdMS_TO_TICKS(5));
                calc_crc = psa_crc8_checksum(rx_buffer + 2u, header->size + 3u);
                if(calc_crc == data[header->size])
                {
                    ESP_LOGD(TAG, "[%s] Valid packet\n", __func__);
                    task_packet = get_uart_recv_buffer();
                    task_packet->iden = header->ident;
                    task_packet->data_size = header->size;
                    memcpy(task_packet->data, data, header->size);
                    global_event.ev_data = task_packet;
                    xQueueSendToBack(main_queue, &global_event, 0);
                }
            }
        }

    }

}

#include "freertos/FreeRTOS.h"
#include <stdio.h>
#include <stdint.h>
```

```c
#include <endian.h>
#include <string.h>

#include "include/mive/common.h"
#include "include/van/van_packet_iden.h"
#include "include/van/van_packet_parser.h"
#include "include/mive/common.h"
#include "include/mive/psa_packet_defs.h"
#include "include/van_structs.h"
#include "include/global_state_def.h"

#define PACKET_BUFFER_NUM 20

static int audio_menu_open = 0;
static int audio_menu_setting = 0; // 0, 1, 2, 3, 4, 5, 6

// static const int audio_menu_duration_ms = 4000;

mive_uart_queue_packet_t packet_buffers[PACKET_BUFFER_NUM];

static int packet_buffer_num = 0;

union dash_mileage
{
   uint32_t number : 24;
   uint8_t buffer[3];
} __attribute__((packed));

static uint16_t swap_endian_uint16(uint16_t const x)
{
   return (x << 8) | (x >> 8);
}
```

```
static uint32_t swap_endian_uint32(uint32_t const x)
{
   uint32_t val = ((x << 8) & 0xFF00FF00) | ((x >> 8) & 0xFF00FF);
   return (val << 16) | (val >> 16);
}

static uint32_t mileage_swap_bytes(union dash_mileage const mileage)
{
   uint32_t temp_mileage = (uint32_t)mileage.buffer[0] << 16 | (uint32_t)mileage.buffer[1] << 8 | mileage.bu
ffer[2];
   return temp_mileage;
}

static mive_uart_queue_packet_t* get_packet_buffer(void)
{
   mive_uart_queue_packet_t* to_ret = &packet_buffers[packet_buffer_num];

   packet_buffer_num = (packet_buffer_num >= (PACKET_BUFFER_NUM - 1)) ? 0 : packet_buffer_num +
1;

   return to_ret;
}

static int psa_parse_buttons_iden(
   uint8_t const *const data,
   struct psa_output_data_buffers *data_buffers)
{
   struct mive_global_event event = {0};

   if (data_buffers->headunit_data == NULL)
   {
      return -MIVE_ERR_INVALID_ARGUMENT;
   }
```

```
    if ((data[1] & 0x0F) == 0x02)
    {
        // Refresh the timer when audio up or down buttons are pressed
        if (((data[2] & 0x1F) == 0x11 || (data[2] & 0x1F) == 0x12))
        {
            event.event = MIVE_EVENT_VAN_UPDATE_AUDIO_MENU;
        }
        // Change the menu when the audio button is let go. Will probably go tits up when the button is held dow
n for more than 2 secs
        if (data[2] == 0x56)
        {
            event.event = MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM;
        }
    }

    if(event.event)
    {
        xQueueSendToBack(g_global_state.global_main_queue, &event, 0);
    }

    return MIVE_OK;
}

static int psa_parse_rpm_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA
    };
    mive_uart_queue_packet_t* queue_packet;

    if (data_buffers->engine_data == NULL)
    {
```

```
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    struct psa_engine_data *engine_data = (struct psa_engine_data *)(data_buffers->engine_data);

    struct psa_van_rpm_speed_struct const *const rpm_data = (struct psa_van_rpm_speed_struct *)data;

    engine_data->rpm = be16toh(rpm_data->rpm);
    engine_data->speed = be16toh(rpm_data->speed);

    queue_packet = get_packet_buffer();

    queue_packet->idens[0] = PSA_IDENT_ENGINE;
    queue_packet->num_idens = 1;

    queue_event.ev_data = (void*)queue_packet;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

static int psa_parse_dash_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    uint32_t mileage;
    union dash_mileage temp_mileage;
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_DASHBOARD, PSA_IDENT_ENGINE},
        .num_idens = 2,
```

```
};


if (data_buffers->dash_data == NULL || data_buffers->engine_data == NULL)
{
    return -MIVE_ERR_INVALID_ARGUMENT;
}

queue_packet = get_packet_buffer();

*queue_packet = queue_packet_c;

struct mive_global_event queue_event = {
    .event = MIVE_EVENT_UART_NEW_DATA,
    .ev_data = queue_packet,
};

struct psa_dash_data *dash_data = (struct psa_dash_data *)(data_buffers->dash_data);
struct psa_engine_data *engine_data = (struct psa_engine_data *)(data_buffers->engine_data);
// struct psa_door_data *door_data = (struct psa_door_data *)(data_buffers->door_data);

struct psa_van_dashboard *van_data = (struct psa_van_dashboard *)data;

dash_data->accesories_on = van_data->accesories_on;
dash_data->backlight_off = van_data->is_backlight_off;
dash_data->brightness = van_data->brightness;
dash_data->door_open = van_data->door_open;
dash_data->economy_mode = van_data->economy_mode;
dash_data->engine_running = van_data->engine_running;
dash_data->external_temp = van_data->external_temp;
dash_data->ignition_on = van_data->ignition_on;

dash_data->backlight_off = van_data->is_backlight_off;
```

```
    dash_data->reverse_gear = van_data->reverse_gear;
    dash_data->water_temp = van_data->water_temp;
    engine_data->engine_temperature = van_data->water_temp;
    engine_data->reverse = van_data->reverse_gear;
    mileage = van_data->mileage;

    temp_mileage.number = van_data->mileage;

    dash_data->mileage = mileage_swap_bytes(temp_mileage);

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

static int psa_parse_trip_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_DOORS, PSA_IDENT_TRIP},
        .num_idens = 2,
    };

    if (data_buffers->door_data == NULL || data_buffers->trip_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();
```

```
*queue_packet = queue_packet_c;

struct mive_global_event queue_event = {
    .event = MIVE_EVENT_UART_NEW_DATA,
    .ev_data = queue_packet,
};

struct psa_trip_data *trip_data = (struct psa_trip_data *)(data_buffers->trip_data);
struct psa_door_data *door_data = (struct psa_door_data *)(data_buffers->door_data);

struct psa_van_trip_computer *van_data = (struct psa_van_trip_computer *)data;

door_data->open_doors = (van_data->door_status.packed);

if (door_data->open_doors)
{
    door_data->door_open = 1;
}
else
{
    door_data->door_open = 0;
}

trip_data->current_fuel_consumption = be16toh(van_data->current_fuel_consumption);
trip_data->distance_to_empty = be16toh(van_data->distance_to_empty);
trip_data->trip_1_distance = be16toh(van_data->trip_1_distance);
trip_data->trip_1_fuel_consumption = be16toh(van_data->trip_1_fuel_consumption);
trip_data->trip_1_speed = van_data->trip_1_speed;

trip_data->trip_2_distance = be16toh(van_data->trip_2_distance);
trip_data->trip_2_fuel_consumption = be16toh(van_data->trip_2_fuel_consumption);
trip_data->trip_2_speed = van_data->trip_2_speed;
trip_data->trip_button_pressed = van_data->trip_button.trip_button;
```

```
    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

// 0x554 len 22
static int psa_parse_radio_tuner_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_RADIO, PSA_IDENT_RADIO_PRESETS},
        .num_idens = 2,
    };

    if (data_buffers->radio_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    if (data[1] != 0xD1) // D1 is frequency info
    {
        return -MIVE_ERR_VAN_UNKNOWN_IDEN;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
```

```
    .event = MIVE_EVENT_UART_NEW_DATA,
    .ev_data = queue_packet,
};

struct psa_van_radio_freq_info *van_data = (struct psa_van_radio_freq_info *)data;
struct psa_radio_data *radio_data = (struct psa_radio_data *)data_buffers->radio_data;

struct psa_preset_data *preset_data = NULL;

memset(radio_data->station, 0, 9);
strncpy(radio_data->station, (const char *)van_data->station, 8);

uint16_t freq = (uint16_t)(van_data->frequency[1]) << 8 | van_data->frequency[0];

radio_data->freq = freq * 5 + 5000;

radio_data->signal_strength = van_data->signal_info & 0x0F;
// radio_data->freq = uint16_t(van_data->frequency[1]) << 8 | van_data->frequency[0]; // swap_endian
_uint16(van_data->frequency);

radio_data->preset = van_data->memory_position;

switch (van_data->band)
{
case PSA_VAN_BAND_FM1:
    radio_data->band = PSA_FM_1;
    preset_data = (struct psa_preset_data *)data_buffers->presets_data_fm_1;
    break;
case PSA_VAN_BAND_FM2:
    radio_data->band = PSA_FM_2;
    preset_data = (struct psa_preset_data *)data_buffers->presets_data_fm_2;
    break;
case PSA_VAN_BAND_FM3:
```

```
        radio_data->band = PSA_FM_3;
        break;
    case PSA_VAN_BAND_FMAST:
        radio_data->band = PSA_FM_AST;
        preset_data = (struct psa_preset_data *)data_buffers->presets_data_fm_ast;

        break;
    case PSA_VAN_BAND_AM:
        radio_data->band = PSA_AM_1;
        preset_data = (struct psa_preset_data *)data_buffers->presets_data_am;
        break;
    case PSA_VAN_BAND_NONE:
    case PSA_VAN_BAND_PTY_SELECT:
    default:
        radio_data->band = PSA_NONE;
        break;
    }

    if (radio_data->preset > 0 && radio_data->preset <= 6 && preset_data != NULL)
    {
        strncpy(preset_data->presets[radio_data->preset - 1].preset_name, (const char *)van_data->station, 8);
        preset_data->presets->preset_num = radio_data->preset;
    }

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

// 0x554 len 10
static int psa_parse_radio_cd_short_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
```

```
mive_uart_queue_packet_t* queue_packet = NULL;

mive_uart_queue_packet_t queue_packet_c = {
    .idens = {PSA_IDENT_CD_PLAYER},
    .num_idens = 1,
};

if (data_buffers->cd_player_data == NULL)
{
    return -MIVE_ERR_INVALID_ARGUMENT;
}

if (data[1] != 0xD6) // D6 is CD info
{
    return -MIVE_ERR_VAN_UNKNOWN_IDEN;
}

queue_packet = get_packet_buffer();

*queue_packet = queue_packet_c;

struct mive_global_event queue_event = {
    .event = MIVE_EVENT_UART_NEW_DATA,
    .ev_data = queue_packet,
};

struct psa_van_radio_cd_info_len_10 *van_data = (struct psa_van_radio_cd_info_len_10 *)data;
struct psa_cd_player_data *cd_data = (struct psa_cd_player_data *)data_buffers->cd_player_data;

cd_data->current_minute = bcd_to_dec(van_data->minutes);
cd_data->current_second = bcd_to_dec(van_data->seconds);
cd_data->current_track = bcd_to_dec(van_data->current_track);
cd_data->total_tracks = bcd_to_dec(van_data->total_tracks);
```

```
cd_data->status = 0;

switch (van_data->status)
{
case 0x10:
   cd_data->status |= PSA_CD_PLAYER_ERROR;
   break;
case 0x11:
   cd_data->status |= PSA_CD_PLAYER_LOADING;
   break;
case 0x12:
   cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PAUSED;
   break;
case 0x13:
   cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PLAYING;
   break;
case 0x02:
   cd_data->status |= PSA_CD_PLAYER_PAUSED;
   break;
case 0x03:
   cd_data->status |= PSA_CD_PLAYER_PLAYING;
   break;
case 0x04:
   cd_data->status |= PSA_CD_PLAYER_FAST_FORWARDING;
   break;
case 0x05:
   cd_data->status |= PSA_CD_PLAYER_REWINDING;
   break;
default:
   cd_data->status = 0;
   break;
}
```

```
    cd_data->total_minutes = 0xff;
    cd_data->total_seconds = 0xff;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

// 0x554 len 19
static int psa_parse_radio_cd_long_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_CD_PLAYER},
        .num_idens = 1,
    };

    if (data_buffers->cd_player_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    if (data[1] != 0xD6) // D6 is CD info
    {
        return -MIVE_ERR_VAN_UNKNOWN_IDEN;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;
```

```
struct mive_global_event queue_event = {
    .event = MIVE_EVENT_UART_NEW_DATA,
    .ev_data = queue_packet,
};

struct psa_van_radio_cd_info_len_19 *van_data = (struct psa_van_radio_cd_info_len_19 *)data;
struct psa_cd_player_data *cd_data = (struct psa_cd_player_data *)data_buffers->cd_player_data;

cd_data->current_minute = bcd_to_dec(van_data->minutes);
cd_data->current_second = bcd_to_dec(van_data->seconds);
cd_data->current_track = bcd_to_dec(van_data->current_track);
cd_data->total_tracks = bcd_to_dec(van_data->total_tracks);
cd_data->total_minutes = bcd_to_dec(van_data->total_minutes);
cd_data->total_seconds = bcd_to_dec(van_data->total_seconds);

cd_data->status = 0;

switch (van_data->status)
{
case 0x10:
    cd_data->status |= PSA_CD_PLAYER_ERROR;
    break;
case 0x11:
    cd_data->status |= PSA_CD_PLAYER_LOADING;
    break;
case 0x12:
    cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PAUSED;
    break;
case 0x13:
    cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PLAYING;
    break;
case 0x02:
    cd_data->status |= PSA_CD_PLAYER_PAUSED;
```

```
        break;
    case 0x03:
        cd_data->status |= PSA_CD_PLAYER_PLAYING;
        break;
    case 0x04:
        cd_data->status |= PSA_CD_PLAYER_FAST_FORWARDING;
        break;
    case 0x05:
        cd_data->status |= PSA_CD_PLAYER_REWINDING;
        break;
    default:
        cd_data->status = 0;
        break;
    }

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

static int psa_parse_radio_preset_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    // if (data_buffers == NULL)
    // {
    //     return -MIVE_ERR_INVALID_ARGUMENT;
    // }

    // if (data_buffers->presets_data == NULL)
    // {
    //     return -MIVE_ERR_INVALID_ARGUMENT;
    // }
```

```
    // if (data[1] != 0xD3) // D3 is preset info
    // {
    //     return -MIVE_ERR_VAN_UNKNOWN_IDEN;
    // }

    // struct psa_van_radio_preset_info *van_data = (struct psa_van_radio_preset_info *)data;

    // struct psa_preset_data *preset_data = (struct psa_preset_data *)data_buffers->presets_data;

    // if (van_data->position > 0 && van_data->position < 7)
    // {
    //     memset(preset_data->presets[van_data->position].preset_name, 0, 10);
    //     strncpy(preset_data->presets[van_data->position].preset_name, (const char *)van_data->station_na
me, 8);

    //     preset_data->presets[van_data->position].preset_num = van_data->position;
    // }
    // else
    // {
    //     return PSA_INVALID_VAN_PACKET;
    // }

    return MIVE_OK;
}

static int psa_parse_headunit_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_HEADUNIT},
```

```
    .num_idens = 1,
};

if (data_buffers->headunit_data == NULL)
{
    return -MIVE_ERR_INVALID_ARGUMENT;
}

queue_packet = get_packet_buffer();

*queue_packet = queue_packet_c;

struct mive_global_event queue_event = {
    .event = MIVE_EVENT_UART_NEW_DATA,
    .ev_data = queue_packet,
};

struct psa_van_radio_settings *van_data = (struct psa_van_radio_settings *)data;

struct psa_headunit_data *headunit_data = (struct psa_headunit_data *)data_buffers->headunit_data;

headunit_data->unit_powered_on = van_data->power.power_on;
headunit_data->auto_volume = van_data->audio_properties.auto_volume;
headunit_data->balance = ((int8_t)0x3f - van_data->balance.value);
headunit_data->fader = ((int8_t)0x3f - van_data->fader.value);
headunit_data->bass = ((int8_t)van_data->bass.value - 0x3f);
headunit_data->treble = ((int8_t)van_data->treble.value - 0x3f);
headunit_data->loudness_on = van_data->audio_properties.loudness_on;
headunit_data->volume = van_data->volume.value;

headunit_data->muted = van_data->audio_properties.external_mute || van_data->audio_properties.stalk_m
ute;
```

```
    switch (van_data->source.source)
    {
    case 0x01: // Tuner
        headunit_data->source = PSA_RADIO_TUNER;
        break;
    case 0x02: // Tape or CD
        headunit_data->source = PSA_RADIO_INTERNAL;
        break;
    case 0x03: // Cd changer
        headunit_data->source = PSA_RADIO_EXTERNAL;
        break;
    case 0x05: // Navigation
        headunit_data->source = PSA_RADIO_NAVIGATION;
        break;
    default:
        headunit_data->source = PSA_RADIO_NONE;
        break;
    }

    headunit_data->cd_present = van_data->source.cd_present;
    headunit_data->tape_present = van_data->source.tape_present;
    // headunit_data->settings_menu_open = van_data->audio_properties.audio_properties_menu_open;

    // headunit_data->settings_changing.balance_changing = van_data->balance.updating;
    // headunit_data->settings_changing.bass_changing = van_data->bass.updating;
    // headunit_data->settings_changing.fader_changing = van_data->fader.updating;
    // headunit_data->settings_changing.treble_changing = van_data->treble.updating;
    // headunit_data->settings_changing.volume_changing = van_data->volume.updating;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}
```

```
static int psa_parse_vin_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_VIN},
        .num_idens = 1,
    };

    if (data_buffers->vin_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };
    // char *vin = (char *)data_buffers->vin_data;
    int const vin_size = 17;

    memcpy(data_buffers->vin_data, data, vin_size);

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}
```

```
// 0x4FC len 11
static int psa_parse_instruments_short_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;
    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_ENGINE},
        .num_idens = 1,
    };

    if (data_buffers->engine_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_engine_data *engine_data = (struct psa_engine_data *)data_buffers->engine_data;
    struct psa_van_instrument_cluster_short *van_data = (struct psa_van_instrument_cluster_short *)data;

    engine_data->fuel_level = van_data->fuel_level;
    engine_data->oil_temperature = van_data->oil_temp;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
```

```
    return MIVE_OK;
}

// 0x4FC len 14
static int psa_parse_instruments_long_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;
    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_ENGINE},
        .num_idens = 1,
    };

    if (data_buffers->engine_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_engine_data *engine_data = (struct psa_engine_data *)data_buffers->engine_data;
    struct psa_van_instrument_cluster_long *van_data = (struct psa_van_instrument_cluster_long *)data;

    engine_data->fuel_level = van_data->fuel_level;
```

```
    engine_data->oil_temperature = van_data->oil_temp;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

// 0x524 len 14
static int psa_parse_car_status_2_short(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;
    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_CAR_STATUS},
        .num_idens = 1,
    };

    if (data_buffers->status_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };
```

```
    struct psa_status_data *status_data = (struct psa_status_data *)data_buffers->status_data;
    struct psa_van_car_status_2_short *van_data = (struct psa_van_car_status_2_short *)data;

    status_data->doors_locked = van_data->Field8.doors_locked;
    status_data->deadlocking_active = van_data->Field8.deadlocking_active;

    status_data->key_in_ignition = van_data->Field5.ignition_key_left_in;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

// 0x524 len 16
static int psa_parse_car_status_2_long(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;
    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_CAR_STATUS},
        .num_idens = 1,
    };

    if (data_buffers->status_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;
```

```
    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_status_data *status_data = (struct psa_status_data *)data_buffers->status_data;
    struct psa_van_car_status_2_long *van_data = (struct psa_van_car_status_2_long *)data;

    status_data->doors_locked = van_data->Field8.doors_locked;
    status_data->deadlocking_active = van_data->Field8.deadlocking_active;

    status_data->key_in_ignition = van_data->Field5.ignition_key_left_in;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

static int psa_parse_cdc_command(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;
    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_CAR_STATUS},
        .num_idens = 1,
    };

    if (data_buffers->status_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }
```

```
    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_status_data *status_data = (struct psa_status_data *)data_buffers->status_data;
    struct psa_van_cd_changer_command_struct *van_data = (struct psa_van_cd_changer_command_struct
*)data;

    uint16_t command = be16toh(van_data->command);

    switch (command)
    {
    case PSA_VAN_CD_CHANGER_PLAY:
        status_data->cd_changer_command = PSA_CD_CHANGER_COMM_PLAY;
        break;
    case PSA_VAN_CD_CHANGER_PAUSE:
        status_data->cd_changer_command = PSA_CD_CHANGER_COMM_PAUSE;
        break;
    case PSA_VAN_CD_CHANGER_NEXT_TRACK:
        status_data->cd_changer_command = PSA_CD_CHANGER_COMM_NEXT_TRACK;
        break;
    case PSA_VAN_CD_CHANGER_PREVIOUS_TRACK:
        status_data->cd_changer_command = PSA_CD_CHANGER_COMM_PREV_TRACK;
        break;
    default:
        status_data->cd_changer_command = PSA_CD_CHANGER_COMM_NONE;
        break;
    }
```

```
    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

int psa_parse_van_packet(
    uint16_t iden,
    uint8_t size,
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    if (data == NULL || data_buffers == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    switch (iden)
    {
    case PSA_VAN_IDEN_RPM:
        if (size != sizeof(struct psa_van_rpm_speed_struct))
        {
            return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
        }

        return psa_parse_rpm_iden(data, (struct psa_output_data_buffers *)data_buffers);
        break;

    case PSA_VAN_IDEN_CAR_STATUS1:
        if (size != sizeof(struct psa_van_trip_computer))
        {
            return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
        }

        return psa_parse_trip_iden(data, (struct psa_output_data_buffers *)data_buffers);
```

```
    break;

case PSA_VAN_IDEN_ENGINE:
    if (size != sizeof(struct psa_van_dashboard))
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

    return psa_parse_dash_iden(data, (struct psa_output_data_buffers *)data_buffers);
    break;
case PSA_VAN_IDEN_VIN:
    if (size != 17)
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

    return psa_parse_vin_iden(data, (struct psa_output_data_buffers *)data_buffers);

    break;

case PSA_VAN_IDEN_HEAD_UNIT:
    if (size == 22)
    {
        return psa_parse_radio_tuner_iden(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else if (size == 10)
    {
        return psa_parse_radio_cd_short_iden(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else if (size == 19)
    {
        return psa_parse_radio_cd_long_iden(data, (struct psa_output_data_buffers *)data_buffers);
    }
    // else if (size == 12)
```

```
    // {
    //     return psa_parse_radio_preset_iden(data, (struct psa_output_data_buffers *)data_buffers);
    // }
    else
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

    break;
case PSA_VAN_IDEN_AUDIO_SETTINGS:
    if (size != 11)
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

    return psa_parse_headunit_iden(data, (struct psa_output_data_buffers *)data_buffers);
    break;

case PSA_VAN_IDEN_LIGHTS_STATUS:
    if (size == 11)
    {
        return psa_parse_instruments_short_iden(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else if (size == 14)
    {
        return psa_parse_instruments_long_iden(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }
    break;

case PSA_VAN_IDEN_CAR_STATUS2:
```

```
    if (size == 14)
    {
        return psa_parse_car_status_2_short(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else if (size == 16)
    {
        return psa_parse_car_status_2_long(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

case PSA_VAN_IDEN_CDCHANGER_COMMAND:
    if (size == 2)
    {
        return psa_parse_cdc_command(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

case PSA_VAN_IDEN_DEVICE_REPORT:
    if (size == 3)
    {
        return psa_parse_buttons_iden(data, (struct psa_output_data_buffers *)data_buffers);
    }
    else
    {
        return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
    }

default:
```

```
        return -MIVE_ERR_VAN_UNKNOWN_IDEN;
        break;
    }

    return MIVE_OK;
}

#include "freertos/FreeRTOS.h"

#include <stdint.h>
#include <stdbool.h>
#include <string.h>

#include "include/mive/common.h"
#include "include/van/van_rmt_rx.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_attr.h"
#include "driver/gpio.h"
#include "driver/rmt_rx.h"

#include "esp_log.h"

static const char *TAG = "RMT_RX";

#define VAN_CRC_POLYNOM = 0x0F9D;

#define VAN_CRC_TABLE_SIZE 256
static const uint16_t crcTable[VAN_CRC_TABLE_SIZE] =
{
    0x0000, 0x0F9D, 0x1F3A, 0x10A7, 0x3E74, 0x31E9, 0x214E, 0x2ED3,
    0x7CE8, 0x7375, 0x63D2, 0x6C4F, 0x429C, 0x4D01, 0x5DA6, 0x523B,
    0x764D, 0x79D0, 0x6977, 0x66EA, 0x4839, 0x47A4, 0x5703, 0x589E,
    0x0AA5, 0x0538, 0x159F, 0x1A02, 0x34D1, 0x3B4C, 0x2BEB, 0x2476,
```

```
    0x6307, 0x6C9A, 0x7C3D, 0x73A0, 0x5D73, 0x52EE, 0x4249, 0x4DD4,
    0x1FEF, 0x1072, 0x00D5, 0x0F48, 0x219B, 0x2E06, 0x3EA1, 0x313C,
    0x154A, 0x1AD7, 0x0A70, 0x05ED, 0x2B3E, 0x24A3, 0x3404, 0x3B99,
    0x69A2, 0x663F, 0x7698, 0x7905, 0x57D6, 0x584B, 0x48EC, 0x4771,
    0x4993, 0x460E, 0x56A9, 0x5934, 0x77E7, 0x787A, 0x68DD, 0x6740,
    0x357B, 0x3AE6, 0x2A41, 0x25DC, 0x0B0F, 0x0492, 0x1435, 0x1BA8,
    0x3FDE, 0x3043, 0x20E4, 0x2F79, 0x01AA, 0x0E37, 0x1E90, 0x110D,
    0x4336, 0x4CAB, 0x5C0C, 0x5391, 0x7D42, 0x72DF, 0x6278, 0x6DE5,
    0x2A94, 0x2509, 0x35AE, 0x3A33, 0x14E0, 0x1B7D, 0x0BDA, 0x0447,
    0x567C, 0x59E1, 0x4946, 0x46DB, 0x6808, 0x6795, 0x7732, 0x78AF,
    0x5CD9, 0x5344, 0x43E3, 0x4C7E, 0x62AD, 0x6D30, 0x7D97, 0x720A,
    0x2031, 0x2FAC, 0x3F0B, 0x3096, 0x1E45, 0x11D8, 0x017F, 0x0EE2,
    0x1CBB, 0x1326, 0x0381, 0x0C1C, 0x22CF, 0x2D52, 0x3DF5, 0x3268,
    0x6053, 0x6FCE, 0x7F69, 0x70F4, 0x5E27, 0x51BA, 0x411D, 0x4E80,
    0x6AF6, 0x656B, 0x75CC, 0x7A51, 0x5482, 0x5B1F, 0x4BB8, 0x4425,
    0x161E, 0x1983, 0x0924, 0x06B9, 0x286A, 0x27F7, 0x3750, 0x38CD,
    0x7FBC, 0x7021, 0x6086, 0x6F1B, 0x41C8, 0x4E55, 0x5EF2, 0x516F,
    0x0354, 0x0CC9, 0x1C6E, 0x13F3, 0x3D20, 0x32BD, 0x221A, 0x2D87,
    0x09F1, 0x066C, 0x16CB, 0x1956, 0x3785, 0x3818, 0x28BF, 0x2722,
    0x7519, 0x7A84, 0x6A23, 0x65BE, 0x4B6D, 0x44F0, 0x5457, 0x5BCA,
    0x5528, 0x5AB5, 0x4A12, 0x458F, 0x6B5C, 0x64C1, 0x7466, 0x7BFB,
    0x29C0, 0x265D, 0x36FA, 0x3967, 0x17B4, 0x1829, 0x088E, 0x0713,
    0x2365, 0x2CF8, 0x3C5F, 0x33C2, 0x1D11, 0x128C, 0x022B, 0x0DB6,
    0x5F8D, 0x5010, 0x40B7, 0x4F2A, 0x61F9, 0x6E64, 0x7EC3, 0x715E,
    0x362F, 0x39B2, 0x2915, 0x2688, 0x085B, 0x07C6, 0x1761, 0x18FC,
    0x4AC7, 0x455A, 0x55FD, 0x5A60, 0x74B3, 0x7B2E, 0x6B89, 0x6414,
    0x4062, 0x4FFF, 0x5F58, 0x50C5, 0x7E16, 0x718B, 0x612C, 0x6EB1,
    0x3C8A, 0x3317, 0x23B0, 0x2C2D, 0x02FE, 0x0D63, 0x1DC4, 0x1259,
};

IRAM_ATTR static bool rmt_rx_done_callback(rmt_channel_handle_t channel, const rmt_rx_done_event_d
ata_t *edata, void *user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
```

```
    van_rmt_rx_instance_t* van_instance = (van_rmt_rx_instance_t*)user_data;
    // send the received RMT symbols to the parser task
    rmt_receive(
        van_instance->rx_chan,
        van_instance->rmt_symbol_buffer,
        van_instance->rmt_symbol_buffer_size,
        &van_instance->rx_recv_config);
    xQueueSendFromISR(van_instance->receive_queue, edata, &high_task_wakeup);
    return high_task_wakeup == pdTRUE;
}

IRAM_ATTR bool van_rmt_rx_parse_byte(
    van_rmt_rx_instance_t* instance,
    uint8_t level,
    uint32_t duration,
    uint8_t *bitCounter,
    uint8_t *tempByte,
    uint8_t *mask,
    uint8_t *finalByte)
{
    bool result = false;
    if (instance->rmt_rx_van_line_level == RX_VAN_LINE_LEVEL_LOW)
    {
        level = !level;
    }

    // on the bus the time slices are a little off from the multiple of the TS microseconds, so we round it to the n
earest multiple of the TS before dividing
    uint8_t countOfTimeSlices = round_to_nearest(duration, instance->rmt_rx_time_slice_divisor) / instance-
>rmt_rx_time_slice_divisor;

    for (int i = 0; i < countOfTimeSlices; i++)
    {
        // every 5th bit is a manchester bit, we must skip them while we are building our byte
```

```
      bool isManchesterBit = (*bitCounter + 1) % 5 == 0;
      if (!isManchesterBit)
      {
        if (level == 1)
        {
          *tempByte |= *mask;
        }
        *mask = *mask >> 1;
      }
      else
      {
        // if we found the second manchester bit, then we have the full byte in the tempByte variable, so we pla
ce it in the finalByte and we can start building the next byte
        if (*bitCounter == 9)
        {
          *bitCounter = -1;
          *mask = 1 << 7;
          *finalByte = *tempByte;
          *tempByte = 0;
          result = true;
        }
      }
      *bitCounter = *bitCounter + 1;
    }

    return result;
}

IRAM_ATTR uint16_t van_rmt_rx_crc15(uint8_t data[], uint8_t lengthOfData)
{
    uint16_t crc15 = 0x7FFF;

    for (int i = 0; i < lengthOfData; i++)  // Skip first byte (SOF, 0x0E) and last 2 (CRC)
    {
```

```
        uint8_t byte = data[i];

        // XOR-in next input byte into MSB of crc, that's our new intermediate divident
        uint8_t pos = (uint8_t)( (crc15 >> 7) ^ byte);

        // Shift out the MSB used for division per lookup table and XOR with the remainder
        crc15 = (uint16_t)((crc15 << 8) ^ (uint16_t)(crcTable[pos]));
    } // for

    crc15 ^= 0x7FFF;
    crc15 <<= 1;  // Shift left 1 bit to turn 15 bit result into 16 bit representation

    return crc15;
}

// IRAM_ATTR uint16_t van_rmt_rx_crc15(uint8_t data[], uint8_t lengthOfData)
// {
//     const uint8_t order = 15;
//     const uint16_t polynom = 0xF9D;
//     const uint16_t xorValue = 0x7FFF;
//     const uint16_t mask = 0x7FFF;

//     uint16_t crc = 0x7FFF;

//     for (uint8_t i = 0; i < lengthOfData; i++)
//     {
//         uint8_t currentByte = data[i];

//         // rotate one data byte including crcmask
//         for (uint8_t j = 0; j < 8; j++)
//         {
//             bool bit = (crc & (1 << (order - 1))) != 0;
//             if ((currentByte & 0x80) != 0)
```

```
//          {
//              bit = !bit;
//          }
//          currentByte <<= 1;

//          crc = ((crc << 1) & mask) ^ (-bit & polynom);
//      }
//  }

//  // perform xor and multiply result by 2 to turn 15 bit result into 16 bit representation
//  return (crc ^ xorValue) << 1;
// }

IRAM_ATTR bool van_rmt_rx_is_crc_ok(uint8_t vanMessage[], int vanMessageLength)
{
  bool retval = false;
  uint8_t crcByte1 = 0;
  uint8_t crcByte2 = 0;
  uint16_t crcValueInMessage = 0;

  // Check if message length is even valid (Start byte + iden + crc)
  if(vanMessageLength < 5)
  {
    return false;
  }

  crcByte1 = vanMessage[vanMessageLength - 2];
  crcByte2 = vanMessage[vanMessageLength - 1];
  crcValueInMessage = crcByte1 << 8 | crcByte2;

  if (vanMessageLength - 3 <= 32)
  {
    uint16_t calculatedCrc = van_rmt_rx_crc15(vanMessage + 1, vanMessageLength - 3);
    if(crcValueInMessage == calculatedCrc)
```

```
        {
            retval = true;
        }
        else
        {
            ESP_LOGE(TAG, "[%s] Calculated: 0x%04x, Received: 0x%04x", __func__, calculatedCrc, crcValu
eInMessage);
            retval = false;
        }
    }
    return retval;
}

/* Uninstall the RMT driver */
void van_rmt_rx_channel_stop_new(van_rmt_rx_instance_t* instance)
{
    rmt_disable(instance->rx_chan);
}

void van_rmt_rx_channel_start_new(van_rmt_rx_instance_t* instance)
{
    rmt_enable(instance->rx_chan);
}

void van_rmt_rx_channel_init_new(
    van_rmt_rx_instance_t* instance,
    uint8_t channel,
    int rxPin,
    int ledPin,
    RX_VAN_LINE_LEVEL vanLineLevel,
    RX_VAN_NETWORK_TYPE vanNetworkType)
{
    rmt_rx_channel_config_t rx_chan_config = {0};
    rmt_receive_config_t rx_recv_config = {0};
```

```
rmt_channel_handle_t rx_chan;
uint32_t timeslot_interval_ns = 0;
uint8_t _rmt_van_rx_time_slice_divisor;
QueueHandle_t receive_queue = NULL;
rmt_symbol_word_t* rmt_symbol_buffer = NULL;

// TS = 8us = 125k
// TS = 16us = 62.5k

if(rxPin < 0)
{
   ESP_LOGE(TAG, "[%s] Invalid rxPin: %d", __func__, rxPin);
   return;
}

rx_chan_config.gpio_num = rxPin;
rx_chan_config.clk_src = RMT_CLK_SRC_DEFAULT;
rx_chan_config.resolution_hz = 1000000; // 1 MHz
rx_chan_config.mem_block_symbols = 512;
// RMT DMA not supported on ESP32
rx_chan_config.flags.with_dma = 0;

if (vanNetworkType == RX_VAN_NETWORK_COMFORT)
{
   _rmt_van_rx_time_slice_divisor = 8;
   timeslot_interval_ns = 8 * 1000;
}
else
{
   _rmt_van_rx_time_slice_divisor = 16;
   timeslot_interval_ns = 16 * 1000;
}

rx_recv_config.signal_range_max_ns = 10 * timeslot_interval_ns;
```

```
    rx_recv_config.signal_range_min_ns = timeslot_interval_ns / _rmt_van_rx_time_slice_divisor;

    rmt_new_rx_channel(&rx_chan_config, &rx_chan);

    receive_queue = xQueueCreate(30, sizeof(rmt_rx_done_event_data_t));

    rmt_rx_event_callbacks_t cbs = {
        .on_recv_done = rmt_rx_done_callback
    };
    rmt_rx_register_event_callbacks(rx_chan, &cbs, instance);

    rmt_symbol_buffer = heap_caps_malloc(2048, MALLOC_CAP_DMA);

    instance->rmt_rx_channel = channel;
    instance->rmt_rx_rxPin = rxPin;
    instance->rmt_rx_van_line_level = vanLineLevel;
    instance->rmt_rx_time_slice_divisor = _rmt_van_rx_time_slice_divisor;
    instance->rx_chan = rx_chan;
    instance->rx_recv_config = rx_recv_config;
    instance->receive_queue = receive_queue;
    instance->rmt_symbol_buffer = rmt_symbol_buffer;
    instance->rmt_symbol_buffer_size = 2048;
}

#include "freertos/FreeRTOS.h"

#include <string.h>
#include <stdlib.h>

#include "esp_attr.h"
#include "freertos/queue.h"

#include "driver/gpio.h"
#include "include/mive/common.h"
```

```
#include "include/global_tasks.h"
#include "include/global_state_def.h"
#include "include/global_config.h"
#include "include/van/van_rmt_rx.h"

van_rmt_rx_instance_t global_van_instance;

#define VAN_MAX_NUM_PACKETS 50

static uint8_t* buffers[VAN_MAX_NUM_PACKETS];

static int van_parse_bytes(van_rmt_rx_instance_t* instance, mive_van_packet_t* packet, rmt_rx_done_event
_data_t rx_data, int packet_index)
{
   rmt_symbol_word_t* items;

   uint8_t bitCounter = 0;
   uint8_t mask = 1 << 7;
   uint8_t tempByte = 0;
   uint8_t finalByte = 0;
   int van_message_length = 0;
   size_t i = 0;
   bool isCompleteByte = false;
   int retval = 0;

   uint8_t temp_array[VAN_MAX_PACKET_LEN + 10] = {0};

   items = rx_data.received_symbols;
   // printf("Got %d symbols\n", rx_data.num_symbols);
   for (i = 0; i < rx_data.num_symbols; i++)
   {
      if(van_message_length >= VAN_MAX_PACKET_LEN)
      {
```

```
            break;
        }
        isCompleteByte = van_rmt_rx_parse_byte(instance, items[i].level0, items[i].duration0, &bitCounter, &tempByte, &mask, &finalByte);
        if (isCompleteByte)
        {
            temp_array[van_message_length] = finalByte;
            van_message_length++;
        }

        isCompleteByte = van_rmt_rx_parse_byte(instance, items[i].level1, items[i].duration1, &bitCounter, &tempByte, &mask, &finalByte);
        if (isCompleteByte)
        {
            temp_array[van_message_length] = finalByte;
            van_message_length++;
        }
    }

    // printf("[%s] Received: ", __func__);
    // for(int i = 0; i < van_message_length; ++i)
    // {
    //     printf("0x%02x ", temp_array[i]);
    // }

    // printf("\n");

    if((van_message_length > 5) &&
       (van_message_length < VAN_MAX_PACKET_LEN) &&
       (van_rmt_rx_is_crc_ok(temp_array, van_message_length)))
    {
        packet->iden = (((uint16_t)temp_array[1] << 8) | ((uint16_t)temp_array[2] & 0xf0)) >> 4;
        packet->packet_size = van_message_length - 5;
        packet->packet = buffers[packet_index];
```

```
        memset(packet->packet, 0, packet->packet_size);
        memcpy(packet->packet, temp_array + 3, packet->packet_size);
        retval = MIVE_OK;
    }
    else
    {
        // printf("[%s] CRC error\n", __func__);
        retval = -MIVE_ERR_VAN_CRC_ERROR;
    }
    //  0e 5e 48 a0 20 11 d4

    return retval;
}

void van_rmt_task(void* params)
{
    int ret = 0;
    unsigned int packet_num = 0;
    mive_global_state_t *state = (mive_global_state_t*)params;
    state->van_rmt_instance = &global_van_instance;
    van_rmt_rx_instance_t* van_instance = &global_van_instance;
    mive_van_packet_t* van_packets = NULL;
    QueueHandle_t receive_queue;
    QueueHandle_t global_queue_to_main;
    rmt_rx_done_event_data_t rx_data;
    struct mive_global_event event_item_to_send = {.event = MIVE_EVENT_RMT_NEW_VAN_FRAME};

    printf("[%s] Starting...\n", __func__);

    van_rmt_rx_channel_init_new(
        van_instance,
        0,
        VAN_RX_PIN,
        VAN_RX_LED_PIN,
```

```
    RX_VAN_LINE_LEVEL_HIGH,
    RX_VAN_NETWORK_COMFORT);


van_rmt_rx_channel_start_new(van_instance);

rmt_receive(
    van_instance->rx_chan,
    van_instance->rmt_symbol_buffer,
    van_instance->rmt_symbol_buffer_size,
    &van_instance->rx_recv_config);

receive_queue = van_instance->receive_queue;
global_queue_to_main = state->global_main_queue;

van_packets = calloc(VAN_MAX_NUM_PACKETS, sizeof(*van_packets));

for(int i = 0; i < VAN_MAX_NUM_PACKETS; ++i)
{
    buffers[i] = calloc(1, VAN_MAX_PACKET_LEN);
}

while (true)
{
    if(xQueueReceive(receive_queue, &rx_data, pdMS_TO_TICKS(1000)) == pdPASS)
    {
        ret = van_parse_bytes(van_instance, &van_packets[packet_num], rx_data, packet_num);
        if(ret == MIVE_OK)
        {
            event_item_to_send.ev_data = &van_packets[packet_num];
            // printf("[%s] Sending to queue...\n", __func__);
            xQueueSendToBack(global_queue_to_main, &event_item_to_send, 0);
```

```
            packet_num = (packet_num == VAN_MAX_NUM_PACKETS - 1) ? 0 : packet_num + 1;
        }
    }
  }

}
#ifndef MIVE_GLOBAL_CONFIG_H
#define MIVE_GLOBAL_CONFIG_H

// #include "soc/gpio_num.h"


#define DEVKIT 1
#define PEZO 2

// #define ESP32_BOARD_TYPE PEZO
#define ESP32_BOARD_TYPE PEZO

#if (ESP32_BOARD_TYPE == PEZO)

#define VAN_RX_PIN GPIO_NUM_21
#define VAN_RX_LED_PIN -1
#define TSS_CS_PIN 22
#define TSS_INT_PIN 19
#define PSA_ADC_UNIT 0
#define PSA_ADC_CHANNEL 5
#define PSA_EXT_REG_PIN 27
#define TSS_OE_ENABLE_PIN 32

#elif (ESP32_BOARD_TYPE == DEVKIT)

#define VAN_RX_PIN GPIO_NUM_21
#define VAN_RX_LED_PIN GPIO_NUM_2
```

```
#define TSS_CS_PIN 5
#define TSS_INT_PIN 27
#define PSA_ADC_UNIT 0
#define PSA_ADC_CHANNEL 5
#define PSA_EXT_REG_PIN -1
#define TSS_OE_ENABLE_PIN -1

#else

#define VAN_RX_PIN -1
#define VAN_RX_LED_PIN -1
#define TSS_CS_PIN -1
#define TSS_INT_PIN -1
#define PSA_ADC_UNIT -1
#define PSA_ADC_CHANNEL -1
#define PSA_EXT_REG_PIN -1
#define TSS_OE_ENABLE_PIN -1

#endif

#endif // GLOBAL_CONFIG_H

#ifndef _PSA_GLOBAL_INIT_H
#define _PSA_GLOBAL_INIT_H

void init_adc(void);

#endif

#ifndef MIVE_GLOBAL_STATE_H
#define MIVE_GLOBAL_STATE_H

#include "freertos/queue.h"
```

```
#include "global_tasks.h"

#include "van/tss463c.h"
#include "van/van_rmt_rx.h"

#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali_scheme.h"

struct mive_global_state_t
{
    // Tasks to VAN
    QueueHandle_t global_van_queue;
    // Tasks to tss
    QueueHandle_t global_tss_queue;
    // Tasks to uart
    QueueHandle_t global_uart_queue;
    // Tasks to main
    QueueHandle_t global_main_queue;

    tss_instance_t *tss_instance;
    van_rmt_rx_instance_t *van_rmt_instance;

    adc_oneshot_unit_handle_t adc_handle;
    adc_cali_handle_t cali_handle;

};

extern struct mive_global_state_t g_global_state;
extern volatile int g_global_car_state;
extern volatile float g_bat_voltage;

#endif // MIVE_GLOBAL_STATE_H
```

```
#ifndef MIVE_GLOBAL_TASKS_H
#define MIVE_GLOBAL_TASKS_H

typedef struct mive_global_state_t mive_global_state_t;

enum mive_global_event_t
{
   MIVE_EVENT_NONE = 0,
   MIVE_EVENT_GLOBAL_SLEEP,
   MIVE_EVENT_RMT_NEW_VAN_FRAME,
   MIVE_EVENT_TSS_RESET,
   MIVE_EVENT_TSS_ACTIVATE,
   MIVE_EVENT_TSS_IDLE,
   MIVE_EVENT_TSS_SLEEP,
   MIVE_EVENT_TSS_WRITE_FRAME,
   MIVE_EVENT_TSS_MONITOR_CHANNEL,

   MIVE_EVENT_VAN_UPDATE_AUDIO_MENU,
   MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM,
   MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,

   MIVE_EVENT_UART_NEW_DATA, // New VAN Data to send
   MIVE_EVENT_UART_RECEIVE, // Received data from UART
   MIVE_EVENT_UART_SEND, // Send data to UART

   MIVE_EVENT_TIMER_UART_SEND,
   MIVE_EVENT_TIMER_1MS,
   MIVE_EVENT_TIMER_1S,

   MIVE_EVENT_ADC,
   MIVE_EVENT_STATE_CHANGE,
   MIVE_EVENT_POWER,
};
```

```
struct mive_global_event
{
   enum mive_global_event_t event;
   void* ev_data;
};

/* VAN Task type definitions */

#define VAN_MAX_PACKET_LEN 40

// ev_data for MIVE_EVENT_RMT_NEW_VAN_FRAME
typedef struct mive_van_packet
{
   uint16_t iden;
   uint8_t packet_size;
   uint8_t* packet;

} mive_van_packet_t;

/* TSS Task type definitions */

typedef struct mive_tss_task_packet
{
   uint16_t iden;
   uint8_t message_type; // enum tss_message_type in tss463.h
   uint8_t message_channel;
   uint8_t packet[VAN_MAX_PACKET_LEN];
   uint8_t packet_size;
} mive_tss_task_packet_t;

/*
UART Task event data
   Main -> UART task = Send packet
```

```
   UART task -> Main = Received packet
*/
typedef struct mive_uart_task_packet
{
   uint16_t iden; // uart message identifier
   uint8_t data_size; // Size of the data
   uint8_t data[128]; // Data portion of the packet
} mive_uart_task_packet_t;

/*
UART Queue event data.
New VAN data parsed -> queue uart send

*/
typedef struct mive_uart_queue_packet
{
   uint16_t num_idens; // Number of idens to send
   uint16_t idens[10]; // uart message identifiers
} mive_uart_queue_packet_t;

/* Task function definitions */

void van_rmt_task(void* params);
void tss_task(void* params);
void uart_task(void* params);

#endif // MIVE_GLOBAL_TASKS_H

#ifndef MIVE_COMMON_H
#define MIVE_COMMON_H

#include <stdint.h>

typedef enum {
```

```
    MIVE_OK = 0,
    MIVE_ERR,
    MIVE_ERR_INVALID_ARGUMENT,
    MIVE_ERR_NOT_IMPLEMENTED,
    MIVE_ERR_OUTOFMEMORY,
    MIVE_ERR_VAN_CRC_ERROR,
    MIVE_ERR_TSS_BUSY,
    MIVE_ERR_CHANNEL_BUSY,
    MIVE_ERR_VAN_UNKNOWN_IDEN,
    MIVE_ERR_VAN_INVALID_PACKET_SIZE,
} mive_error;

uint8_t round_to_nearest(uint8_t num_to_round, uint8_t multiple);
uint16_t get_iden_from_bytes(uint8_t const byte1, uint8_t const byte2);
uint8_t dec_to_bcd(uint8_t input);
uint8_t bcd_to_dec(uint8_t value);
uint8_t psa_crc8_checksum(const uint8_t * ptr, uint32_t len);



#endif // MIVE_COMMON_H

#ifndef _PSA_HELPER_H
#define _PSA_HELPER_H

#include <stdint.h>
#include "include/global_tasks.h"
#include "include/van/van_packet_parser.h"

#define PSA_MAIN_UART_SEND_BUFFERS_NUM 20

enum psa_radio_preset_band
{
    PSA_PRESET_AM = 0,
    PSA_PRESET_FM_1 = 1,
```

```
    PSA_PRESET_FM_2 = 2,
    PSA_PRESET_FMAST = 3,
    PSA_PRESET_MAX = 4,
};

extern struct psa_output_data_buffers global_libpsa_buffers;

extern mive_uart_task_packet_t* global_uart_send_buffers;
extern mive_uart_task_packet_t* global_uart_receive_buffers;

extern void libpsa_send_packet(uint16_t ident);
extern void init_libpsa_packets(void);
extern void libpsa_update_audio_settings_packet(int audio_menu_open, int audio_menu_setting);

#endif // _PSA_HELPER_H

#ifndef PSA_PACKET_DEFS_H
#define PSA_PACKET_DEFS_H

#include <stdint.h>

#define PSA_MSP_START_MAGIC 0x69
#define PSA_MSP_MAX_SIZE 100


enum psa_idents
{
    PSA_IDENT_VERSIONS = 0x4000,      // Send API and firmware version numbers
    PSA_IDENT_VIN = 0x4001,           // Send the VIN packet
    PSA_IDENT_ENGINE = 0x4002,        // Send the Engine packet
    PSA_IDENT_RADIO = 0x4003,         // Send the Radio packet
    PSA_IDENT_RADIO_PRESETS = 0x4004, // Sent the Presets packet
    PSA_IDENT_DOORS = 0x4005,         // Send the Doors packet
    PSA_IDENT_DASHBOARD = 0x4006,     // Send the Dashboard packet
```

```
    PSA_IDENT_TRIP = 0x4007,          // Send the Trip packet
    PSA_IDENT_HEADUNIT = 0x4008,      // Send the Headunit packet
    PSA_IDENT_CAR_STATUS = 0x4009,    // Send the Car status packet
    PSA_IDENT_CD_PLAYER = 0x4010,     // Send the CD player packet

    PSA_IDENT_SET_CD_CHANGER_DATA = 0x4501, // Send CD changer player data
    PSA_IDENT_SET_TRIP_RESET = 0x4502,      // Send trip reset request

    PSA_IDENT_TIME = 0x5000,       // Send the ESP RTC time
    PSA_IDENT_ESP32_TEMP = 0x5100, // Send the ESP temperature, borked

    PSA_IDENT_MAIN_APP = 0x6000, // Special ident used for my app, sends all packets as one packet. UN
USED
};

#define PSA_MSP_MIN_IDENT PSA_IDENT_VERSIONS

enum psa_msp_response
{
    PSA_MSP_OK = 0x0E,              // Incoming packet parsed successfully
    PSA_MSP_CRC_ERROR = 0xAA,       // Incoming packet has invalid CRC
    PSA_MSP_UNKNOWN_IDENT = 0xBB,   // Incoming packet has unknown ident
    PSA_MSP_INVALID_DATA_SIZE = 0xCC, // Received data size is not valid for sent ident
    PSA_MSP_INVALID_DATA = 0xDD,    // Received data is invalid, possibly out of bounds
    PSA_MSP_UNKNOWN_ERROR = 0xFF,   // Worst case scenario error, shouldn't appear normally
};

enum psa_radio_band
{
    PSA_NONE = 0,
    PSA_AM_1 = 1, // AM 1 band
    PSA_AM_2,     // AM 2 band
    PSA_AM_3,     // AM 3 band
    PSA_FM_1,     // FM 1 band
```

```
    PSA_FM_2,    // FM 2 band
    PSA_FM_3,    // FM 3 band
    PSA_FM_AST   // FMast band
};

enum psa_radio_source
{
    PSA_RADIO_NONE = (uint8_t)0,
    PSA_RADIO_TUNER = (uint8_t)1,     // Radio tuner
    PSA_RADIO_INTERNAL = (uint8_t)2,  // Internal CD or tape player
    PSA_RADIO_EXTERNAL = (uint8_t)3,  // CD changer
    PSA_RADIO_NAVIGATION = (uint8_t)4, // Navigation audio
};

/**
 * @brief Check the Documentation for an explanation
 *
 */
enum psa_car_state
{
    PSA_STATE_CAR_SLEEP = 0, // Car in sleep, no power to VAN+
    PSA_STATE_ECONOMY_MODE,  // Car is in Battery-saving mode
    PSA_STATE_CAR_LOCKED,    // Car off and doors locked
    PSA_STATE_CAR_OFF,       // Car off, radio off
    PSA_STATE_RADIO_ON,      // Car off, radio on
    PSA_STATE_ACCESSORY,     // Key in accessory position
    PSA_STATE_IGNITION,      // Key in ignition position
    PSA_STATE_CRANKING,      // Engine cranking
    PSA_STATE_ENGINE_ON,     // Engine ON.
    PSA_STATE_UNKNOWN,       // Some logic is wrong, this should not appear
};

enum psa_trip_meter
{
```

```
    PSA_TRIP_A = 1,
    PSA_TRIP_B = 2,
};

/**
 * @brief Doesnt work at the moment
 *
 */
enum psa_cd_player_status
{
    PSA_CD_PLAYER_PLAYING = (uint8_t)(1 << 0),
    PSA_CD_PLAYER_PAUSED = (uint8_t)(1 << 1),
    PSA_CD_PLAYER_FAST_FORWARDING = (uint8_t)(1 << 2),
    PSA_CD_PLAYER_REWINDING = (uint8_t)(1 << 3),
    PSA_CD_PLAYER_LOADING = (uint8_t)(1 << 4),
    PSA_CD_PLAYER_EJECTING = (uint8_t)(1 << 5),
    PSA_CD_PLAYER_ERROR = (uint8_t)(1 << 6),
};

enum psa_cd_changer_status
{
    PSA_CD_CHANGER_OFF = (uint8_t)0x41,
    PSA_CD_CHANGER_ON = (uint8_t)0xC1,
    PSA_CD_CHANGER_BUSY = (uint8_t)0xD3,
    PSA_CD_CHANGER_PLAYING = (uint8_t)0xC3,
    PSA_CD_CHANGER_FAST_FORWARD = (uint8_t)0xC4,
    PSA_CD_CHANGER_FAST_REWIND = (uint8_t)0xC5,
};

enum psa_cd_changer_commands
{
    PSA_CD_CHANGER_COMM_NONE = 0x00,      // No new command
    PSA_CD_CHANGER_COMM_PLAY = 0x10,      // Command to start playback
    PSA_CD_CHANGER_COMM_PAUSE = 0x11,     // Command to pause playback
```

```
    PSA_CD_CHANGER_COMM_PREV_TRACK = 0x20, // Command to go to previous track
    PSA_CD_CHANGER_COMM_NEXT_TRACK = 0x21, // Command to go to next track
};

struct psa_header
{
    /* Packet start, 0x69 */
    uint8_t start;
    /* < when sending to the esp, > when receiving from the esp */
    uint8_t direction;
    /* Packet ident. Taken into CRC calc */
    uint16_t ident;
    /* Payload size. Taken into CRC calc */
    uint8_t size;
} __attribute__((packed));

struct psa_version_data
{
    uint8_t firmware_version_major;
    uint8_t firmware_version_minor;
    uint8_t api_version;
} __attribute__((packed));

struct psa_version_packet
{
    struct psa_header header;
    struct psa_version_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_engine_data
{
    uint16_t rpm;             // RPM * 8
    uint16_t speed;           // km/h * 100
```

```
    uint8_t fuel_level;        // Fuel level in %. Does not appear on all cars
    uint8_t engine_temperature; // Engine water temperature + 40 in Celcius. To display, subtract 40. 0xFF w
hen invalid
    uint8_t reverse;           // Is the car in reverse
    uint8_t oil_temperature;    // Oil temperature + 40 in Celcius. 0xFF when invalid
    uint8_t reserved;          // reserved
} __attribute__((packed));

struct psa_engine_packet
{
    struct psa_header header;
    struct psa_engine_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_radio_data
{
    uint16_t freq;          // freq * 100
    uint8_t band;           // Radio band, enum psa_radio_band
    uint8_t preset;         // Preset number 1 - 6
    uint8_t signal_strength; // Signal Strength 0 - 15, probably. The values are random
    char station[9];        // ASCII station name. Do not treat as NULL-terminated.
} __attribute__((packed));

struct psa_radio_packet
{
    struct psa_header header;
    struct psa_radio_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_preset_info
{
    uint8_t preset_num;   // Preset number 1 - 6
```

```
    char preset_name[10]; // Preset station name
} __attribute__((packed));

struct psa_preset_data
{
    struct psa_preset_info presets[6];
} __attribute__((packed));

struct psa_preset_packet
{
    struct psa_header header;
    struct psa_preset_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_cd_player_data
{
    uint8_t status; // enum psa_cd_player_status

    uint8_t current_minute; // Current minute in decimal
    uint8_t total_minutes;  // Total minuts in decimal
    uint8_t current_second; // Current second in decimal
    uint8_t total_seconds;  // Not total_minutes*60, but ss in mm:ss. In decimal
    uint8_t current_track;  // Current track number in decimal
    uint8_t total_tracks;   // Total number of tracks in decimal
} __attribute__((packed));

struct psa_cd_player_packet
{
    struct psa_header header;
    struct psa_cd_player_data data;
    uint8_t crc;
} __attribute__((packed));
```

```
/**
 * @brief What settings are being changed (selected).
 * Used for displaying audio settings display
 *
 */
struct psa_headunit_settings
{
    uint8_t volume_changing : 1;      // The volume is changing. Not used
    uint8_t bass_changing : 1;        // The bass value is changing / selected
    uint8_t treble_changing : 1;      // The treble value is changing / selected
    uint8_t fader_changing : 1;       // The fader value is changing / selected
    uint8_t balance_changing : 1;     // The balance value is changing / selected
    uint8_t auto_volume_changing : 1; // The auto-volume value is changing / selected
    uint8_t loudness_changing : 1;    // The LOUDNESS value is changing / selected
    uint8_t : 1;                      // Unused
} __attribute__((packed));

struct psa_headunit_data
{
    uint8_t unit_powered_on;    // 1 if powered on
    uint8_t source;             // enum psa_radio_source
    uint8_t muted;              // is muted
    uint8_t cd_present;         // is CD present
    uint8_t tape_present;       // is TAPE present
    uint8_t volume;             // Current volume
    int8_t bass;                // Current bass
    int8_t treble;              // Current treble
    int8_t fader;               // Current fader
    int8_t balance;             // Current balance
    uint8_t auto_volume;        // Current auto volume setting
    uint8_t loudness_on;        // Current loudness setting
    uint8_t settings_menu_open; // Is settings menu open. Not always accurate.

    struct psa_headunit_settings settings_changing; // What settings are being updated
```

```
} __attribute__((packed));

struct psa_headunit_packet
{
    struct psa_header header;
    struct psa_headunit_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_door_data
{
    uint8_t door_open; // Is a door open?

    uint8_t open_doors; // What doors are open. Check below comment for clarification

    /*
        Open doors bitfields:

        uint8_t fuel_flap : 1;   // bit 0
        uint8_t sunroof : 1;     // bit 1
        uint8_t hood : 1;        // bit 2
        uint8_t boot_lid : 1;    // bit 3
        uint8_t rear_left : 1;   // bit 4
        uint8_t rear_right : 1;  // bit 5
        uint8_t front_left : 1;  // bit 6
        uint8_t front_right : 1; // bit 7
    */
} __attribute__((packed));

struct psa_door_packet
{
    struct psa_header header;
```

```
    struct psa_door_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_dash_data
{
    uint8_t brightness;       // 0 to 15. Dashboard brightness
    uint8_t backlight_off : 1; // 1 when interior backlight is off

    uint8_t accesories_on : 1;  // Key in ACC
    uint8_t ignition_on : 1;    // Key in IGN
    uint8_t engine_running : 1; // Engine on
    uint8_t door_open : 1;      // A door is open
    uint8_t economy_mode : 1;   // ECO MODE for the radio.
    uint8_t reverse_gear : 1;   // Gearbox in reverse

    uint8_t water_temp;    // Subtract by 40
    uint32_t mileage;      // Divide by 10
    uint8_t external_temp; // 0xFF if not present, otherwise divide by 2 and subtract 40.

} __attribute__((packed));

struct psa_dash_packet
{
    struct psa_header header;
    struct psa_dash_data data;
    uint8_t crc;

} __attribute__((packed));

/**
 * @brief Data from the trip computer.
 *
```

```
 *
 */
struct psa_trip_data
{
    uint16_t trip_1_distance;
    uint16_t trip_2_distance;
    uint16_t trip_1_fuel_consumption;
    uint16_t trip_2_fuel_consumption;
    uint16_t current_fuel_consumption;
    uint16_t distance_to_empty;

    uint8_t trip_1_speed;
    uint8_t trip_2_speed;
    uint8_t trip_button_pressed;

} __attribute__((packed));

struct psa_trip_packet
{
    struct psa_header header;
    struct psa_trip_data data;
    uint8_t crc;

} __attribute__((packed));

/**
 * @brief Random assortment of data that doesnt fit anywhere else.
 * Also send the car state, and cd changer command.
 *
 */
struct psa_status_data
{
    uint8_t doors_locked;
    uint8_t deadlocking_active;
```

```
    uint8_t car_state;
    uint8_t cd_changer_command;
    uint16_t voltage;
    uint8_t key_in_ignition;
    uint8_t reserved[11];
} __attribute__((packed));

struct psa_status_packet
{
    struct psa_header header;
    struct psa_status_data data;
    uint8_t crc;
} __attribute__((packed));

/**
 * @brief The current time as reported by the ESP
 *
 */
struct psa_time_data
{
    uint8_t valid_time; // 1 if time is set
    uint8_t day;        // 1 - 31
    uint8_t month;      // 0 - 11
    uint8_t year;       // years since 1900
    uint8_t hour;       // 0 - 24
    uint8_t minutes;    // 0 - 59
    uint8_t seconds;    // 0 - 61
} __attribute__((packed));

struct psa_time_packet
{
    struct psa_header header;
    struct psa_time_data data;
    uint8_t crc;
```

```
} __attribute__((packed));

struct psa_esp32_temp_data
{
   uint16_t temperature;
   uint16_t cpu_freq;
} __attribute__((packed));

struct psa_esp32_temp_packet
{
   struct psa_header header;
   struct psa_esp32_temp_data data;
   uint8_t crc;
} __attribute__((packed));

struct psa_vin_packet
{
   struct psa_header header;
   uint8_t vin[17]; // VIN in ASCII. NOT null-terminated
   uint8_t crc;

} __attribute__((packed));

// SETTER STRUCTS

struct psa_cd_changer_data
{
   uint8_t status;            // enum psa_cd_changer_status
   uint8_t track_time_seconds; // Track seconds in decimal
   uint8_t track_time_minutes; // Track minutes in decimal
   uint8_t track_number;      // Track number in decimal
   uint8_t cd_number;         // CD number in decimal
   uint8_t total_tracks;      // Total tracks in decimal
   uint8_t cds_present;       // Bitwise presentation of present cds
```

```
} __attribute__((packed));

struct psa_cd_changer_packet
{
   struct psa_header header;
   struct psa_cd_changer_data data;
   uint8_t crc;
} __attribute__((packed));

struct psa_trip_reset_data
{
   uint8_t trip_meter; // enum psa_trip_meter, trip meter to reset
} __attribute__((packed));

struct psa_trip_reset_packet
{
   struct psa_header header;
   struct psa_trip_reset_data data;
   uint8_t crc;
} __attribute__((packed));

#endif

#ifndef TSS463C_REGS_H
#define TSS463C_REGS_H

#include <stdint.h>

// Register addresses

#define TSS_LINECONTROL 0x00       // r/w   - 0x00
#define TSS_TRANSMITCONTROL 0x01   // r/w   - 0x02
#define TSS_DIAGNOSISCONTROL 0x02  // r/w   - 0x00
#define TSS_COMMANDREGISTER 0x03   // w     - 0x00
```

```
#define TSS_LINESTATUS 0x04        // r     - 0bx01xxx00
#define TSS_TRANSMITSTATUS 0x05    // r     - 0x00
#define TSS_LASTMESSAGESTATUS 0x06 // r     - 0x00
#define TSS_LASTERRORSTATUS 0x07   // r     - 0x00

/* Interrupt registers */

#define TSS_INTERRUPTSTATUS 0x09   // r     - 0x80
#define TSS_INTERRUPTENABLE 0x0A   // r/w   - 0x80
#define TSS_INTERRUPTRESET 0x0B    // w

// Used with Interrupt Status regiter, Interrupt enable register and Interrupt reset register
typedef union
{
   struct
   {
      // Receive "with no RAK (RAK=0)" OK Status Flag
      uint8_t RNOK : 1;
      // Receive "with RAK (RAK=1)" OK Status Flag
      uint8_t ROK : 1;
      // Receive Error Status Flag
      uint8_t RE : 1;
      // Transmit OK Status Flag
      uint8_t TOK : 1;
      // Transmit Error Status Flag (or Exceeded Retry)
      uint8_t TE : 1;
      // Must be 0
      uint8_t ZERO : 2;
      // Interrupt on chip reset, must be 1 in interrupt enable
      uint8_t RESET : 1;
   } data;
   uint8_t Value;
} interrupt_register_t;
```

```
typedef union
{
  struct
  {
    uint8_t CHRx : 1; // Channel message received
    uint8_t CHTx : 1; // Channel message transmitted
    /*
    As status, this bit is set by the TSS463C when error occurs in transmission or on a received frame. The u
ser must reset it
    */
    uint8_t CHER : 1;
    /*
    The 5 high bits of this register allows the user to specify either the length of the message to be transmitte
d, or the maximum length of a message receivable in the pointed reception buffer
    The first byte in this register does not contain data, but the length of the message received. This implies t
hat the length value has to be equal to or greater than the maximum length of a message
    to be received in this buffer (or the length of a message to be transmitted) plus 1
    */
    uint8_t M_L : 5;
  } data;
  uint8_t Value;
} message_length_and_status_register_t;

typedef union
{
  struct
  {
    // Length of the received message
    uint8_t received_message_len : 5;
    // Status of the received RTR bit
    uint8_t RTR : 1;
    // Status of the received RNW bit
    uint8_t RNW : 1;
    // Status of the received RAK bit
```

```
        uint8_t RRAK : 1;
    } data;
    uint8_t Value;
} received_message_and_status_register_t;

typedef union
{
    struct
    {
        uint8_t RTR : 1;
        uint8_t RNW : 1;
        uint8_t RAK : 1;
        uint8_t EXT : 1;
        uint8_t ID : 4;
    } data;
    uint8_t Value;
} id2_and_command_register_t;

typedef union
{
    struct
    {
        // Message Pointer the value in this register is the offset from 0x80
        uint8_t message_pointer : 7;
        /*Disable RAK (Used in 'Spy Mode')
        In reception: whatever is the RAK bit of the incoming valid frame, no ACK answer will be set. If the mes
sage was successfully received, an IT is set (ROK or RNOK).
        In transmission: no action.
        One: disable active, 'spy mode'.
        Zero: disable inactive, normal operation
        */
        uint8_t DRAK : 1;
    } data;
    uint8_t Value;
```

```
} message_pointer_register_t;

typedef union
{
  struct
  {
    // Module type:
    // 1 - Master/autonomous module (rank 0,1, and 16)
    // 0 - Synchronous/slave module (rank 1 and 16)
    uint8_t module_type : 1;
    // Must be 0b001
    uint8_t VER : 3;
    // Maximum retries
    uint8_t max_retries : 4;
  } data;
  uint8_t Value;
} transmit_control_register_t;

typedef union
{
  struct
  {
    uint8_t ESDC : 1;
    uint8_t ETIP : 1;
    uint8_t Mb : 1;
    uint8_t Ma : 1;
    uint8_t SDC : 4;
  } data;
  uint8_t Value;
} diagnosis_control_register_t;

// Page 30
typedef union
{
```

```
    struct
    {
        uint8_t MSDC : 1;
        uint8_t ZERO : 2; // Must be 0
        // Rearbitrate -> Reset the retry counter and rearbitrate all messages
        uint8_t REAR : 1;
        // Activate command -> Page 51
        uint8_t ACTI : 1;
        // Idle command -> Page 51
        uint8_t IDLE : 1;
        // Sleep command -> puts the chip to sleep
        uint8_t SLEEP : 1;
        // General reset -> resets the chip
        uint8_t GRES : 1;
    } data;
    uint8_t Value;
} command_register_t;

// Page 31
typedef union
{
    struct
    {
        // Receiving status -> 1 when there is activity on the bus
        uint8_t RXG : 1;
        // Transmitting status -> TSS is transimitting a message
        uint8_t TXG : 1;
        // System diagnostics bits -> Table 8
        uint8_t Sa : 1;
        uint8_t Sb : 1;
        uint8_t Sc : 1;
        // Chip is in Idle state
        uint8_t IDG : 1;
        // Chip is in sleep state
```

```
        uint8_t SPG : 1;
        uint8_t : 1;
    } data;
    uint8_t Value;
} line_status_register_t;

// Page 32
typedef union
{
    struct
    {
        // Identifier currently transmitting
        uint8_t IDT : 4;
        // Number of retries done
        uint8_t NRT : 4;
    } data;
    uint8_t Value;
} transmission_status_register_t;

// Page 32
typedef union
{
    struct
    {
        // Identifier transmitted, received, or exceeded the retry count
        uint8_t IDTr : 4;
        // Number of retries done
        uint8_t NRTr : 4;
    } data;
    uint8_t Value;
} last_message_status_register_t;

// Page 32
typedef union
```

```
{
  struct
  {
    uint8_t frame_violation : 1;
    uint8_t code_violation : 1;
    uint8_t ack_error : 1;
    uint8_t crc_error : 1;
    uint8_t : 1;
    uint8_t buffer_overflow : 1;
    uint8_t buffer_occupied : 1;
    uint8_t : 1;
  } data;
  uint8_t Value;
} last_error_status_register_t;

#endif

#ifndef TSS463C_H
#define TSS463C_H

#include <stdint.h>
#include "driver/spi_master.h"
#include "freertos/semphr.h"

#define TSS_CHANNEL_ADDR(x) (0x10 + (0x08 * x))
#define TSS_CHANNEL_MESS_PTR(x) TSS_CHANNEL_ADDR(x) + 2
#define TSS_CHANNEL_MESS_LEN_AND_STATUS(x) TSS_CHANNEL_ADDR(x) + 3
#define TSS_MAILBOX_OFFSET 0x80
#define TSS_GETMAIL(x) (TSS_MAILBOX_OFFSET + x)

#define TSS_8MHz_62k5BPS 0x30
#define TSS_8MHz_125kBPS 0x20

#define TSS_SELECT() gpio_set_level(instance->cs_pin, 0)
```

```
// #define TSS_SELECT()
#define TSS_DESELECT() gpio_set_level(instance->cs_pin, 1)
// #define TSS_DESELECT()


#define TSS_WRITE 0xE0
#define TSS_READ 0x60

#define TSS_MAX_CHANNEL 14
#define TSS_MAX_MEMORY_ADDR 0x7f

enum tss_message_type {
    TSS_NONE = 0,
    TSS_TRANSMIT,
    TSS_TRANSMIT_NOACK,
    TSS_REPLY_REQUEST,
    TSS_IMM_REPLY,
    TSS_DEF_REPLY,
};

enum tss_chip_mode {
    TSS_MODE_IDLE = 0,
    TSS_MODE_ACTIVE,
    TSS_MODE_SLEEP,
};

struct tss_channel_t
{
    uint16_t identifier; // Ident registered to the channel
    uint8_t ram_address; // The RAM address in use
    uint8_t data_size;   // Memory size allocated
    uint8_t message_type; // Message type, see enum tss_message_type
    uint8_t is_occupied; // Is the channel registered?
    uint8_t is_busy;     // Is the channel doing something?
```

```
    uint8_t tx_attempt;  // Number of attempts to transmit said channel (starts at 1)
    uint8_t message_len_and_status_reg_value;
};

typedef struct tss_channel_t tss_channel_t;

typedef struct
{
    uint8_t start;
    uint8_t end;
} tss_ringbuffer_t;

struct tss_instance_t
{
    spi_device_handle_t tss_handle;
    uint8_t *tx_buffer;
    uint8_t *rx_buffer;
    uint32_t buffers_size;
    uint8_t cs_pin;
    uint8_t interrupt_pin;
    enum tss_chip_mode chip_mode;

    SemaphoreHandle_t tss_spi_semaphore;
    tss_ringbuffer_t tss_ringbuffer;
    tss_channel_t channels[TSS_MAX_CHANNEL];
};

typedef struct tss_instance_t tss_instance_t;

typedef enum
{
    TSS_EVENT_TRANSMIT_CHANNEL = 0, // Queue specified channel for transmission (first time)
    TSS_EVENT_CHECK_CHANNEL, // Check on the channel's status
    TSS_EVENT_RETRANSMIT_CHANNEL, // Retransmit specified channel
```

```
    TSS_EVENT_CLEAN_CHANNEL, // Cleanup channel after transmission
    TSS_EVENT_CANCEL_CHANNEL, // Abort sending channel

    TSS_EVENT_MAX,
} tss_event_t;

typedef struct
{
    tss_event_t event_type;
    int channel_num;
} tss_event_command;


extern tss_instance_t global_tss_instance;

/* ================================================================
= */

int tss_register_get(
    tss_instance_t *const instance,
    uint8_t const reg_addr,
    uint8_t *const reg_value);

int tss_register_set(
    tss_instance_t *instance,
    uint8_t reg_addr,
    uint8_t reg_value);

int tss_registers_set(
    tss_instance_t *const instance,
    uint8_t const reg_addr,
    uint8_t const *const values,
    uint8_t const count);
```

```
int tss_registers_get(
    tss_instance_t *const instance,
    uint8_t const reg_addr,
    uint8_t *const buffer,
    uint8_t count);

/* ===================================================================
= */

/**
 * @brief Get the next available mailbox address to use for channel
 *
 *
 * @param instance
 * @param buf_size
 * @param addr
 * @return MIVE_OK or MIVE_ERR_OUTOFMEMORY or MIVE_ERR_INVALID_ARGUMENT
 */
int tss_get_memory_address(
    tss_instance_t* const instance,
    uint8_t const channel_num,
    uint8_t const buf_size,
    uint8_t *const addr);

/* ===================================================================
= */

int tss_abort_channel(
    tss_instance_t* const instance,
    uint8_t const channel_num);

int tss_free_channel(
    tss_instance_t* const instance,
```

```
    uint8_t const channel_num);

int tss_retransmit_channel(
    tss_instance_t* const instance,
    uint8_t const channel_num);

/* ================================================================
= */
/* Convinience functions to get certain register values */

uint8_t tss_get_channel_status_register(
    tss_instance_t* const instance,
    uint8_t const channel_num);

/* ================================================================
= */
/* Only transmissions are handled by the TSS, the receiving is done */
/* by the RMT peripheral                                            */

int tss_transmit_message(
    tss_instance_t* const instance,
    uint8_t channel,
    uint16_t const iden,
    uint8_t *const data,
    uint8_t const data_size,
    uint8_t const memory_offset,
    uint8_t const rak);

int tss_reply_request_message(
    tss_instance_t* const instance,
    uint8_t channel,
    uint16_t const iden,
    uint8_t const data_size,
    uint8_t const memory_offset);
```

```
int tss_immediate_reply_message(
    tss_instance_t* const instance,
    uint8_t channel,
    uint16_t const iden,
    uint8_t *const data,
    uint8_t const data_size,
    uint8_t const memory_offset);

int tss_deferred_reply_message(
    tss_instance_t* const instance,
    uint8_t channel,
    uint16_t const iden,
    uint8_t *const data,
    uint8_t const data_size,
    uint8_t const memory_offset);

/* ================================================================
= */

/**
 * @brief Create and allocate resources for use with the TSS463C.
 * Initializes the SPI driver.
 *
 * @param instance
 * @return int
 */
int tss_create(tss_instance_t* instance);

/**
 * @brief Reset the TSS463C and put it in active mode.
 *
 * @param instance
 * @return int
```

```
 */
int tss_start(
    tss_instance_t* const instance);

void tss_task(void *params);

/* ================================================================
= */

int tss_activate(tss_instance_t *instance);
int tss_sleep(tss_instance_t *instance);
int tss_idle(tss_instance_t *instance);

#endif

#ifndef PSA_VAN_PACKET_IDEN_H
#define PSA_VAN_PACKET_IDEN_H

// VAN IDENtifiers

enum psa_van_iden
{
    PSA_VAN_IDEN_VIN = 0xE24,
    PSA_VAN_IDEN_ENGINE = 0x8A4,
    PSA_VAN_IDEN_HEAD_UNIT_STALK = 0x9C4,
    PSA_VAN_IDEN_LIGHTS_STATUS = 0x4FC,
    PSA_VAN_IDEN_DEVICE_REPORT = 0x8C4,
    PSA_VAN_IDEN_CAR_STATUS1 = 0x564,
    PSA_VAN_IDEN_CAR_STATUS2 = 0x524,
    PSA_VAN_IDEN_RPM = 0x824,
    PSA_VAN_IDEN_DASHBOARD_BUTTONS = 0x664,
    PSA_VAN_IDEN_HEAD_UNIT = 0x554,
    PSA_VAN_IDEN_TIME = 0x984,
    PSA_VAN_IDEN_AUDIO_SETTINGS = 0x4D4,
```

```
    PSA_VAN_IDEN_MFD_STATUS = 0x5E4,
    PSA_VAN_IDEN_AIRCON1 = 0x464,
    PSA_VAN_IDEN_AIRCON2 = 0x4DC,
    PSA_VAN_IDEN_CDCHANGER = 0x4EC,
    PSA_VAN_IDEN_SATNAV_STATUS_1 = 0x54E,
    PSA_VAN_IDEN_SATNAV_STATUS_2 = 0x7CE,
    PSA_VAN_IDEN_SATNAV_STATUS_3 = 0x8CE,
    PSA_VAN_IDEN_SATNAV_GUIDANCE_DATA = 0x9CE,
    PSA_VAN_IDEN_SATNAV_GUIDANCE = 0x64E,
    PSA_VAN_IDEN_SATNAV_REPORT = 0x6CE,
    PSA_VAN_IDEN_MFD_TO_SATNAV = 0x94E,
    PSA_VAN_IDEN_SATNAV_TO_MFD = 0x74E,
    PSA_VAN_IDEN_SATNAV_DOWNLOADING = 0x6F4,
    PSA_VAN_IDEN_SATNAV_DOWNLOADED1 = 0xA44,
    PSA_VAN_IDEN_SATNAV_DOWNLOADED2 = 0xAC4,
    PSA_VAN_IDEN_WHEEL_SPEED = 0x744,
    PSA_VAN_IDEN_ODOMETER = 0x8FC,
    PSA_VAN_IDEN_COM2000 = 0x450,
    PSA_VAN_IDEN_CDCHANGER_COMMAND = 0x8EC,
    PSA_VAN_IDEN_MFD_TO_HEAD_UNIT = 0x8D4,
    PSA_VAN_IDEN_AIR_CONDITIONER_DIAG = 0xADC,
    PSA_VAN_IDEN_AIR_CONDITIONER_DIAG_COMMAND = 0xA5C,
    PSA_VAN_IDEN_ECU = 0xB0E,
};

#endif

#ifndef VAN_PACKET_PARSER_H
#define VAN_PACKET_PARSER_H

#include <stdint.h>

#include "../mive/psa_packet_defs.h"
```

```
struct psa_output_data_buffers
{
    struct psa_engine_data *engine_data; // PSA_IDENT_ENGINE
    struct psa_radio_data *radio_data; // PSA_IDENT_RADIO
    struct psa_preset_data *presets_data_am; // PSA_IDENT_RADIO_PRESETS
    struct psa_preset_data *presets_data_fm_1; // PSA_IDENT_RADIO_PRESETS
    struct psa_preset_data *presets_data_fm_2; // PSA_IDENT_RADIO_PRESETS
    struct psa_preset_data *presets_data_fm_ast; // PSA_IDENT_RADIO_PRESETS
    struct psa_cd_player_data *cd_player_data; // PSA_IDENT_CD_PLAYER
    struct psa_headunit_data *headunit_data; // PSA_IDENT_HEADUNIT
    struct psa_door_data *door_data; // PSA_IDENT_DOORS
    uint8_t *vin_data; // PSA_IDENT_VIN
    struct psa_dash_data *dash_data; // PSA_IDENT_DASHBOARD
    struct psa_trip_data *trip_data; // PSA_IDENT_TRIP
    struct psa_status_data *status_data; // PSA_IDENT_CAR_STATUS
};

extern int psa_parse_van_packet(
    uint16_t iden,
    uint8_t size,
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers);

#endif // VAN_PACKET_PARSER_H

#ifndef VAN_RMT_RX_H
#define VAN_RMT_RX_H

#include <stdint.h>
#include <stdbool.h>

#include "driver/rmt_rx.h"
#include "freertos/queue.h"
```

```
typedef enum
{
    RX_VAN_LINE_LEVEL_LOW = 0,
    RX_VAN_LINE_LEVEL_HIGH = 1,
} RX_VAN_LINE_LEVEL;

typedef enum
{
    RX_VAN_NETWORK_BODY = 0,
    RX_VAN_NETWORK_COMFORT = 1,
} RX_VAN_NETWORK_TYPE;

typedef struct {
    int rmt_rx_ledPin;
    int rmt_rx_rxPin;
    uint8_t rmt_rx_channel;
    uint8_t rmt_rx_time_slice_divisor;
    RX_VAN_LINE_LEVEL rmt_rx_van_line_level;

    rmt_channel_handle_t rx_chan;
    rmt_receive_config_t rx_recv_config;
    QueueHandle_t receive_queue;
    rmt_symbol_word_t *rmt_symbol_buffer;
    size_t rmt_symbol_buffer_size;

} van_rmt_rx_instance_t;

void van_rmt_rx_channel_stop_new(van_rmt_rx_instance_t* instance);
void van_rmt_rx_channel_init_new(
    van_rmt_rx_instance_t* instance,
    uint8_t channel,
    int rxPin,
    int ledPin,
    RX_VAN_LINE_LEVEL vanLineLevel,
```

```
    RX_VAN_NETWORK_TYPE vanNetworkType);
void van_rmt_rx_channel_start_new(van_rmt_rx_instance_t* instance);

bool van_rmt_rx_parse_byte(van_rmt_rx_instance_t* instance, uint8_t level, uint32_t duration, uint8_t *bitC
ounter, uint8_t *tempByte, uint8_t *mask, uint8_t *finalByte);
uint16_t van_rmt_rx_crc15(uint8_t data[], uint8_t lengthOfData);
bool van_rmt_rx_is_crc_ok(uint8_t vanMessage[], int vanMessageLength);



#endif // VAN_RMT_RX_H

#ifndef PSA_VAN_CAR_STATUS_STRUCT_H
#define PSA_VAN_CAR_STATUS_STRUCT_H

#include <stdint.h>

// Iden 0x524

typedef struct {
    uint8_t tyre_pressure_low                  : 1; // bit 0
    uint8_t no_door_open                       : 1; // bit 1
    uint8_t automatic_gearbox_temperature_too_high : 1; // bit 2
    uint8_t brake_system_fault                 : 1; // bit 3
    uint8_t hydraulic_suspension_pressure_faulty   : 1; // bit 4
    uint8_t suspension_faulty                  : 1; // bit 5
    uint8_t engine_oil_temperature_too_high        : 1; // bit 6
    uint8_t engine_overheating                 : 1; // bit 7
} VanDisplayMsgV2Byte0Struct;

typedef struct {
    uint8_t unblock_diesel_filter     : 1; // bit 0
    uint8_t stop_car_icon             : 1; // bit 1
    uint8_t diesel_additive_too_low   : 1; // bit 2
    uint8_t fuel_tank_access_open     : 1; // bit 3
```

```
    uint8_t tyre_punctured          : 1; // bit 4
    uint8_t coolant_level_low       : 1; // bit 5
    uint8_t oil_pressure_too_low    : 1; // bit 6
    uint8_t oil_level_too_low       : 1; // bit 7
} VanDisplayMsgV2Byte1Struct;

typedef struct {
    uint8_t mil                     : 1; // bit 0  displays Antipollution fault
    uint8_t brake_pads_worn         : 1; // bit 1
    uint8_t diagnosis_ok            : 1; // bit 2
    uint8_t automatic_gearbox_faulty : 1; // bit 3
    uint8_t esp                     : 1; // bit 4  displays ESP faulty
    uint8_t abs                     : 1; // bit 5  displays ABS fault
    uint8_t suspension_or_steering_fault : 1; // bit 6
    uint8_t braking_system_faulty   : 1; // bit 7
} VanDisplayMsgV2Byte2Struct;

typedef struct {
    uint8_t side_airbag_faulty      : 1; // bit 0
    uint8_t airbags_faulty          : 1; // bit 1
    uint8_t cruise_control_faulty   : 1; // bit 2
    uint8_t engine_temperature_too_high : 1; // bit 3
    uint8_t fault_load_shedding_in_progress : 1; // bit 4
    uint8_t ambient_brightness_sensor_fault : 1; // bit 5
    uint8_t rain_sensor_fault       : 1; // bit 6
    uint8_t water_in_diesel_fuel_filter : 1; // bit 7
} VanDisplayMsgV2Byte3Struct;

typedef struct {
    uint8_t left_rear_sliding_door_faulty : 1; // bit 0
    uint8_t headlight_corrector_fault : 1; // bit 1
    uint8_t right_rear_sliding_door_faulty : 1; // bit 2
    uint8_t no_broken_lamp          : 1; // bit 3
    uint8_t battery_low             : 1; // bit 4
```

```
    uint8_t battery_charge_fault          : 1; // bit 5
    uint8_t diesel_particle_filter_faulty  : 1; // bit 6
    uint8_t catalytic_converter_fault      : 1; // bit 7
} VanDisplayMsgV2Byte4Struct;

typedef struct {
    uint8_t handbrake                      : 1; // bit 0
    uint8_t seatbelt_warning               : 1; // bit 1  lights the instrument cluster also
    uint8_t passenger_airbag_deactivated   : 1; // bit 2
    uint8_t screen_washer_liquid_level_too_low : 1; // bit 3
    uint8_t current_speed_too_high         : 1; // bit 4
    uint8_t ignition_key_left_in           : 1; // bit 5
    uint8_t sidelights_left_on             : 1; // bit 6
    uint8_t hill_holder_active             : 1; // bit 7  on RT3 monochrome: Driver's seatbelt not fastened
} VanDisplayMsgV2Byte5Struct;

typedef struct {
    uint8_t shock_sensor_faulty            : 1; // bit 0
    uint8_t seatbelt_warning_              : 1; // bit 1 not sure if it is seatbelt warning
    uint8_t check_and_reinitialize_tyre_pressure : 1; // bit 2
    uint8_t remote_control_battery_low     : 1; // bit 3
    uint8_t left_stick_button              : 1; // bit 4
    uint8_t put_automatic_gearbox_in_P_position  : 1; // bit 5
    uint8_t stop_lights_test_brake_gently  : 1; // bit 6
    uint8_t fuel_level_low                 : 1; // bit 7  lights the instrument cluster also
} VanDisplayMsgV2Byte6Struct;

typedef struct {
    uint8_t automatic_headlamp_lighting_deactivated : 1; // bit 0
    uint8_t rear_lh_passenger_seatbelt_not_fastened : 1; // bit 1
    uint8_t rear_rh_passenger_seatbelt_not_fastened : 1; // bit 2
    uint8_t front_passenger_seatbelt_not_fastened   : 1; // bit 3
    uint8_t driving_school_pedals_indication        : 1; // bit 4
    uint8_t number_of_tpms_missing                  : 3; // bit 5-7  RT3 displays: X tyre pressure sensors missing w
```

```
here x equals this number
} VanDisplayMsgV2Byte7Struct;

typedef struct {
    uint8_t doors_locked              : 1; // bit 0
    uint8_t esp_asr_deactivated       : 1; // bit 1
    uint8_t child_safety_activated    : 1; // bit 2
    uint8_t deadlocking_active        : 1; // bit 3
    uint8_t automatic_lighting_active : 1; // bit 4
    uint8_t automatic_wiping_active   : 1; // bit 5
    uint8_t engine_immobilizer_fault  : 1; // bit 6
    uint8_t sport_suspension_mode_active : 1; // bit 7
} VanDisplayMsgV2Byte8Struct;

typedef struct {
    uint8_t                           : 1; // bit 0
    uint8_t                           : 1; // bit 1
    uint8_t                           : 1; // bit 2
    uint8_t change_of_fuel_used_in_progress : 1; // bit 3
    uint8_t lpg_fuel_refused          : 1; // bit 4
    uint8_t lpg_system_faulty         : 1; // bit 5
    uint8_t lpg_in_use                : 1; // bit 6
    uint8_t min_level_lpg             : 1; // bit 7
} VanDisplayMsgV2Byte10Struct;

typedef struct {
    uint8_t adin_fault                : 1; // bit 0
    uint8_t use_stop_start            : 1; // bit 1
    uint8_t stop_start_available      : 1; // bit 2
    uint8_t stop_start_activated      : 1; // bit 3
    uint8_t stop_start_deactivated    : 1; // bit 4
    uint8_t stop_start_deferred       : 1; // bit 5
    uint8_t unknown6                  : 1; // bit 6 displays empty messagebox on RT3 monochrome when the Mess
age byte is 0x5E
```

```
    uint8_t unknown7                : 1; // bit 7 displays empty messagebox on RT3 monochrome when the Mess
age byte is 0x5F
} VanDisplayMsgV2Byte11Struct;

typedef struct {
    uint8_t                         : 1; // bit 0
    uint8_t                         : 1; // bit 1
    uint8_t                         : 1; // bit 2
    uint8_t                         : 1; // bit 3
    uint8_t                         : 1; // bit 4
    uint8_t                         : 1; // bit 5
    uint8_t                         : 1; // bit 6
    uint8_t change_to_neutral       : 1; // bit 7
} VanDisplayMsgV2Byte12Struct;


//Read left to right in documentation
struct psa_van_car_status_2_long {
    VanDisplayMsgV2Byte0Struct Field0;
    VanDisplayMsgV2Byte1Struct Field1;
    VanDisplayMsgV2Byte2Struct Field2;
    VanDisplayMsgV2Byte3Struct Field3;
    VanDisplayMsgV2Byte4Struct Field4;
    VanDisplayMsgV2Byte5Struct Field5;
    VanDisplayMsgV2Byte6Struct Field6;
    VanDisplayMsgV2Byte7Struct Field7;
    VanDisplayMsgV2Byte8Struct Field8;
    uint8_t Message;
    VanDisplayMsgV2Byte10Struct Field10;
    VanDisplayMsgV2Byte11Struct Field11;
    VanDisplayMsgV2Byte12Struct Field12;
    uint8_t Field13;
    uint8_t Field14;
    uint8_t Field15;
```

```
} __attribute__((packed));

struct psa_van_car_status_2_short {
    VanDisplayMsgV2Byte0Struct Field0;
    VanDisplayMsgV2Byte1Struct Field1;
    VanDisplayMsgV2Byte2Struct Field2;
    VanDisplayMsgV2Byte3Struct Field3;
    VanDisplayMsgV2Byte4Struct Field4;
    VanDisplayMsgV2Byte5Struct Field5;
    VanDisplayMsgV2Byte6Struct Field6;
    VanDisplayMsgV2Byte7Struct Field7;
    VanDisplayMsgV2Byte8Struct Field8;
    uint8_t Message;
    VanDisplayMsgV2Byte10Struct Field10;
    VanDisplayMsgV2Byte11Struct Field11;
    VanDisplayMsgV2Byte12Struct Field12;
    uint8_t Field13;
} __attribute__((packed));

#endif // PSA_VAN_CAR_STATUS_STRUCT_H

#ifndef VAN_CD_CHANGER_COMMANDS_H
#define VAN_CD_CHANGER_COMMANDS_H

#include <stdint.h>

enum psa_van_cd_changer_commands
{
    PSA_VAN_CD_CHANGER_POWER_OFF = 0x1101,
    PSA_VAN_CD_CHANGER_POWER_OFF_2 = 0x2101,
    PSA_VAN_CD_CHANGER_PAUSE = 0x1181,
    PSA_VAN_CD_CHANGER_PLAY = 0x1183,
    PSA_VAN_CD_CHANGER_PREVIOUS_TRACK = 0x31FE,
    PSA_VAN_CD_CHANGER_NEXT_TRACK = 0X31FF,
```

```
    PSA_VAN_CD_CHANGER_CD_1 = 0X4101,
    PSA_VAN_CD_CHANGER_CD_2 = 0X4102,
    PSA_VAN_CD_CHANGER_CD_3 = 0X4103,
    PSA_VAN_CD_CHANGER_CD_4 = 0X4104,
    PSA_VAN_CD_CHANGER_CD_5 = 0X4105,
    PSA_VAN_CD_CHANGER_CD_6 = 0X4106,
    PSA_VAN_CD_CHANGER_PREVIOUS_CD = 0X41FE,
    PSA_VAN_CD_CHANGER_NEXT_CD = 0X41FF,
};

struct psa_van_cd_changer_command_struct
{
    uint16_t command; // Big endian
};

#endif

#ifndef PSA_VAN_RADIO_INFO_STRUCT_H
#define PSA_VAN_RADIO_INFO_STRUCT_H

#include <stdint.h>

// Radio audio settings 0x4D4 PSA_VAN_IDEN_AUDIO_SETTINGS

const uint8_t PSA_RADIO_INFO_SOURCE_RADIO = 0x51;
const uint8_t PSA_RADIO_INFO_SOURCE_CD = 0x52;

struct psa_radio_settings_byte_1
{
    uint8_t stalk_mute : 1;
    uint8_t external_mute : 1;
    uint8_t auto_volume : 1;
    uint8_t : 1;
    uint8_t loudness_on : 1;
```

```
    uint8_t audio_properties_menu_open : 1;
    uint8_t : 2;
} __attribute__((packed));

struct psa_radio_settings_byte_2
{
    uint8_t power_on : 1;
    uint8_t : 7;
} __attribute__((packed));

struct psa_radio_settings_byte_4
{
    uint8_t source : 4;
    uint8_t : 1;
    uint8_t tape_present : 1;
    uint8_t cd_present : 1;
    uint8_t : 1;
} __attribute__((packed));

struct psa_radio_settings_values
{
    uint8_t value : 7; // Add 0x3f to get actual value
    uint8_t updating : 1;
} __attribute__((packed));

/**
 * @brief Radio info struct. Ident 0x4D4
 *
 */
struct psa_van_radio_settings
{
    uint8_t header;
    struct psa_radio_settings_byte_1 audio_properties;
    struct psa_radio_settings_byte_2 power;
```

```
    uint8_t field3;
    struct psa_radio_settings_byte_4 source;
    struct psa_radio_settings_values volume;
    struct psa_radio_settings_values balance;
    struct psa_radio_settings_values fader;
    struct psa_radio_settings_values bass;
    struct psa_radio_settings_values treble;
    uint8_t footer;
} __attribute__((packed));

int a = sizeof(struct psa_van_radio_settings);

#endif

#ifndef PSA_VAN_RPM_STRUCT_H
#define PSA_VAN_RPM_STRUCT_H

#include <stdint.h>

/**
 * @brief VAN rpm and speed data struct. Iden 0x824
 *
 */
struct psa_van_rpm_speed_struct
{

    uint16_t rpm;          // RPM in big endian. Divide by 8 to get actual rpm;
    uint16_t speed;        // Speed in big endian. Divide by 100 to get actual speed;
    uint16_t distance;     // Packet sequence in big endian. Increments with each passed decimeter
    uint8_t packet_changed; // Increments each time information significantly changes
} __attribute__((packed));

#endif
```

```
#ifndef VAN_CDC_EMU_H
#define VAN_CDC_EMU_H

#include <stdint.h>

enum van_cd_changer_status
{
   VAN_CDC_OFF = 0x41,
   VAN_CDC_ON_PAUSED = 0xC1,
   VAN_CDC_ON_BUSY = 0xD3,
   VAN_CDC_ON_PLAYING = 0xC3,
   VAN_CDC_ON_FAST_FORWARD = 0xC4,
   VAN_CDC_ON_FAST_REWIND = 0xC5,
};

enum van_cd_changer_cd_presence
{
   VAN_CDC_CD_PRESENT = 0x16,
   VAN_CDC_CD_NOT_PRESENT = 0x06,
};

struct van_cd_changer_packet
{
   uint8_t header;      // 0x80 to 0x87. Same as footer
   uint8_t shuffle;     // 1 if tracks are shuffled
   uint8_t status;      // enum van_cd_changer_status
   uint8_t cd_present;  // enum van_cd_changer_cd_presence
   uint8_t minutes;     // Track minute progress in bcd
   uint8_t seconds;     // Track seconds progress in the current minute in bcd
   uint8_t track_num;   // Number of the current track in bcd
   uint8_t cd_num;      // Number of the current CD in bcd
   uint8_t total_tracks; // Total number of tracks in bcd
   uint8_t unknown;
   uint8_t cd_flag; // bitwise representation of present CDs
```

```
    uint8_t footer;
} __attribute__((packed));

union van_cd_changer_union
{
    struct van_cd_changer_packet packet;
    uint8_t buffer[sizeof(struct van_cd_changer_packet)];
};

#endif // VAN_CDC_EMU_H

#ifndef PSA_VAN_DASHBOARD_STRUCT_H
#define PSA_VAN_DASHBOARD_STRUCT_H

#include <stdint.h>

/**
 * @brief Dashboard struct. Ident 0x8A4. PSA_VAN_IDEN_ENGINE
 *
 */
struct psa_van_dashboard
{
    uint8_t brightness : 4; // 0 - 15
    uint8_t wiper : 1;
    uint8_t : 2;
    uint8_t is_backlight_off : 1; // 1 when light position is off

    uint8_t accesories_on : 1;   // Key in ACC
    uint8_t ignition_on : 1;     // Key in IGN
    uint8_t engine_running : 1;  // Engine on
    uint8_t door_open : 1;       // A door is open
    uint8_t economy_mode : 1;    // Engine in economy mode
    uint8_t reverse_gear : 1;    // Gearbox in reverse
    uint8_t trailer_present : 1; // A trailer is hooked up
```

```
    uint8_t : 1;

    uint8_t water_temp;    // Subtract by 40
    uint32_t mileage : 24; // Should be 3 bytes in big endian format, divide by 10 to get value
    uint8_t external_temp; // 0xff if sensor not present (Subtract 0x50 and divide by 2 to get celcius value)

} __attribute__((packed));

/**
 * @brief Instrument cluster struct. Ident 0x4FC. Len 11. PSA_VAN_IDEN_LIGHTS_STATUS
 *
 */
struct psa_van_instrument_cluster_short
{
    uint8_t cluster_lights_1;
    uint8_t door_led;
    uint16_t km_to_service;
    uint8_t auto_gearbox_gear;
    uint8_t light_indicators;
    uint8_t oil_temp;
    uint8_t fuel_level;
    uint8_t oil_level; // 0 - 254
    uint8_t unknown_1;
    uint8_t lpg_level;
} __attribute__((packed));

/**
 * @brief Instrument cluster struct. Ident 0x4FC. Len 14. PSA_VAN_IDEN_LIGHTS_STATUS
 *
 */
struct psa_van_instrument_cluster_long
{
    uint8_t cluster_lights_1;
    uint8_t door_led;
```

```
    uint16_t km_to_service;
    uint8_t auto_gearbox_gear;
    uint8_t light_indicators;
    uint8_t oil_temp;
    uint8_t fuel_level;
    uint8_t oil_level; // 0 - 254
    uint8_t unknown_1;
    uint8_t lpg_level;
    uint8_t cruise_control;
    uint8_t cruise_control_speed;
    uint8_t unknown_2;
} __attribute__((packed));

#endif

#ifndef VAN_RADIO_STRUCTS_H
#define VAN_RADIO_STRUCTS_H

#include <stdint.h>

// Packet iden 0x554

enum psa_van_radio_band
{
    PSA_VAN_BAND_NONE = 0,
    PSA_VAN_BAND_FM1,
    PSA_VAN_BAND_FM2,
    PSA_VAN_BAND_FM3,
    PSA_VAN_BAND_FMAST,
    PSA_VAN_BAND_AM,
    PSA_VAN_BAND_PTY_SELECT,
};

struct psa_van_radio_freq_info_scan_info
```

```
{
   uint8_t : 3;
   uint8_t manual_scan_in_progress : 1;
   uint8_t freq_scan_is_running : 1;
   uint8_t : 2;
   uint8_t freq_scan_direction : 1;
} __attribute__((packed));

struct psa_van_radio_freq_info_ta_rds_flags
{
   uint8_t ta_active : 1;
   uint8_t rds_active : 1;
   uint8_t : 3;
   uint8_t rds_data_present : 1;
   uint8_t ta_data_present : 1;
   uint8_t : 1;
} __attribute__((packed));

// Len 22
struct psa_van_radio_freq_info
{
   uint8_t header;                              // 0
   uint8_t data_type;                           // Should be 0xD1 for frequency info // 1
   uint8_t band : 3;                            // 2
   uint8_t memory_position : 5;                 // 2
   struct psa_van_radio_freq_info_scan_info scan_info;      // 3
   uint8_t frequency[2];                        // 4,5
   uint8_t signal_info;                         // 6 AND with 0x0F to get signal strength, not sure if accurate
   struct psa_van_radio_freq_info_ta_rds_flags rds_ta_flags; // 7
   uint8_t unknown2[4];                         // 8,9,10,11
   uint8_t station[9];                          // 12,13,14,15,16,17,18,19,20
   uint8_t footer;
} __attribute__((packed));
```

```
// Len 10, didn't appear in my car
struct psa_van_radio_cd_info_len_10
{
   uint8_t header;
   uint8_t data_type; // Should be 0xD6 for CD track info
   uint8_t shuffle;
   uint8_t status; // loading, error, etc... to research
   uint8_t unknown1;
   uint8_t minutes;
   uint8_t seconds;
   uint8_t current_track;
   uint8_t total_tracks;
   uint8_t footer;
} __attribute__((packed));

// Len 19
struct psa_van_radio_cd_info_len_19
{
   uint8_t header;
   uint8_t data_type; // Should be 0xD6 for CD track info
   uint8_t shuffle;
   uint8_t status; // loading, error, etc... to research
   uint8_t unknown1;
   uint8_t minutes;
   uint8_t seconds;
   uint8_t current_track;
   uint8_t total_tracks;
   uint8_t total_minutes;
   uint8_t total_seconds;
   uint8_t unknown2[7];
   uint8_t footer;
} __attribute__((packed));

// Len 12, didn't appear in my car
```

```
struct psa_van_radio_preset_info
{
   uint8_t header;
   uint8_t data_type;      // Should be D3 for preset info
   uint8_t position : 4;   // Preset position
   uint8_t : 4;            // Unknown
   uint8_t station_name[8]; // ASCII, not NULL-terminated
   uint8_t footer;
} __attribute__((packed));

// int a = sizeof(struct psa_van_radio_preset_info);
#endif

#ifndef PSA_VAN_TRIP_COMPUTER_STRUCT_H
#define PSA_VAN_TRIP_COMPUTER_STRUCT_H

#include <stdint.h>

// IDEN 0x564 or PSA_VAN_IDEN_CAR_STATUS1

struct psa_van_trip_computer_door_struct
{
   uint8_t fuel_flap : 1;   // bit 0
   uint8_t sunroof : 1;     // bit 1
   uint8_t hood : 1;        // bit 2
   uint8_t boot_lid : 1;    // bit 3
   uint8_t rear_left : 1;   // bit 4
   uint8_t rear_right : 1;  // bit 5
   uint8_t front_left : 1;  // bit 6
   uint8_t front_right : 1; // bit 7
} __attribute__((packed));

union psa_van_trip_computer_door_union
{
```

```
    struct psa_van_trip_computer_door_struct unpacked;
    uint8_t packed;
};

struct psa_van_trip_computer_button_struct_maybe
{
    uint8_t trip_button : 1;
    uint8_t : 7;
} __attribute__((packed));

/**
 * @brief Trip computer info. Also includes door info. Iden 0x564
 *
 */
struct psa_van_trip_computer
{

    uint8_t header;
    uint8_t unknown1[6];

    union psa_van_trip_computer_door_union door_status;

    uint8_t unknown2[2];

    struct psa_van_trip_computer_button_struct_maybe trip_button;

    uint8_t trip_1_speed;
    uint8_t trip_2_speed;
    uint8_t unknown3;

    uint16_t trip_1_distance;         // I'm guessing in KM
    uint16_t trip_1_fuel_consumption;  // Says divide by 10 (probably in big endian)
    uint16_t trip_2_distance;         // I'm guessing in KM
```

```
    uint16_t trip_2_fuel_consumption;  // Says divide by 10 (probably in big endian)
    uint16_t current_fuel_consumption; // Says divide by 10 (probably in big endian)
    uint16_t distance_to_empty;        // I'm guessing in KM
    uint8_t footer;

} __attribute__((packed));

#endif
```